SAA-C03 Topic Guides — Study Index

37 self-contained topic guides for the AWS Solutions Architect Associate (SAA-C03) exam. Each file follows the same pattern:

1. **The idea** — the service explained from zero with a real-world analogy
2. **Core concepts** — the exam-tested facts, numbers, tables, and explicit "THE trap" callouts
3. **Question patterns** — realistic exam scenarios with answers and the signal keywords that crack them
4. **Pocket card** — a keyword → answer table for rapid review

Suggested reading order (grouped by exam weight)

🏗️ Core infrastructure (heaviest-tested — do these first)

#	Guide	Owns the questions about
06	VPC	subnets, NAT, SG vs NACL, endpoints, peering/TGW/PrivateLink
02	S3	storage classes, encryption, Object Lock
03	EC2	instance families, pricing options, placement groups
04	EBS, EFS & Instance Store	volume types, shared storage, ephemeral
05	ELB & Auto Scaling	ALB/NLB/GLB, scaling policies
01	IAM	roles, policies, SCPs, STS

🗄️ Databases

#	Guide	
09	RDS & Aurora	Multi-AZ vs Read Replicas (the #1 DB question)
10	DynamoDB	DAX, Global Tables, Streams, GSI/LSI
11	ElastiCache	Redis vs Memcached, caching strategies
12	Other Databases	the keyword zoo: Redshift, Athena, Neptune, QLDB…

⚡ Serverless & application integration

#	Guide	
13	Lambda	15-min limit, concurrency, VPC access
14	ECS, EKS & Fargate	containers, the two roles
15	Elastic Beanstalk	deployment policies
16	API Gateway	auth trio, 29s timeout, throttling
17	SQS, SNS & Kinesis	decoupling — the heaviest single question source
18	EventBridge & Step Functions	events, workflows

🌍 Networking & edge

#	Guide	
07	Route 53	routing policies, Alias vs CNAME
08	CloudFront & Global Accelerator	CDN vs backbone
32	Direct Connect & VPN	hybrid connectivity
33	Transit Gateway	hub-and-spoke

🔐 Security & identity

#	Guide	
20	KMS & CloudHSM	encryption keys
21	Secrets Manager & Parameter Store	rotation
22	WAF, Shield & Security Services	+ GuardDuty/Macie/Inspector zoo
23	Cognito	User Pools vs Identity Pools
24	Directory Services	Managed AD / Connector / Simple

📊 Monitoring, management & governance

#	Guide	
19	CloudWatch, CloudTrail & Config	who watches what
28	CloudFormation	IaC
29	Organizations & Control Tower	SCPs, multi-account
30	Service Catalog & Trusted Advisor	governance tools
31	Cost Management	Explorer/Budgets/CUR

📦 Hybrid storage & migration

#	Guide	
25	Storage Gateway, DataSync & Transfer Family	hybrid bridges
26	FSx Family	Windows/Lustre/ONTAP/OpenZFS
27	Snow Family	offline transfer

🎯 Final review (read last, and on exam morning)

#	Guide	
34	Well-Architected Framework	the 6 pillars as question framing
35	Disaster Recovery	RPO/RTO, the 4 strategies
37	Gap-Fill Services	MQ, ACM, Textract, SSM… the 1-question services
36	Exam Traps & Key Patterns	⭐ the greatest-hits trap list + exam technique

How to use these guides

1. **First pass:** read in the suggested order above — the analogies do the heavy lifting.
2. **Second pass:** cover the pocket cards' right columns and quiz yourself from the keywords.
3. **After practice tests:** for every miss, come back to that topic's "Question patterns" section — the crack explanations show which signal words you skipped.
4. **Exam morning:** skim guide 36 (traps) and each guide's pocket card only. Nothing new on exam day.

↑ Back to index

Section 1: IAM — Identity and Access Management

The idea

Every single thing that happens in AWS — launching a server, reading a file, deleting a database — starts with the same two questions: **who are you?** and **are you allowed to do that?** IAM (Identity and Access Management) is the service that answers both. It's the front door, the ID checker, and the rulebook, all in one — and it's **free** and **global** (not tied to any region).

Here's the analogy to carry through this whole section: **AWS is a giant office building.** IAM hands out the badges. A **user** is a permanent employee badge with your name on it. A **group** is a department — everyone in "Developers" gets whatever the department is allowed. A **role** is a **temporary costume**: a visitor's vest that anyone (a person, a server, another company) can put on for a while, gain its powers, and then take off. And **policies** are the written rules pinned to the wall saying which badges open which doors.

That "temporary costume" idea is the single most exam-tested concept in IAM, so let's start there.

Users, groups, roles

Identity	What it is	When to use
User	Permanent identity for one human (or one legacy app), with a password and/or long-lived access keys	A specific person who needs AWS access
Group	A bucket of users that share policies. Groups contain **only users** — never other groups, never roles	Manage permissions by team, not per person
Role	An identity with permissions but **no password and no permanent keys**. It's *assumed* — put on like a costume — and hands out **temporary credentials**	Services, cross-account access, federated humans

The golden rule: never put long-lived access keys on compute. If an EC2 instance needs to read from S3, you do NOT create a user, generate access keys, and paste them into a config file on the box. Keys on a server can leak, never rotate themselves, and show up in every "what's wrong with this architecture?" exam question. Instead:

- **EC2 → attach an instance role** (delivered via an *instance profile*). The instance fetches auto-rotating temporary credentials from its metadata — no keys ever touch the disk.
- **Lambda → execution role.** Same idea: the function assumes a role every time it runs.
- **ECS tasks → task role.** Same pattern again.

Two defaults worth knowing cold:

- A **brand-new IAM user has NO permissions** (implicit deny) until a policy is attached — "*new user can't launch EC2*" → attach a policy/add to a group.
- **Access keys** (access key ID + secret) are the **long-term credentials** of a user (or root) used to sign **CLI/SDK/API** calls. **Attaching a policy does NOT create access keys** — they're generated separately. Rule of thumb: **groups manage humans' permissions, roles manage services'**.

THE trap: any answer choice that says "*store access keys on the instance* / *in the AMI* / *in environment variables* / *in the code*" is wrong. The right answer is always **the role**. If the exam says "credentials found hardcoded in an application on EC2" — the fix is *attach an IAM role to the instance*.

Policy JSON anatomy

A policy is a JSON document. You don't need to write one on the exam, but you must be able to *read* one. Four load-bearing parts:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowReadReports",
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": "arn:aws:s3:::finance-reports/*",
    "Condition": {
      "Bool": { "aws:MultiFactorAuthPresent": "true" }
    }
  }]
}
```

- **Effect** — `Allow` or `Deny`. That's it, two values.
- **Action** — which API calls (`s3:GetObject`, `ec2:*`...). Service prefix + verb.
- **Resource** — which things, named by **ARN** (Amazon Resource Name — AWS's globally unique ID string for every object it manages).
- **Condition** — optional "only if" clauses: only with MFA, only from this IP range, only over HTTPS.

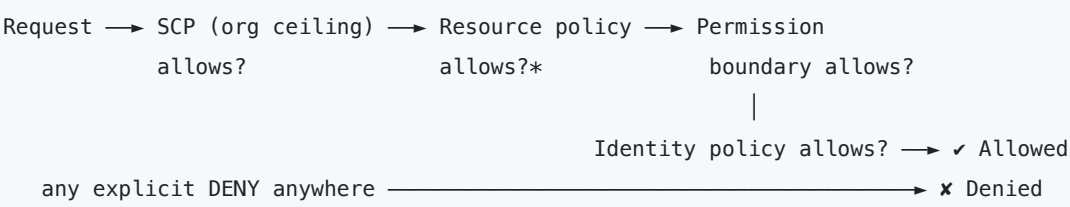
Resource-based policies (we'll meet them next) add a **Principal** field — *who* the statement applies to — because the policy isn't attached to an identity, so it has to name one.

How AWS decides yes or no

Memorize this evaluation logic — it's worth several exam questions:

1. **Default is implicit deny.** Nothing is allowed until something allows it.
2. An **Allow** in an applicable policy opens the door.
3. An **explicit Deny ALWAYS wins.** Over any Allow, anywhere, no exceptions, full stop.

And for a request to succeed, it must pass through **every applicable gate**:



(Fine print you don't need for the exam: within the same account, a resource-policy Allow alone can be enough. The pattern you DO need: an SCP or explicit Deny blocks everything downstream.)

THE trap: "*A user's policy clearly says Allow, but the action fails.*" The answer is always one of: **an explicit Deny somewhere**, **an SCP on the account** (common when the question mentions AWS Organizations), or **a permission boundary** clipping the identity policy. Allow + Deny = Deny. Every time.

The policy types table

Policy type	Attached to	What it does
Identity-based	User, group, or role	The everyday "what can this identity do"
Resource-based	The resource itself (S3 bucket policy, SQS queue policy...)	"Who can touch *me*" — has a `Principal`, enables cross-account grants
Permission boundary	A user or role	A **maximum ceiling** for that one identity. Grants nothing itself — effective perms = identity policy ∩ boundary
SCP (Service Control Policy)	Accounts/OUs in **AWS Organizations**	An **org-wide ceiling**. Grants nothing; caps everyone in the account — **including the account's root user**
Session policy	Passed when assuming a role	Shrinks that one session's permissions even further
ACL (Access Control List)	S3 buckets/objects	**Legacy** S3-only mechanism. Can't use JSON conditions and **cannot grant to users in its own account** — AWS says avoid; use bucket policies. Don't confuse with **Network ACLs** (a VPC subnet firewall — totally unrelated)

Ceilings vs grants, one line: **boundary = ceiling for one identity; SCP = ceiling for a whole account; neither grants anything by itself.**

Exam scenario for boundaries: "*Developers may create their own IAM roles, but must never be able to create roles more powerful than X*" → **permission boundary** (require it on every role they create).

Execution role vs resource-based policy (Lambda, and the same logic everywhere):

Question the policy answers	Policy	Example
"Who can invoke/touch ME?"	**Resource-based policy** on the function	Allow S3 / API Gateway / another account to invoke the Lambda
"What can I access?"	**Execution role** (identity policy)	Lambda reading S3, writing DynamoDB, calling KMS/SQS

THE trap: when Lambda calls another service, AWS evaluates the **execution role**, not the function ARN. **KMS permissions follow the principal** — the key policy/grant must name the **execution role**, not the Lambda function. Likewise for ECS tasks, grant access through the **task role**, not a resource policy naming the task.

STS and roles: how the costume gets put on

STS (Security Token Service) is the coat-check counter that hands out costumes. When anything calls `sts:AssumeRole`, STS returns **temporary credentials** — an access key, secret key, and session token — valid for a limited time (15 minutes to 12 hours, **default 1 hour**), after which they expire on their own. Temporary, auto-expiring, nothing to rotate or leak long-term: that's why roles beat users for anything automated.

Every role has TWO policies, and the exam loves this:

- **Permissions policy** — what the costume lets you do.
- **Trust policy** — **who is allowed to put the costume on.** It's a resource-based policy on the role itself, naming the trusted principal (a service like `ec2.amazonaws.com`, another account, a SAML provider).

THE trap: "*Role has the right permissions but the user/service can't assume it*" → check the **trust policy**. Permissions say what the role can do; trust says who can wear it.

Cross-account access

Company A's auditors need to read Company B's S3 bucket. The canonical recipe:

1. **Account B (the target)** creates a role with the needed permissions.
2. B sets the role's **trust policy** to trust Account A (and can require MFA or an `ExternalId` — an agreed secret string used with third parties to block the "confused deputy" problem).
3. **Account A's users** get permission to call `sts:AssumeRole` on that role's ARN.
4. They assume it, get temporary credentials, do the work, credentials expire.

No shared passwords, no duplicated users, no keys emailed around. "*Give another AWS account access*" → **cross-account role + trust policy**, essentially always. (Fine for a handful of accounts; for **dozens of accounts** and a workforce, move up to **Organizations + IAM Identity Center**.)

Cross-account S3 specifically: a **bucket policy** can grant other accounts (or specific IAM principals) access to all or some objects; an **ACL** can grant only to **whole AWS accounts** (not to individual users) and is legacy. Prefer the bucket policy or a cross-account role.

IAM Identity Center (the artist formerly known as AWS SSO)

One human, twelve AWS accounts, twelve passwords? No. **IAM Identity Center** gives your workforce **single sign-on (SSO): one login, a portal, and access to multiple AWS accounts** (plus business apps) — under the hood it just assumes roles in each account for you.

It also **federates**: connect your existing corporate identity provider — **Okta, Azure AD / Microsoft Entra ID, or on-prem Active Directory** — via **SAML 2.0** (Security Assertion Markup Language, the standard XML handshake that lets one system vouch "yes, this is Alice" to another). Employees keep their corporate password; AWS never stores it; someone leaves the company, disable them once in the IdP and every AWS door closes.

Signal decoding: "*employees already have Active Directory / Okta credentials and shouldn't get separate IAM users*" or "*single sign-on across many AWS accounts*" → **IAM Identity Center**. (For federating *app customers* — sign in with Google/Facebook — that's **Amazon Cognito**, a different topic.)

The federation flavors

Flavor	How it works	Exam signal
SAML 2.0 federation	Corporate IdP that speaks SAML 2.0 (e.g., **Microsoft AD FS**, Okta, Entra ID) asserts identity → role assumed via `AssumeRoleWithSAML` / Identity Center	"Corporate login, IdP is SAML-compatible"
Custom identity broker	Your own app authenticates users against the **non-SAML** store (e.g., on-prem **LDAP**), then calls **STS** (`AssumeRole` / `GetFederationToken`) to hand out temporary credentials	"Identity store is **not SAML 2.0-compatible**"
Web identity federation	Login with Amazon/Facebook/Google or any **OIDC** IdP → temporary creds (today: via **Cognito**) — no custom sign-in code	"Mobile/web app users, social login"
AD Connector + IAM roles	Existing on-prem AD stays put; **assign IAM roles to AD users/groups** for console access	"Use existing AD accounts for AWS console"

Memory hook: **SAML available → Identity Center; no SAML → identity broker + STS.** All of these are **enterprise identity federation** — users authenticate in their own network and reach AWS **without new IAM users**. IAM itself is **not** for managing thousands of mobile-app users — that's Cognito.

IAM database authentication

RDS MySQL/PostgreSQL (and Aurora) can authenticate DB logins with **IAM instead of a password**: the app (e.g., EC2 using its **instance profile**) generates a short-lived **auth token** (15 min), connects over **SSL** (traffic encrypted), and access is **managed centrally in IAM**. MySQL users map to it via the **`AWSAuthenticationPlugin`**. "*No DB passwords, centrally managed access, EC2 role connects to RDS*" → **IAM DB authentication**.

IAM certificate store (legacy)

IAM can hold **SSL/TLS server certificates** and use them in any Region — but **only certificates from an external CA** (it can't issue them) and you must renew manually. It's the legacy fallback for Regions/cases without **ACM**; today **use ACM** (free, auto-renewing).

MFA and the root account

MFA (multi-factor authentication) = password *plus* a second factor (authenticator app, hardware key). Policies can demand it via the condition key **`aws:MultiFactorAuthPresent`** — e.g., deny `s3:DeleteObject` unless it's `true`. "*Require MFA for destructive/sensitive actions*" → that condition key.

The **root account** (the email you created the account with) can do absolutely everything and can't be restricted by IAM policies (only an SCP can cap it, and never in the org's management account). Best practice, exam-tested verbatim:

- **Enable MFA on root** immediately.

- **Delete root access keys** (never create them).
- **Don't use root for daily work** — create an admin IAM identity instead.
- Root only for the handful of root-only tasks (closing the account, changing support plans...).

Root-only tasks (the exam lists these as "requires the root user"): change **account name, root email or root password**; change the **support plan**; **close the account**; **enable MFA Delete on an S3 bucket**; create **CloudFront key pairs** (legacy signed-URL keys); **sign up for GovCloud**.

Least privilege: start with the minimum permissions and add only as needed. And remember IAM is **global** — users, groups, roles, and policies exist once and apply in every Region.

Two quick tools worth a flashcard: **IAM Access Analyzer** finds resources shared with outside entities; **credentials report / access advisor** show stale users and unused permissions (the "least privilege cleanup" answers). The **IAM credential report** is an **account-wide CSV of every IAM user and their credential status** (password age/last used, access keys rotated/last used, MFA enabled) — the audit answer, not a CloudTrail or Config job.

Question patterns

> "*An application on EC2 needs to write to DynamoDB. What's the MOST secure way to provide credentials?*" → **IAM role attached to the instance (instance profile)** — never access keys on the box.

> "*A Lambda function must read from an S3 bucket. How should it authenticate?*" → **Lambda execution role** with S3 read permissions — same costume pattern, serverless flavor.

> "*Users in Account A need temporary access to resources in Account B.*" → **Create a role in Account B with a trust policy trusting Account A; users AssumeRole via STS** — "temporary" + "another account" = cross-account role.

> "*A user's identity policy allows s3:PutObject, but uploads fail with Access Denied.*" → **Look for an explicit Deny — bucket policy, SCP, or permission boundary** — explicit Deny always beats Allow.

> "*An action works in a standalone account but fails in an account inside AWS Organizations.*" → **A Service Control Policy is blocking it** — "Organizations" is the tell.

> "*Developers may create IAM roles for their apps, but security wants a guarantee those roles can never exceed a defined permission set.*" → **Permission boundaries** — "maximum permissions" / "cannot exceed" is the signal.

> *"5,000 employees with existing Active Directory logins need access to multiple AWS accounts without creating IAM users."* → **IAM Identity Center with SAML federation to AD** — "existing corporate directory" + "multiple accounts" + "no IAM users".

> *"An application needs short-lived credentials that expire automatically."* → **STS AssumeRole** — "temporary/short-lived credentials" is literally STS's product.

> *"Require that users can only terminate EC2 instances if they've signed in with MFA."* → **Policy Condition with** `aws:MultiFactorAuthPresent` — sensitive action + MFA = that condition key.

> *"A new AWS account was just created. What should be done FIRST to secure it?"* → **Enable MFA on the root user, delete/avoid root access keys, create an admin user for daily work.**

> *"Enable MFA Delete on an S3 bucket — who can do it?"* → **Only the root user** (one of the root-only tasks).

> *"Company's on-prem LDAP directory is not SAML 2.0-compatible; employees need AWS access without IAM users."* → **Custom identity broker that authenticates against LDAP and calls STS** (no SAML = broker).

> *"A Lambda function can't decrypt with a KMS key, even though the key policy allows the Lambda function ARN."* → **Grant the Lambda execution role in the key policy** (KMS permissions follow the principal).

> *"Allow an S3 bucket in another account to trigger a Lambda function."* → **Lambda resource-based policy** ("who can invoke me").

> *"EC2 app must connect to RDS MySQL without storing a database password, with access managed centrally."* → **IAM database authentication** (instance profile → auth token over SSL).

> *"Auditor needs a list of all IAM users with password, access key, and MFA status."* → **IAM credential report**.

> *"A newly created IAM user can't perform any EC2 actions."* → **New users have no permissions by default — attach a policy / add to a group** (and attaching a policy doesn't create access keys).

Pocket card

Keyword in the question	Answer
EC2/app needs AWS access, "most secure"	Instance role (instance profile), never keys
Lambda needs permissions	Execution role
Hardcoded / stored access keys	Wrong — replace with a role
Temporary / short-lived credentials	STS AssumeRole
Access across AWS accounts	Role in target account + trust policy
Third party assumes your role safely	ExternalId in trust policy
Role exists but "can't be assumed"	Fix the trust policy
Allow exists but action denied	Explicit Deny / SCP / permission boundary
Org-wide restriction, applies even to root	SCP (AWS Organizations)
Cap max permissions of ONE user/role	Permission boundary
Policy on the resource, has a Principal	Resource-based policy (e.g., bucket policy)
Legacy S3 grant mechanism, avoid	S3 ACL (≠ Network ACL)
SSO, one login → many AWS accounts	IAM Identity Center
Federate Okta / Azure AD / corporate AD	IAM Identity Center + SAML 2.0
Federate app customers (Google/Facebook login)	Cognito
Require MFA for an action	Condition: aws:MultiFactorAuthPresent
Root account	MFA on, no access keys, don't use daily
Who is my resource shared with externally?	IAM Access Analyzer
Find unused users / stale permissions	Credentials report / access advisor
List all users + password/key/MFA status	IAM credential report
MFA Delete, close account, change support plan, GovCloud	Root user only
Corporate IdP is SAML 2.0 (AD FS)	SAML federation / Identity Center
Identity store NOT SAML-compatible (LDAP)	Custom identity broker + STS
"Who can invoke my Lambda?"	Resource-based policy
"What can my Lambda access?" / KMS grant target	Execution role (permissions follow the principal)
DB login without password, MySQL/PostgreSQL	IAM database authentication
SSL certs in IAM	Legacy IAM certificate store — use ACM
Cross-account S3 to specific users	Bucket policy (ACL = whole accounts only)

IAM is the "who may act" layer under everything else — next up, S3, where you'll see these same policies show up on the resource side as bucket policies.

↑ Back to index

Section 2: S3 — Simple Storage Service

The idea

S3 is where AWS keeps *stuff*. Not a hard drive, not a filesystem — an **object store**. Think of it as an **infinite valet parking garage for files**: you hand over your file (the "object"), you get a claim ticket (the "key" — its full name, like `photos/2026/cat.jpg`), and whenever you come back with the ticket, you get the whole file back. You can't pop the hood and change the engine while it's parked — **you can't edit a byte in the middle of an object; you replace the whole object**. That's the core difference from a disk (that's EBS) or a shared network filesystem (that's EFS).

Objects live in **buckets**. **Bucket names are globally unique across all of AWS** (no two customers anywhere can have the same bucket name), but **the bucket itself lives in one region** you choose. Capacity is unlimited; a single object maxes out at **5 TB**. For big uploads, **multipart upload** splits the file into parallel chunks — **recommended over 100 MB, REQUIRED over 5 GB** (a single PUT caps at 5 GB). S3 promises **eleven 9s of durability** (99.999999999% — your data essentially never evaporates) across every storage class; what varies between classes is cost, availability, and how fast you can get things back.

Vocabulary the exam assumes: **bucket** (globally unique name, one shared namespace across all accounts) → **object key** (the unique name inside the bucket) → **object metadata** (name-value pairs describing the object) → **object tags** (key-value pairs to categorize objects, usable in lifecycle and IAM rules).

Two behaviors worth knowing:

- **Consistency:** S3 has been **strongly read-after-write consistent** for all PUTs, overwrites, deletes, and LIST operations **since December 2020**. Read it right after writing it and you get the new data. (Older study material that describes "eventual consistency": a new object missing from LIST, or an overwrite returning stale data, is **outdated**. Don't pick it.)
- **Concurrent writes:** two simultaneous PUTs to the **same key** → the one with the **latest timestamp wins**. S3 has **no built-in locking for concurrent updates** (Object Lock is WORM retention, not a mutex). If that matters, build the locking into your application.
- **Requester Pays** bucket: the **requester** pays for requests and data download; the **bucket owner still pays for storage**.

That "how fast can I get it back and how much does it cost" question is the heart of the exam's S3 questions, so let's build the machine that answers all of them.

Storage classes and THE 3-filter algorithm

The classes, top shelf to deep freeze:

Class	Retrieval	Min storage duration	Use when
Standard	milliseconds	none	Frequently accessed, hot data

Class	Retrieval	Min storage duration	Use when
Intelligent-Tiering	milliseconds	none	**Unknown/unpredictable** access patterns
Standard-IA	milliseconds	**30 days**	Infrequent access, needs instant retrieval
One Zone-IA	milliseconds	30 days	Infrequent AND **re-creatable** (one AZ only!)
Glacier Instant Retrieval	**milliseconds**	**90 days**	Archive touched ~quarterly, needed instantly
Glacier Flexible Retrieval	Expedited **1–5 min** / Standard **3–5 hr** / Bulk **5–12 hr**	90 days	Archive, minutes-to-hours retrieval OK
Glacier Deep Archive	Standard **12 hr** / Bulk **48 hr**	**180 days**	Cheapest. Retrieval is HOURS — **can NEVER beat 12 hr**

("IA" = Infrequent Access; "AZ" = Availability Zone, one physical data-center cluster.)

The fine print for the four millisecond-access classes (all have **11 9s durability** and **millisecond first-byte latency**):

	Standard	Standard-IA	One Zone-IA	Intelligent-Tiering
Availability (designed)	**99.99%**	99.9%	**99.5%**	99.9%
Availability SLA	99.9%	99%	99%	99%
AZs	≥ 3	≥ 3	**only 1**	≥ 3
Minimum billable object size	none	**128 KB**	**128 KB**	none (but < 128 KB never tiered)
Retrieval fee	none	**per GB retrieved**	**per GB retrieved**	none

Standard-IA costs more than One Zone-IA because it stores data redundantly across AZs. **IA is not for archiving.** Archives belong in the Glacier family.

Hot / warm / cold vocabulary: hot storage = frequently accessed data, warm = less frequently accessed, cold = rarely accessed. **The colder the data, the cheaper it is to store and the costlier it is to access.**

Every "most cost-effective storage class" question yields to the same three filters:

Filter 1 — access frequency picks the family. Daily/constant → Standard family. Monthly-ish → IA. A few times a year → Glacier family. Unknown → Intelligent-Tiering.

Filter 2 — the retrieval-time requirement eliminates classes. "*Needed immediately / within milliseconds*" kills Flexible and Deep Archive (Glacier **Instant** survives — it retrieves in milliseconds). "*Within minutes/hours*" — Flexible is fine (Expedited 1–5 min, Standard 3–5 hr, Bulk 5–12 hr). "*Within 12 hours or more*" — now Deep Archive is on the table. Burn this in: **Deep Archive can NEVER deliver faster than 12 hours.** Any deadline under 12 hours disqualifies it, no matter how cheap it is.

Filter 3 — minimum storage duration. IA classes bill a minimum **30 days**, Glacier Instant and Flexible **90 days**, Deep Archive **180 days** — delete early and you pay for the minimum anyway. **Data that lives (or is only kept) under 30 days → plain Standard wins**, because every "cheaper" class charges you 30+ days regardless.

Cheapest class that survives all three filters wins. Worked example: "*accessed twice a year, auditors need it within 10 minutes, most cost-effective*" → twice a year = Glacier family; "10 minutes" kills Deep Archive (12 hr floor) but Flexible's Expedited (1–5 min) passes; Instant also passes but costs more → **Glacier Flexible Retrieval**.

Glacier extras that show up as answer choices:

- **Provisioned capacity**: makes sure **Expedited** retrieval capacity is there when you need it. Each unit guarantees **at least 3 Expedited retrievals every 5 minutes** and **up to 150 MB/s** of retrieval throughput. Signal: "*Expedited retrievals must ALWAYS be available / never be rejected.*"
- **Glacier Select**: run simple SQL filters **directly on archived data** without restoring the whole archive first.
- **THE trap: Athena can't query objects sitting in Glacier Flexible Retrieval or Deep Archive.** You have to restore them first. If the data must stay queryable by Athena, keep it in **Standard-IA** (or Glacier Instant Retrieval) instead of archiving it.
- **Vault Lock** (for Glacier vaults): a vault can have one access policy plus one **Vault Lock policy**. A Vault Lock policy is an access policy you **lock**, and once locked it's immutable. Signal: "*enforce regulatory/compliance controls on the archive vault.*"

The two "special" classes and their exact signals

One Zone-IA stores data in a **single Availability Zone** — the AZ burns down, the data is gone. So it's ~20% cheaper and correct **ONLY when the data is re-creatable or a secondary copy**: thumbnails you can regenerate, a replica of on-prem data, secondary backups. The exam signal is the phrase **"can be regenerated / re-created / is a secondary copy."** No such phrase → do not pick One Zone.

Intelligent-Tiering watches each object and moves it between tiers automatically. Sounds like a free lunch; it isn't:

- There's a **monitoring fee per object** — millions of tiny objects = real money.
- **Objects under 128 KB are never tiered** (they just sit at Standard rates).
- An object pays **Standard price for ~30 days** of observed inactivity before IT moves it down.

So: **if you KNOW the pattern ("accessed heavily for 30 days, then rarely"), a lifecycle rule beats Intelligent-Tiering** — it's cheaper and deterministic. IT is correct **ONLY when the question says access is unknown, unpredictable, or changing**.

Lifecycle, versioning, replication

Lifecycle rules = the automated conveyor belt: *transition* actions move objects down the class ladder on a schedule ("after 30 days → Standard-IA, after 90 → Glacier, after 365 → delete"), *expiration* actions delete them. Also great for cleaning up old versions and **aborting incomplete multipart uploads** (which silently cost money forever otherwise).

THE lifecycle timing trap: objects must sit in Standard for **at least 30 days** before a lifecycle rule can move them to **Standard-IA or One Zone-IA**. A rule like "move to Standard-IA after 7 days" is invalid. Transitions straight to the **Glacier classes have no 30-day floor**, so "move to Glacier Flexible / Deep Archive after 7 days" (or even day 0) is fine. Remember the Glacier minimum-duration charges still apply after the move.

Versioning keeps every version of every object. Overwrite = new version on top; delete = a **delete marker** placed on top (the old versions are still there — remove the marker and the object "reappears"). It's the answer to "*protect against accidental overwrite/deletion.*" Once enabled, you can only suspend it, never fully un-enable. Add **MFA Delete** and permanently deleting a version (or suspending versioning) requires the **root user's MFA device** — the "extra protection against accidental or malicious deletes" answer.

Replication copies objects to another bucket automatically:

- **CRR** (Cross-Region Replication) → different region: compliance, disaster recovery (it **protects against a full regional outage**, which versioning alone can't), lower latency for far-away users.
- **SRR** (Same-Region Replication) → same region: log aggregation into one bucket, prod → test copies.
- Requires **versioning ON in both buckets**.
- **THE trap:** replication copies **NEW objects only**, from the moment you enable it. "*Existing objects must be replicated too*" → **S3 Batch Replication**.

The encryption ladder (sorted by how little you trust AWS)

Quick jargon: encryption **at rest** = scrambled while sitting on disk. Encryption **in transit** = scrambled while traveling over the network, done by **TLS** (Transport Layer Security — the "s" in https, the standard protocol that encrypts web traffic). **KMS** = AWS Key Management Service, the managed vault for encryption keys.

Climb the ladder as trust in AWS decreases and your effort increases:

Rung	Who holds the key	Why pick it
SSE-S3 (AES-256)	AWS, fully managed	**Default** on all new objects. Zero effort. "Just encrypt it"
SSE-KMS	A KMS key you control in your account	**Audit every key use in CloudTrail**, control rotation, decide who may decrypt. **"Audit" is its keyword**
SSE-C	**You** supply the key **on every request**	AWS's CPUs do the encrypting but never store your key; key travels per request **over HTTPS** (mandatory)
Client-side	You, entirely	You **encrypt BEFORE upload; AWS never sees plaintext**, ever

(SSE = Server-Side Encryption — S3's servers do the crypto work for the first three rungs.)

Rung details:

- **SSE-S3** uses **AES-256**. Each object is encrypted with a unique key, and that key is itself encrypted with a root key that AWS rotates regularly. S3 server-side encryption is **AES-256 only**, so an answer offering "AES-128" is a distractor.
- **Client-side** encryption: either use a **KMS key** to get data keys, or keep a **root key in your own application** and never send it to AWS. Client-side is also the answer when you must use **your own proprietary encryption algorithm**.

SSE-KMS fine print worth points: heavy traffic can hit **KMS API request limits** (every object touch calls KMS). **S3 Bucket Key** fixes this — one bucket-level key cuts **KMS API costs ~99%** — BUT fewer KMS calls means a **coarser per-object audit trail** in CloudTrail. Cost fix with an audit trade-off.

Three encryption traps:

- **"Encrypt in transit" = TLS/HTTPS**, not KMS. Don't let a KMS answer seduce you on a transit question.
- **Enforce HTTPS** → bucket policy that **Denies** requests where `aws:SecureTransport` = `false`. Classic exam answer, verbatim.
- Turning encryption on **does not touch existing objects** — encryption applies at write time. "*Encrypt all EXISTING objects*" → **S3 Batch Operations** (re-writes them encrypted).

Object Lock: WORM

Object Lock makes objects **WORM** — Write Once, Read Many: written, then **impossible to delete or overwrite** for a set retention period. Requirements: **versioning must be on**, and Object Lock can only be enabled **at bucket creation time** (exam loves this detail).

Mode	Who can delete during retention	Exam signal
Compliance mode	**NOBODY. Not admins, not even root.** Period.	"regulators require," "even administrators must not delete"
Governance mode	Users with a special bypass permission	"protected, but privileged users may override"
Legal Hold	No one, **indefinitely — no expiry date, stays until explicitly removed**	"**lawsuit**," litigation, "until the case closes"

THE trap: "even administrators / even the root user cannot delete" → **Compliance**, never Governance.

More Object Lock fine print:

- **One-way door:** once a bucket has Object Lock enabled, you **can't disable Object Lock** and **can't suspend versioning** on it.
- **Retention period** = a fixed amount of time an object version is protected. A **legal hold has no time period**; it stays until someone with `s3:PutObjectLegalHold` removes it.
- Retention and legal hold are **independent and can be stacked** on the same object version. A legal hold **keeps blocking deletes/overwrites even after the retention period has expired**.

Who can get in: access controls

- **Block Public Access** — the master kill-switch, **on by default**, overrides any policy or ACL that tries to make things public. "*Bucket policy allows public read but access still fails*" → Block Public Access is on. "*Guarantee nothing is ever public org-wide*" → keep it on everywhere.
- **Bucket policies** — resource-based JSON policies (Section 1!) on the bucket: cross-account grants, enforce HTTPS, require encryption headers, IP restrictions.
- **Presigned URLs** — a time-limited signed link to one object: "*let a customer download this private file for 20 minutes, no AWS account needed.*" The URL **inherits the permissions of whoever created it** (creator can't read it → URL doesn't work either) and lives **at most 7 days**.
- **OAC (Origin Access Control)** — when CloudFront (AWS's CDN — content delivery network, a global fleet of caching edge servers) sits in front of your bucket, OAC lets **only CloudFront** fetch from the bucket, so nobody can bypass the CDN and hit S3 directly. Signal: "*users must access content only through CloudFront.*" (You may see the older name OAI — OAC is the current answer.)

- **S3 presigned URL vs CloudFront signed URL:** a presigned URL hits S3 **directly and bypasses CloudFront**. If content is served through CloudFront, use **CloudFront signed URLs** (one file) or **signed cookies** (many files) instead. Presigned URLs also don't scale when many users need many files. See Section 8.
- **Cross-account access**: either a **bucket policy** that grants the other account, or an **IAM role in the bucket's account** that the other account assumes.
- **Object ownership (the cross-account upload trap):** by default an object is **owned by the account that uploaded it**, even if it's in your bucket, so the bucket owner may not be able to read it. Classic fix: require the `bucket-owner-full-control` **canned ACL**, enforced in the **bucket policy** (not an IAM policy). Modern fix: set **S3 Object Ownership = Bucket owner enforced**, which disables ACLs and makes the bucket owner own everything automatically.
- **ACLs (legacy):** they can grant access only to **other AWS accounts or predefined groups, not to individual IAM users**. They're **disabled by default** on new buckets (Bucket owner enforced). AWS recommends policies instead, so treat ACL-based answers as legacy unless the question forces them.
- **S3 Access Points**: named entry points to a bucket, each with its own policy. You can configure one to **accept requests only from a specific VPC**. Signal: "*restrict S3 data access to a private network / per-application access policies on a shared bucket.*"
- **VPC endpoint policy**: to limit which S3 buckets instances in a VPC may reach ("*only trusted buckets*"), write **one endpoint policy on the gateway endpoint**. That's far less work than editing every bucket's policy.
- **S3 has no security groups.** Network-level control comes from endpoint policies, Access Points, and bucket-policy conditions such as `aws:SourceVpce` / `aws:SourceIp`.
- **MFA-protected API access**: a bucket-policy condition (`aws:MultiFactorAuthPresent`) that requires MFA-authenticated requests. This is separate from **MFA Delete** (versioning).
- **CORS (Cross-Origin Resource Sharing)**: lets browser JavaScript from **one domain** load objects from a bucket served on **another domain** (for example, a web page on `palawan.com` loading assets from `travel.cebu.com`). **CORS does not make objects public.** It only relaxes the browser's same-origin rule.
- **Static website hosting**: S3 serves **static content only (HTML/CSS/JS/images), with no server-side code**, and it scales automatically. To use your domain through Route 53, the **bucket name must equal the domain/subdomain** (`portal.example.com` → bucket `portal.example.com`), and Route 53 must be the DNS service (Alias record). Need HTTPS? Put CloudFront in front.

Access Logs vs CloudTrail

S3 Server Access Logs record every request to a bucket, delivered as log files to *another* bucket. **THE trap: never log a bucket to itself** — each delivered log file is a new request, which writes a new log... an **infinite loop** that bloats your bill. Always a **separate target bucket**. Analyze the logs with **Athena** (SQL directly over files in S3 — no servers, pay per query).

	S3 Access Logs	**CloudTrail data events**
Cost	Basically free (storage only)	Paid per event
Delivery	**Delayed**, best-effort	**Near real-time**
Detail	Request-level, complete	**Rich identity info** (exactly which IAM principal)
Pick when	"cheap," "complete record," "analyze with Athena"	"real-time," "who (which IAM user) accessed," compliance alerting

THE detail trap: only **server access logs** record per-request fields such as the **referrer** and **turn-around time**, and they also feed billing/usage analysis. CloudTrail can't give you those. CloudTrail covers **bucket-level and object-level API calls** with identity.

Performance

- Baseline: **3,500 PUT/COPY/POST/DELETE and 5,500 GET/HEAD requests per second *per prefix*** — and unlimited prefixes. Need more throughput? **Spread objects across more prefixes** (that's what "add randomness/date partitions to key names" answers mean).
- **Multipart upload** — parallel chunks for big files: recommended >100 MB, required >5 GB, and a failed part retries alone.
- **Transfer Acceleration** — for **UPLOADS from users around the globe into ONE central bucket**: they upload to the nearest CloudFront edge location, then the data rides **AWS's private backbone** to the bucket. You **pay only if it's actually faster** than a normal upload. Signal: "*users worldwide upload large files to a central bucket, speed it up.*" For global **downloads**, the answer is **CloudFront**, not TA.
- **Byte-range fetches** — GET just a slice of an object (`Range` header): parallel downloads, or grab only the header/footer of a huge file.

Events and querying

- **Event notifications** — "when an object lands (or is deleted), do something": target **Lambda, SQS, SNS**, or **EventBridge** (EventBridge unlocks richer filtering and fan-out to many services). Classic: "*generate a thumbnail whenever an image is uploaded*" → S3 event → Lambda.
  - Event types: **object created**, **object removed**, **restore** (Glacier restore initiated/completed), **RRS object lost** (legacy class), **replication** events.
  - `s3:ObjectRemoved:DeleteMarkerCreated` fires **only when a delete marker is created** on a versioned object. It doesn't fire for `s3:ObjectRemoved:Delete`, which is a plain delete or permanent version deletion.
  - **THE fan-out trap:** each event notification configuration targets **exactly one destination**, and you can't have two configurations for the same event type with overlapping prefix/suffix filters. To deliver the same event to several SQS queues or consumers, send it to **one SNS topic and fan out** to the queues, or use **EventBridge**. Delivery is **at least once**.
- **S3 Select** — run SQL against the contents of **ONE object** (CSV/JSON/Parquet) and pull back only the matching rows — the app downloads kilobytes instead of the whole gigabyte. Signal: "*retrieve only a subset of data from a single large object.*"
- **Athena** — SQL **across many objects/whole buckets**, serverless, pay-per-query. Signal: "*analyze/query logs stored in S3 with standard SQL.*" One object → S3 Select; a data set → Athena.

Question patterns

> "*Compliance data accessed about twice a year; auditors must receive it within 10 minutes. Most cost-effective class?*" → **Glacier Flexible Retrieval** (Deep Archive can never beat 12 hr; Instant passes but costs more; Flexible Expedited = 1–5 min).

"*Records retained 7 years, almost never accessed, retrieval within 2 business days is acceptable. Cheapest?*" → **Glacier Deep Archive** (deadline ≥ 12 hr finally lets the cheapest class survive).

"*Reports are heavily used for 30 days, rarely after 90, must always be retrievable in milliseconds.*" → **Lifecycle rule: Standard → Standard-IA at 30 days → Glacier Instant Retrieval at 90** (known pattern → lifecycle; "milliseconds" keeps you in Instant, out of Flexible).

"*Access patterns are unknown and unpredictable; minimize cost without performance impact.*" → **Intelligent-Tiering** ("unknown/unpredictable" is its one and only signal).

"*Thumbnails can be regenerated from originals at any time; store infrequently accessed copies cheaply.*" → **One Zone-IA** ("can be regenerated" = single-AZ risk is acceptable).

"*Processing artifacts are deleted after 20 days. Cheapest class?*" → **S3 Standard** (under every minimum duration — IA/Glacier would bill 30–180 days anyway).

"*Security must audit exactly when each encryption key was used, and control key rotation.*" → **SSE-KMS** ("audit key usage" is the KMS keyword; add S3 Bucket Key if the question moans about KMS request costs).

"*Regulatory requirement: data must be impossible to delete for 5 years — even by administrators.*" → **Object Lock, Compliance mode** ("even administrators" rules out Governance).

"*Objects must be preserved indefinitely while litigation is pending.*" → **Legal Hold** ("lawsuit" + no fixed end date).

"*Users on every continent upload multi-GB video files to a single S3 bucket; speed up uploads.*" → **Transfer Acceleration + multipart upload** (global UPLOAD to one bucket; global download would be CloudFront).

"*All objects must be encrypted in transit.*" → **Bucket policy denying requests where `aws:SecureTransport` is false** (in transit = HTTPS/TLS, not KMS).

"*Enable access logging for a bucket — what's the key configuration rule?*" → **Deliver logs to a DIFFERENT bucket** (same bucket = infinite logging loop); query them with **Athena**.

"*Replication was enabled but existing objects didn't copy over.*" → **S3 Batch Replication** (CRR/SRR only replicate NEW objects).

"*Expedited Glacier retrievals must be available whenever needed, with guaranteed throughput*" → **Purchase provisioned capacity** (each unit = ≥ 3 Expedited retrievals per 5 min, up to 150 MB/s).

"*Lifecycle rule: move objects to Standard-IA after 7 days*" → **Not allowed. Standard-IA/One Zone-IA need ≥ 30 days in Standard** (Glacier transitions have no such floor).

"*Partner account uploads objects into our bucket, but we can't read them*" → **S3 Object Ownership = Bucket owner enforced** (or require the `bucket-owner-full-control` ACL in the **bucket policy**). The uploader owns objects by default.

"*Instances in a VPC must only reach approved S3 buckets*" → **VPC gateway endpoint policy** (one policy beats editing every bucket policy; S3 has no security groups).

"*JavaScript on www.example.com must fetch images from a bucket on another domain*" → **Enable CORS on the bucket** (cross-origin permission, not public access).

"*Need the referrer and turn-around time of every request to a bucket*" → **S3 Server Access Logs** (CloudTrail doesn't record those fields).

"*Send an S3 event to two SQS queues*" → **S3 event** → **SNS topic** → **both SQS queues** (each event config has one destination; overlapping configs are rejected).

"*Trigger only when a versioned object gets a delete marker*" → `s3:ObjectRemoved:DeleteMarkerCreated`.

"*Keep archived data queryable by Athena at low cost*" → **S3 Standard-IA** (Athena can't read Glacier Flexible/Deep Archive without a restore).

"*Host a static site at portal.example.com from S3*" → **Bucket named `portal.example.com` + static website hosting + Route 53 Alias** (bucket name must match the domain; static content only).

Pocket card

Keyword	Answer
Frequently accessed / hot	S3 Standard
Lives < 30 days	Standard (minimum-duration charges kill the rest)
Infrequent + instant retrieval	Standard-IA
"Can be regenerated / secondary copy"	One Zone-IA
"Unknown / unpredictable access"	Intelligent-Tiering
Known pattern over time	Lifecycle rule (beats Intelligent-Tiering)
Archive + milliseconds retrieval	Glacier Instant Retrieval
Archive + minutes-to-hours OK	Glacier Flexible (Expedited 1–5 min / Std 3–5 hr / Bulk 5–12 hr)
Cheapest, 12+ hr retrieval OK	Glacier Deep Archive (never faster than 12 hr)
Object > 5 GB (or > 100 MB)	Multipart upload (required / recommended)
Accidental delete/overwrite protection	Versioning (+ MFA Delete for root-MFA-gated deletes)
Copy to another region, DR/compliance	CRR (versioning on both sides)
Aggregate logs / copy within region	SRR
Replicate EXISTING objects	S3 Batch Replication
Default encryption, zero effort	SSE-S3
"Audit key usage" / control rotation	SSE-KMS (in CloudTrail)
KMS request costs too high	S3 Bucket Key (~99% fewer KMS calls, coarser audit)
Customer supplies key per request	SSE-C (HTTPS mandatory)
"AWS must never see plaintext"	Client-side encryption
Encrypt in transit / enforce HTTPS	Deny aws:SecureTransport = false
Encrypt EXISTING objects	S3 Batch Operations
WORM, "not even root can delete"	Object Lock — Compliance mode
WORM, privileged users may bypass	Object Lock — Governance mode
"Lawsuit," indefinite hold	Legal Hold
Temporary download link, no AWS account	Presigned URL (creator's perms, ≤ 7 days)
Public policy exists but access fails	Block Public Access is on
S3 reachable only via CloudFront	OAC
Who accessed — cheap/complete/delayed	S3 Server Access Logs (different bucket! + Athena)
Who accessed — real-time + IAM identity	CloudTrail data events
More requests/sec	More prefixes (3,500 PUT / 5,500 GET per prefix)

Keyword	Answer
Global UPLOADS to one bucket	Transfer Acceleration (pay only if faster)
Global DOWNLOADS	CloudFront
Download part of an object	Byte-range fetch
"When object uploaded, trigger..."	Event notification → Lambda/SQS/SNS/EventBridge
SQL on ONE object	S3 Select
SQL across S3 data sets	Athena
Read right after write — stale?	No: strong read-after-write consistency since Dec 2020
Two PUTs to same key at once	Latest timestamp wins; app-level locking needed
Lifecycle → Standard-IA / One Zone-IA	Only after ≥ 30 days in Standard
Expedited Glacier retrieval must always be available	Provisioned capacity (≥ 3 per 5 min, 150 MB/s)
SQL on archived data in place	Glacier Select
Athena on Glacier data?	No: restore first (or keep in Standard-IA)
Lockable compliance policy on a Glacier vault	Vault Lock policy
Legal hold duration	None; stays until removed, outlives retention
Disable Object Lock later?	Never; versioning can't be suspended either
Cross-account uploads unreadable by bucket owner	Object Ownership = Bucket owner enforced (or require `bucket-owner-full-control` in bucket policy)
ACL grants to an IAM user?	No: ACLs grant to accounts/groups only (legacy)
Restrict bucket access to one VPC, per-app policies	S3 Access Points
Limit VPC instances to trusted buckets only	VPC endpoint policy
Page on domain A loads S3 objects on domain B	CORS (doesn't make objects public)
Requester pays download, owner pays storage	Requester Pays bucket
Static website on custom domain	Bucket name = domain + Route 53 Alias (CloudFront for HTTPS)
Content served via CloudFront, restrict access	CloudFront signed URL/cookies (not S3 presigned)
Referrer / turn-around time per request	S3 Server Access Logs
Only when a delete marker is created	`s3:ObjectRemoved:DeleteMarkerCreated`
Same S3 event to several SQS queues	SNS fan-out (or EventBridge)
SSE with AES-128?	No: S3 SSE is AES-256

S3's bucket policies are just Section 1's resource-based policies wearing a storage hat — and its classes ladder is the warm-up for the same cost-vs-speed reasoning you'll reuse on EBS, EFS, and Glacier-style questions everywhere else on the exam.

↑ Back to index

Section 3: EC2 — Elastic Compute Cloud

The idea

EC2 (Elastic Compute Cloud) is renting a computer in AWS's data center. That's it. You pick the size, you pick how you pay, you press launch, and seconds later you have a virtual machine (an "instance") with an IP address that you can SSH into.

Here's the analogy that makes the whole section click: **EC2 is a rental car agency.** The instance *family* is the car type — you don't rent a pickup truck to drive to a wedding, and you don't rent a Miata to haul furniture. The *purchasing option* is the rental contract — daily rate for a weekend trip, a yearly lease if you commute every day, or a crazy-cheap standby rate where the agency can take the car back with two minutes' notice. Almost every EC2 exam question is really asking one of two things: **which car?** or **which contract?**

Let's learn to answer both on sight.

Instance families: which car?

The name tells you everything. Take **m5.2xlarge**:

```
m        5       .2xlarge
|        |        |
family   generation  size
(what    (newer =    (bigger = more
 it's     better $)   vCPU + RAM)
 good at)
```

Now the families, with mnemonics so they stick:

Letter	Mnemonic	Good for	Exam keywords
M	**M**iddle-of-the-road	General purpose, balanced CPU:RAM	"web app server", no special demands
T	**T**urbo (burstable)	Spiky, mostly-idle workloads	"occasional spikes", "dev/test", CPU credits
C	**C**PU / Compute	CPU-hungry work	"batch processing", "encoding", "HPC", "gaming servers"
R	**R**AM	Memory-hungry work	"in-memory cache", "real-time big data analytics"
X	e**X**treme RAM	Massive in-memory databases	**"SAP HANA"** — this word alone means X family
I	**I**OPS	Fast local NVMe storage	"high random I/O", "NoSQL needing local disk"
D	**D**ense **D**isk	Lots of local HDD storage	"distributed file systems", "data warehousing"
P	GPU (**P**ower training)	**ML model training**	"train a deep learning model"

Letter	Mnemonic	Good for	Exam keywords
G	GPU (**G**raphics)	Graphics rendering, ML **inference**	"video rendering", "game streaming"
Inf	**Inf**erence	Purpose-built ML inference chips	"cost-effective inference at scale"

AWS groups these letters into **five categories**, and the exam often uses the category name instead of the letter:

Category	Families	Signal
General Purpose	M, T	web servers, dev environments, general apps
Compute Optimized	C	compute-intensive work
Memory Optimized	R, X	big in-memory datasets and analytics
Accelerated Computing	P, G, Inf	**GPUs / FPGAs** / specialized hardware
Storage Optimized	I, D	**high sequential read/write to very large local datasets** (e.g., log processing), tens of thousands of low-latency random IOPS

THE trap: "*high sequential read and write access to very large data sets on local storage*" → **Storage Optimized**, NOT Memory Optimized. "Large dataset" tempts you toward RAM, but "local storage + sequential I/O" is the disk family.

And one magic suffix: a **g** after the generation number (like **m7g**, **c7g**) means **Graviton — AWS's own ARM processor**. The exam signals for Graviton are **"best price-performance"**, **"reduce compute cost"**, or **"sustainability"**. See those words with a Linux workload that can run on ARM? Answer: Graviton.

The T-family trap (learn this cold)

T instances are "burstable." Think of a phone battery: while the instance idles, it **earns CPU credits**; when load spikes, it **spends credits** to burst above its tiny baseline. Great for workloads that are quiet most of the day.

THE trap: "*Our t3 instance was fine for hours, then performance degraded under sustained load.*" What happened? **Credits ran out → the instance is throttled down to its baseline CPU.** The fix depends on the workload:

- Sustained, steady load → **move to an M or C family** (they don't play the credit game).
- Keep the T instance but never throttle → enable **T Unlimited mode**, which lets it keep bursting and **bills you for the overage**.

Corollary trap: "*We enabled a setting and now the bill increased during a traffic spike*" → that's **Unlimited mode overage charges**. Bursting past credits isn't free — it's a loan.

Purchasing options: which contract?

This is the most-tested pricing topic on the exam. The decision hinges on two words in the question: **how long** and **how interruptible**.

Option	Discount	Commitment	When the exam wants it
On-Demand	none	none	Short-term, **unpredictable**, can't commit. **Anything under 1 year** → On-Demand, because RIs/Savings Plans start at 1 year
Reserved Instances	up to **72%** (Standard)	**1 or 3 years**	Steady, predictable, long-running (e.g., a database that runs 24/7)
Convertible RI	up to **66%**	1 or 3 years	Steady load but **"instance family might change"** — you trade discount for flexibility
Savings Plans	~same as RI	commit **$/hour** for 1 or 3 years	Steady **spend**, flexible everything else
Spot	up to **90%**	none — AWS can reclaim with a **2-minute warning**	Fault-tolerant, stateless, batch, interruptible
Dedicated Host	pricey	a whole physical server	**BYOL** (bring your own license) with **per-socket / per-core** licensing
Dedicated Instance	cheaper than Host	isolated hardware	"Must not share hardware with other customers" but no license/visibility needs

Now the nuances the exam actually tests:

Zonal RI = discount AND a seat reservation. A regular (regional) RI is just a billing discount. But a **zonal RI** — one scoped to a specific Availability Zone — **also reserves capacity** in that AZ. So "*we must guarantee capacity will be available for our month-end batch run*" → **zonal RI or an On-Demand Capacity Reservation**. **Savings Plans never reserve capacity** — they are billing constructs only. That distinction is a favorite trick.

The capacity-vs-discount grid (worth memorizing as a table):

Option	Term	Reserves capacity?	Billing discount?	Instance limits
On-Demand Capacity Reservation	**none** — any duration, cancel anytime	✅ yes (one AZ)	❌ none by itself — **pair with a Savings Plan or regional RI** for the discount	your On-Demand limits
Zonal RI	1 or 3 years	✅ yes	✅ yes	20 per AZ
Regional RI	1 or 3 years	❌ no	✅ yes	20 per Region
Savings Plans	1 or 3 years	❌ no	✅ yes	no limit

So "*guarantee capacity for a short event, no long-term commitment*" → **On-Demand Capacity Reservation** (billing starts when it goes active and stops when you cancel it). "*Guarantee capacity AND get a discount for 3 years*" → **Zonal RI**.

RI fine print the exam likes:

- When an RI **expires**, the instances it covered keep running but are **billed at On-Demand rates** — the discount silently disappears.
- **Terminating** an instance does **not** cancel its RI — you **keep paying until the term ends**.
- An RI is scoped to **one Region (or one AZ)**; one purchase can't cover several Regions.
- **Convertible RIs** can be exchanged only for **other Convertible RIs**.

- Unused **Standard RIs** can be sold on the **Reserved Instance Marketplace**; Convertible RIs can't.

Billing granularity: On-Demand Linux/Windows instances are **billed per second** (60-second minimum). The old "billing hour" is legacy. While an instance is **stopping to hibernate you ARE billed**; while it's **stopping to stop, you are not**. On-Demand capacity is capped by a **vCPU-based limit per Region** — if you hit it, **request a limit increase**; you don't need to move Regions.

Savings Plans come in two flavors:

- **Compute Savings Plan** — applies to **any instance family, any region, and even Fargate and Lambda**. Maximum flexibility, slightly smaller discount. Keyword: "*flexibility to change instance types and regions.*"
- **EC2 Instance Savings Plan** — locked to **one family in one region** (e.g., "m5 in us-east-1"), but a **bigger discount**. Keyword: "*committed to this family, wants the largest savings.*"

Spot rules of engagement. Up to 90% off, but AWS can pull the instance with a **2-minute interruption notice**. So Spot fits work that is **stateless, fault-tolerant, checkpointable, batch, or deadline-flexible**: image rendering, CI builds, big data analytics, containerized workers. **NEVER** put databases or anything critical/uninterruptible on Spot — "*cannot tolerate interruption*" in a question is a flashing sign that Spot is the wrong answer, no matter how much the question emphasizes cost.

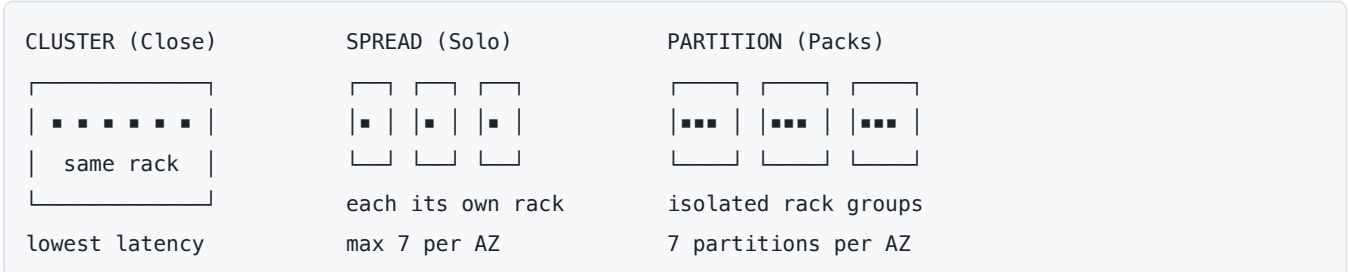
The classic combo: "*predictable baseline traffic plus unpredictable surges, minimize cost*" → **Reserved Instances (or a Savings Plan) for the baseline + Spot (or On-Demand) for the surge**.

Host vs Instance. Both give you hardware not shared with other AWS customers. The difference:

- **Dedicated Host** — you get **visibility into the physical server itself: sockets, cores, host ID**. That's exactly what **per-socket or per-core BYOL licenses** (old-school Windows Server, SQL Server, Oracle licenses) require. **The word "socket" or "BYOL" in the question = Dedicated Host.**
- **Dedicated Instance** — hardware isolation, but **no visibility or control over the physical server**, and cheaper. Compliance-only isolation, no licensing story. Fine print: a Dedicated Instance **may share hardware with non-dedicated instances from your own account** — it's isolated from *other customers*, not from you.

Placement groups: where the cars park

Three strategies. Mnemonic: **Cluster = Close, Spread = Solo, Partition = Packs.**



- **Cluster** — everything packed on the **same rack** in one AZ. **Lowest latency, highest throughput.** Keyword: **HPC, tightly-coupled, node-to-node network performance**. Risk: rack dies, everything dies.
- **Spread** — every instance on **its own rack** with its own power/network. Maximum blast-radius isolation, but capped at **7 instances per AZ**. Keyword: **small number of critical instances that**

must not fail together.
- **Partition** — up to **7 partitions per AZ**, each partition is an isolated group of racks, and each partition can hold **hundreds of instances**. A rack failure takes out one partition, not the fleet — and the instances can **read their partition placement from instance metadata**, which topology-aware software uses. Keywords: **Hadoop, Cassandra, Kafka, HDFS** — big distributed systems.

THE trap: "*critical fleet of 20 instances, each must be isolated from hardware failure*" — Spread sounds right but caps at 7 per AZ; either spread across 3 AZs (7+7+6 works) or the answer is Partition. Count the instances.

Capacity errors in a placement group. Best practice: launch **all the instances you need in a single launch request, all the same instance type**. Adding instances later or mixing types raises the odds of an **insufficient capacity error**. If you hit one while the group already has running instances → **stop and start all instances in the group, then retry** (the restart can migrate them onto hardware with room for everyone). Cluster and partition groups have no fixed instance limit; only Spread is capped at 7 per AZ.

ENI: the detachable network identity card

An **Elastic Network Interface (ENI)** is a virtual network card, and here's the key mental model: it's a **detachable identity badge**. The ENI carries the **private IP address, MAC address, and security groups** — not the instance. Detach the badge from a failed instance, clip it onto a standby instance, and the standby instantly answers to the **same private IP**.

That's the exam's favorite **failover trick**: "*on failure, redirect traffic to a standby instance without changing the IP the clients use*" → **move the ENI**.

THE trap: an ENI lives in one subnet, so **it can only move to an instance in the SAME Availability Zone**. Cross-AZ failover with an ENI is impossible — for that you need DNS, a load balancer, or Route 53.

One more pattern: attaching **multiple ENIs** to one instance = **dual-homing** — for example, one interface on the public network and one on a separate **management network**.

Enhanced networking: ENA, EFA, and Nitro

- **Enhanced networking** uses **SR-IOV (single root I/O virtualization)** to give higher bandwidth, higher packets per second (PPS), and lower, more consistent inter-instance latency — at **no extra charge**.
- **ENA (Elastic Network Adapter)** — the standard enhanced-networking device, **up to 100 Gbps** on supported types (more on the newest network-optimized types). It provides the normal IP networking that VPCs need. The **Intel 82599 VF** interface (up to 10 Gbps) is the **legacy** option.
- **EFA (Elastic Fabric Adapter)** — everything ENA does **plus OS-bypass**, which lets HPC and ML apps (MPI, NCCL) talk directly to the network hardware for ultra-low-latency **inter-node communication**. Signal: "*tightly-coupled HPC / MPI*" → **EFA + Cluster placement group**. **THE trap: EFA's OS-bypass is NOT supported on Windows** — Linux only.
- **ENI vs ENA vs EFA:** ENI = a virtual network *card* (identity: IP, MAC, SGs, above); ENA = *fast* networking; EFA = *HPC* networking.
- **Nitro System** — AWS's hardware virtualization platform under every current-generation instance. It offloads networking, storage, and security to dedicated hardware, which unlocks things like the highest EBS IOPS and enhanced networking.

Elastic IP: the phone number you own

THE trap first, because it's tested constantly: the default auto-assigned **public IP changes every time you stop and start the instance**. (It survives a reboot, but stop/start = new IP.) So "*clients could no longer reach the server after maintenance stop/start*" → the public IP changed → the fix is an **Elastic IP**.

An **Elastic IP (EIP)** is a static public IPv4 address that **you own** and can remap to any instance — **even across AZs** (unlike an ENI move). The fine print:

- **Charged when NOT attached** to a running instance (idle = billed — AWS wants you using them or releasing them).
- Soft limit of **5 per region**.
- Classic use: **mask an instance failure** by remapping the EIP to a standby instance — clients keep the same address.
- **Best practice per the exam:** avoid EIPs when you can — put a **load balancer with a DNS name** in front, or use **Global Accelerator** (which gives you 2 static anycast IPs). "Architect wants to avoid Elastic IPs" → ELB + DNS.

Boot-time toolkit: user data, metadata, hibernate

- **User data** — a script you paste at launch. It **runs once, at first boot, as root**. Keyword: "*automatically install/configure software when the instance launches*" → user data. (Runs every boot? No — once. Need every boot, you'd modify cloud-init config, but the exam wants "once at launch.") Because it runs as root, the script **doesn't need** `sudo`. You **can't change user data while the instance is running** — stop it first.
- **Instance metadata** — from inside the instance, hit `http://169.254.169.254/latest/meta-data/` to learn about yourself: instance ID, AZ, attached IAM role credentials, partition number. Memorize that IP. **IMDSv2** is the hardened version: it requires a **session token first** (a PUT request, then use the token), which blocks the SSRF attacks that plagued v1. "Protect metadata from SSRF" → **enforce IMDSv2**.
- **Hibernate** — instead of stopping cold, the instance's **RAM is written to the encrypted EBS root volume**; on start, RAM is restored and the app resumes mid-thought. Keywords: "*long-running in-memory state*", "*reduce boot time / pre-warmed caches*". Requirements: root volume must be **encrypted EBS**, and RAM must fit on it.

AMIs: the car's factory image

An **AMI (Amazon Machine Image)** is the template an instance boots from: OS, installed software, and block-device mapping.

- **AMIs are Region-bound.** To launch the same image in another Region (for example, for **disaster recovery**), you must **copy the AMI** to that Region first.
- **Golden AMI** — a standardized, **patched, hardened** image your organization approves. The boot-speed pattern is **hybrid**: bake the **static** parts (OS, packages, agents) into the golden AMI and do only the **dynamic** parts (fetch config, join cluster) in **user data** at boot. "*Reduce instance boot/bootstrap time*" → **golden AMI**, not a longer user data script.
- **Instance-store-backed AMI** — the root volume lives on **instance store**, so the instance **cannot be stopped** (only rebooted or terminated) and its **data is gone on termination**. EBS-backed AMIs are the default and what you want almost always.

Question patterns

"*Nightly batch job processes images; jobs can be re-run if they fail; minimize cost*" → **Spot instances** (fault-tolerant + interruptible + cheapest = Spot's home turf)

"*Critical payment workload, cannot tolerate interruption, minimize cost*" → **NOT Spot — Reserved Instances / Savings Plan** ("cannot tolerate interruption" disqualifies Spot no matter how loudly the question says "cheapest")

"*SAP HANA in-memory database on EC2*" → **X family** (eXtreme RAM; plain "in-memory cache/analytics" → R family)

"*Train a deep learning model*" *vs* "*serve ML predictions cheaply*" → **P family for training, G or Inf for inference** (P = Power training, G/Inf = inference/graphics)

"*Company has per-socket Windows Server licenses to bring to AWS*" → **Dedicated Host** (sockets/cores/BYOL = Host; mere hardware isolation = Dedicated Instance)

"*Must guarantee EC2 capacity is available in a specific AZ for month-end processing*" → **Zonal Reserved Instance or On-Demand Capacity Reservation** (Savings Plans and regional RIs are billing-only — they hold no seats)

"*t3 instance performs well, then slows dramatically after hours of sustained load*" → **CPU credits exhausted, throttled to baseline — move to M/C family or enable Unlimited mode** (and "bill spiked after enabling a setting" = Unlimited overage)

"*After stopping and starting the instance, users can't reach it at the old address*" → **Public IP changed on stop/start — assign an Elastic IP** (default public IPs are not sticky)

"*On primary failure, standby must take over with the same private IP, no client changes*" → **Move the ENI to the standby** (works only within the **same AZ**)

"*Tightly-coupled HPC workload needs the lowest inter-node latency*" → **Cluster placement group** (Close = same rack; Kafka/Cassandra/Hadoop at scale = Partition; 7 critical solo instances = Spread)

> "*Steady $ commitment but wants freedom to switch families, regions, and even Fargate/Lambda*" → **Compute Savings Plan** (one family, one region, max discount = EC2 Instance Savings Plan)

> "*Predictable baseline + unpredictable spikes, optimize cost*" → **RI/Savings Plan for baseline + Spot for the spikes** (the classic combo)

> "*Need guaranteed capacity in one AZ for a two-week event, no 1-year commitment*" → **On-Demand Capacity Reservation** (no term; add a Savings Plan/regional RI if you also want a discount)

> "*Our 3-year RI ended and the bill jumped, though nothing changed*" → **RI expired → instances now billed On-Demand** (renew or buy a Savings Plan)

> "*Launch the same server image in another Region for DR*" → **Copy the AMI to that Region** (AMIs are Region-bound)

> "*Instances take too long to bootstrap at scale-out*" → **Golden AMI with static software baked in + small user data for dynamic config**

> "*HPC/MPI app needs OS-bypass, lowest inter-node latency*" → **EFA (Linux) + Cluster placement group** (EFA OS-bypass isn't supported on Windows; plain high bandwidth = ENA)

> "*Insufficient capacity error adding instances to an existing placement group*" → **Stop and start all instances in the group, then retry** (and next time launch them all in one request, same type)

> "*Large sequential reads/writes to huge local datasets, e.g., log processing*" → **Storage Optimized** (not Memory Optimized)

Pocket card

Keyword	Answer
general purpose / balanced	M family
spiky, mostly idle / CPU credits	T family (burstable)
t3 slow after sustained load	credits exhausted → M/C family or Unlimited mode
bill increased during spike	T Unlimited mode overage
batch / encoding / HPC compute	C family

Keyword	Answer
in-memory analytics / cache	R family
SAP HANA	X family
high local NVMe IOPS	I family
dense local HDD storage	D family
ML training	P family (GPU)
graphics / ML inference	G family / Inf
price-performance / cost / sustainability + ARM-friendly	Graviton (g suffix, e.g. m7g)
short-term, unpredictable, < 1 year	On-Demand
steady 24/7 for 1–3 years	Reserved Instance (up to 72%)
RI but family may change	Convertible RI (66%)
guarantee capacity in an AZ	Zonal RI / Capacity Reservation
commit $/hr, any family/region + Fargate/Lambda	Compute Savings Plan
commit $/hr, one family one region, max discount	EC2 Instance Savings Plan
interruptible / fault-tolerant / cheapest (up to 90%)	Spot (2-min warning)
database / cannot tolerate interruption	never Spot
baseline + surge, optimize cost	RI/SP baseline + Spot surge
BYOL / per-socket / per-core licenses	Dedicated Host
hardware isolation only, cheaper	Dedicated Instance
lowest latency / HPC / same rack	Cluster placement group
few critical instances, isolate failures (max 7/AZ)	Spread placement group
Hadoop / Cassandra / Kafka at scale	Partition placement group
failover keeping same private IP (same AZ)	move the ENI
separate management network / dual-homed	multiple ENIs
public IP changed after stop/start	Elastic IP
avoid EIPs / static entry point best practice	ELB + DNS or Global Accelerator
run script once at launch	user data (runs as root)
instance info from inside / SSRF protection	metadata at 169.254.169.254 / enforce IMDSv2
preserve RAM state across stop	Hibernate (RAM → encrypted EBS root)
sequential I/O on huge local datasets	Storage Optimized (I/D), not Memory Optimized
GPUs / FPGAs	Accelerated Computing (P/G/Inf)
capacity, no term commitment	On-Demand Capacity Reservation (no discount alone)
RI expired	instances billed at On-Demand rates
terminated instance with RI	RI still billed until term ends
sell unused RIs	RI Marketplace (Standard only)
Convertible RI exchange	only for other Convertible RIs

Keyword	Answer
billed while stopping?	yes if hibernating, no if stopping
hit On-Demand vCPU limit	request a limit increase (not a new Region)
placement group capacity error	stop + start all instances, retry
Dedicated Instance shares hardware with?	only your own account's non-dedicated instances
same image in another Region / DR	copy the AMI (Region-bound)
faster boot, standardized hardened image	golden AMI + user data for dynamic parts
instance-store-backed AMI	can't stop; data lost on terminate
enhanced networking tech	SR-IOV (ENA up to 100 Gbps, no extra cost)
HPC / MPI / OS-bypass	EFA (not on Windows)
current-gen virtualization platform	Nitro
user data needs sudo?	no — runs as root; can't edit while running

Every EC2 instance needs disks to boot from and store data on — that's EBS, EFS, and Instance Store, up next in Section 4.

↑ Back to index

Section 4: EBS, EFS & Instance Store

The idea

Your EC2 instance needs somewhere to put its files. AWS gives you three very different kinds of "somewhere," and the exam loves making you pick between them.

Here's the analogy: think of your instance as a desk in an office.

- **EBS (Elastic Block Store)** is your **personal external hard drive** plugged into your desk. It's actually a network-attached drive (the data lives on separate storage servers, not inside your computer), but it feels local. Unplug it from one desk, plug it into another. It's *yours alone*.
- **EFS (Elastic File System)** is the **shared network folder** the whole office mounts. Everyone sees the same files at the same time.
- **Instance Store** is the **notepad glued to the desk itself**. Blazing fast to scribble on — but when you give up the desk, the notepad goes in the shredder.

Three storage types, three personalities: **private and persistent (EBS), shared (EFS), fast but disposable (Instance Store).** Let's take them in turn.

EBS: your network-attached hard drive

Rules of the game — each one is exam-tested:

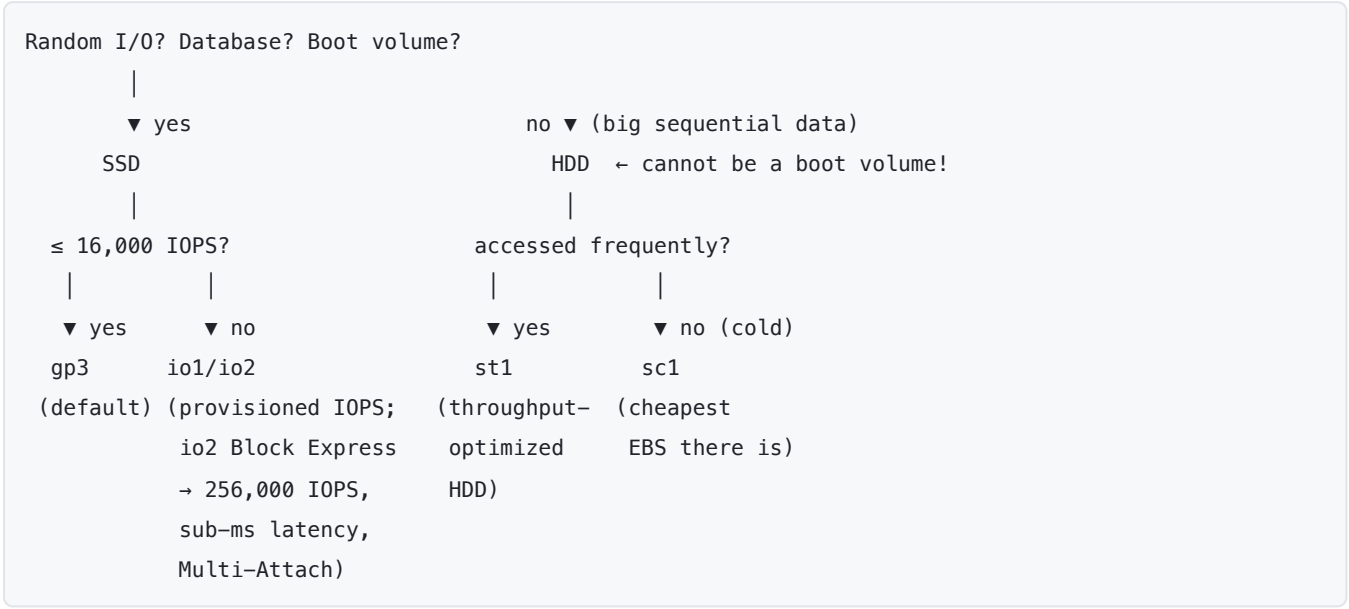
- An EBS volume lives in **exactly one Availability Zone**. To move it to another AZ, you snapshot it and restore the snapshot there. Within that AZ, AWS **automatically replicates** the volume to protect against component failure. That protection isn't cross-AZ; for that you need snapshots.
- **Elastic Volumes**: you can change a volume's **type, size, and IOPS/throughput live**, while it's attached and in use. No detach or downtime is needed.
- It attaches to **one instance at a time** — with one exception: **io1/io2 Multi-Attach**, which lets up to **16 instances in the same AZ** attach the same volume (for cluster-aware software only, not ordinary filesystems).
- It **persists independently** of the instance — the drive outlives the desk...
- ...**except** for the deletion flags: the **root volume** is created automatically and has **DeleteOnTermination = true by default** (terminate the instance, lose the root disk). **Extra attached volumes default to false** — they survive. So "*the root volume must survive termination*" → **turn DeleteOnTermination off** for the root volume.
- A freshly attached extra volume is a **raw disk**: you must **format it, mount it, and add it to /etc/fstab** yourself, or it vanishes from the mount table on reboot.

The one axis that decides everything: IOPS vs Throughput

Every EBS question secretly asks: is this workload about **many small random operations** or **big sequential streams**?

- **IOPS** (input/output operations per second) = **lots of small, random reads/writes**. That's what **databases** do — hop around the disk grabbing tiny rows.
- **Throughput** (MB/s) = **large, sequential** reads/writes. That's **big data, log processing, ETL, streaming** — chewing through giant files front to back.

The full decision tree:



The 16,000 IOPS threshold decides most questions. Need 12,000 IOPS? gp3 does that (and cheaply). Need 50,000? Only io1/io2 can. Need up to **256,000 IOPS with sub-millisecond latency**? **io2 Block Express.** And remember: **HDD volumes (st1/sc1) can never be boot volumes.** Provisioned IOPS SSD is the pick for **> 16,000 IOPS or > 250 MiB/s per volume**. The **magnetic "standard"** volume type is **previous-generation (legacy)**, so don't choose it for new workloads.

The SSD vs HDD family at a glance:

	SSD (gp2/gp3/io1/io2)	**HDD (st1/sc1)**
Best for	**Small, random** I/O	**Large, sequential** I/O
Bootable	**Yes**	**No**
Use cases	Transactional workloads, sustained IOPS, databases (MongoDB, Oracle, SQL Server)	Streaming, big data, data warehouses, log processing, infrequently accessed throughput data
Cost	Moderate / high	Low
Dominant performance attribute	**IOPS**	**Throughput (MiB/s)**

The io1 IOPS math (the 50:1 rule): io1 volumes are **4 GiB – 16 TiB** and can be provisioned with **100 to 64,000 IOPS** on Nitro instances (32,000 on other instances). The catch: the **maximum IOPS-to-size ratio is 50:1**, so **max IOPS = size in GiB × 50**.

io1 volume size	**Max provisionable IOPS**
10 GiB	10 × 50 = **500 IOPS**

io1 volume size	Max provisionable IOPS
100 GiB	5,000 IOPS
640 GiB	**32,000 IOPS** (the non-Nitro ceiling)
1,280 GiB+	64,000 IOPS (the Nitro ceiling)

Worked trap: "*a 10 GiB io1 volume provisioned with 1,000 IOPS*" is impossible, because 1,000 > 10 × 50. (**io2** allows **500 IOPS per GiB**.)

gp2 fine print (legacy, but still asked): 1 GiB – 16 TiB, at 3 IOPS/GiB with a **100 IOPS floor** (volumes ≤ 33.33 GiB) and a **16,000 IOPS ceiling**. Small volumes **burst to 3,000 IOPS** using credits.

RAID on EBS (configured inside the OS, not by AWS):

	RAID 0 (striping)	RAID 1 (mirroring)
Goal	**Performance**: combined IOPS/throughput of all volumes	**Redundancy**: same data written to 2 volumes
Fault tolerance	**None**: lose one volume, lose the array	Survives a volume loss
Exam signal	"more IOPS/throughput than one volume gives"	"mirror the data on two volumes"

(RAID is an **EBS** technique. EFS is a managed network file system and isn't assembled into a RAID array.)

gp2 vs gp3: the free-money question

	gp2 (old)	gp3 (new)
IOPS	**chained to size: 3 IOPS per GB**, bursts to 3,000	**baseline 3,000 IOPS + 125 MB/s at ANY size**
Scaling performance	must grow the volume to grow IOPS	**provision IOPS and throughput independently**
Price	baseline	**~20% cheaper**

On gp2, wanting more IOPS meant buying storage you didn't need. gp3 broke that chain. So the moment a question mentions **gp2**, the answer is almost always **"migrate to gp3"**. And "*needs more IOPS without paying for more storage*" → **gp3** (independent provisioning). It's the exam's favorite free lunch.

Snapshots: the time machine

- Snapshots are **incremental backups stored in S3** — only changed blocks after the first one, so they're cheap. Taking a snapshot is **point-in-time and non-blocking**: the volume keeps serving **reads and writes** while the snapshot is in progress.
- You can **copy a snapshot to another region** — that's the answer to **cross-region disaster recovery** ("back up EBS data to another region" → snapshot, copy it over).
- **Snapshot Archive tier**: **~75% cheaper**, but restore takes **24–72 hours**. Keyword: "rarely restored, minimize snapshot cost."
- **Recycle Bin**: retention rules that catch accidentally deleted snapshots so you can recover them.
- **Fast Snapshot Restore (FSR)**: normally a volume restored from a snapshot is lazily loaded (first read of each block is slow). FSR pre-warms it for **full performance instantly — but it's expensive**.

Keyword: "no latency on first access after restore."
- **Data Lifecycle Manager (DLM)**: **automates** snapshot creation, retention, and deletion on a schedule. "Automate EBS backups" → DLM.
- **DLM vs AWS Backup:** DLM covers **EBS snapshots and EBS-backed AMIs only**. If the question spans **several services** (EBS + RDS + DynamoDB + EFS…) or wants central, cross-account backup policies, the answer is **AWS Backup** (see Section 35).

Encryption: the snapshot-copy dance

Encryption is sticky in the happy direction: **an encrypted volume produces encrypted snapshots, and encrypted snapshots produce encrypted volumes.** Everything downstream inherits it, with essentially no performance cost.

But **you cannot flip encryption on for an existing unencrypted volume in place**. The exam's required choreography:

```
unencrypted volume → snapshot it → COPY the snapshot (enable encryption
on the copy) → create a new volume from the encrypted copy → swap it in
```

Any question about "encrypt an existing EBS volume" is testing whether you know this **snapshot → copy-encrypted → restore** dance.

Encryption fine print:

- **What's covered:** data **at rest** on the volume, data **in transit between the instance and the volume**, all **snapshots** created from it, and all **volumes created from those snapshots**.
- **The crypto: AES-256** with **KMS keys**. EBS supports only **symmetric** KMS keys, never asymmetric ones.
- **Encryption by default** is a **per-Region account setting**. Once you turn it on, every new volume and snapshot copy in that Region is encrypted, and you **can't opt out for an individual volume**. The **instance type must support EBS encryption** (all current-generation types do). Signal: "*ensure ALL new EBS volumes are encrypted with no per-launch effort*" → **enable EBS encryption by default**.

EFS: the shared network folder

EFS (Elastic File System) is a managed **NFS** (Network File System — the classic Linux shared-folder protocol) file system. The personality traits:

- **Mounted by many instances at once, across multiple AZs.** This is the whole point — EBS is one-instance-one-AZ; EFS is the office-wide shared drive.
- **Linux ONLY.** THE trap: **Windows instances cannot mount EFS. Windows shared storage → FSx for Windows File Server** (SMB protocol, Active Directory integration). Read the OS in the question before answering "EFS."
- **Elastic**: it auto-grows and shrinks; **you pay per GB actually used**, no capacity planning.
- Pricier per GB than EBS — but you buy zero unused space, and lifecycle tiers claw back cost.
- **POSIX-compliant** (standard Linux file permissions and semantics). It scales from **GB to PB**, serves **thousands of concurrent instances**, and can be mounted by **ECS tasks, Lambda**, and **on-premises servers** (over Direct Connect/VPN).

- **Latency:** EBS has the **lowest latency**. EFS is a network file system, so expect **higher latency than EBS**. S3 has no low-latency file operations at all. EFS has **no native "present S3 objects as files"** integration; that's **FSx for Lustre**.

Dials you can turn (exam wants recognition, not depth):

Dial	Options	Pick when
Performance mode	**General Purpose** (default) / **Max I/O**	Max I/O = thousands of concurrent clients, tolerate higher latency (big data)
Throughput mode	**Elastic** (recommended) / Provisioned / Bursting	Elastic = auto-scales throughput, the modern default
Lifecycle	Standard → **Infrequent Access (IA)** → **Archive**	Auto-move cold files to cheaper tiers → big savings (~90%+ on IA)

The signature use case: a fleet of instances behind an Auto Scaling Group that must all see the same files — **user uploads, CMS content, WordPress wp-content**. "Multiple EC2 instances need shared access to the same files (Linux)" → **EFS**, every time.

Instance Store: the notepad glued to the desk

Instance Store is physical disk **inside the host machine itself** — no network hop at all. Consequences:

- **Fastest storage EC2 offers** — NVMe instance store reaches **millions of IOPS**, far beyond anything EBS can do.
- **EPHEMERAL** (temporary): data is **lost when the instance stops, hibernates, or terminates** — though it **survives a reboot** (a reboot stays on the same host; a stop can move you to a different one, and someone else's desk has a different notepad).
- **Cannot be detached, cannot be snapshotted** through EBS mechanisms, size is fixed by the instance type.

Legit uses: **cache, buffer, scratch space, temp processing files**, or data that's **replicated at the application layer** anyway (Cassandra nodes replicate to each other, so losing one node's local disk is survivable).

THE trap: "*highest possible IOPS AND data must persist*" — instance store wins the IOPS contest but flunks persistence. The answer is **io2 (Block Express) EBS**. The word **"persist"** disqualifies instance store instantly.

The five-storage lineup

Storage	One-liner
Instance Store	Fastest, physical, ephemeral — cache/scratch/replicated data only
EBS	Network block device, one instance (mostly), one AZ, persistent — boot volumes and databases
EFS	Shared NFS folder, multi-AZ, many Linux instances, pay-per-use — shared content
FSx	Managed third-party file systems — **Windows (SMB/AD)**, or Lustre for HPC
S3	Object storage over HTTP — not a disk at all; apps talk to it via API, effectively unlimited

Match the storage need to the service:

Need	Service
Temporary storage (scratch, buffers, queues, caches)	**Instance Store**
Multi-instance (shared) storage	**EFS**
Highly durable storage	**S3** or **EFS** (EBS gets there via snapshots stored in S3)
Static data / web content	**S3** (no web server needed) or EFS

Neither an instance store nor a standard EBS volume can be **shared across instances** (io1/io2 Multi-Attach is the narrow exception). Shared means EFS or FSx.

Question patterns

> "*Database needs 12,000 IOPS, cost-effective*" → **gp3** (under the 16,000 line, and gp3 provisions IOPS cheaply)

> "*Critical database needs 50,000 IOPS*" → **io1/io2** (past 16,000, only provisioned-IOPS SSD plays; sub-ms or 256k IOPS → io2 Block Express)

> "*Sequential processing of large log files, throughput-focused, cost-effective*" → **st1** (big + sequential = HDD; frequent access = st1, cold archive = sc1 — and neither can boot)

> "*gp2 volume needs more IOPS but not more storage*" → **migrate to gp3** (gp2 chains IOPS to size; gp3 provisions them independently and is ~20% cheaper)

> "*Fleet of Linux instances in an ASG must share uploaded files*" → **EFS** (many mounters + multi-AZ + NFS = the shared folder)

> "*Windows instances need shared storage with Active Directory integration*" → **FSx for Windows File Server** (EFS is Linux-only — THE trap)

> "*Fastest possible storage for temporary scratch/cache data*" → **Instance Store** ("temporary" unlocks it; millions of IOPS, ephemeral is fine here)

> "*High IOPS storage whose data must persist after the instance stops*" → **io2 EBS, not instance store** ("persist" disqualifies the notepad)

> "*Data volume must survive instance termination*" → **DeleteOnTermination = false** (root defaults true, extra volumes default false — check which one the question means)

> "*Encrypt an existing unencrypted EBS volume*" → **snapshot → copy with encryption enabled → create volume from the encrypted copy** (no in-place flip exists)

> "*Back up EBS volumes to another region for DR, automatically*" → **DLM-scheduled snapshots + cross-region snapshot copy** (snapshots live in S3 and copy across regions)

> "*Multiple instances must attach the same block volume in one AZ (cluster software)*" → **io1/io2 Multi-Attach** (up to 16 instances, same AZ — block storage, not a file system)

> "*Reduce cost of EFS for files untouched for months*" → **lifecycle policy to EFS IA/Archive** (same trick as S3 tiering, applied to the shared folder)

> "*Maximum IOPS for a 10 GiB io1 volume?*" → **500 IOPS** (50:1 ratio: 10 × 50)

> "*Combine several EBS volumes for more I/O performance than one volume can give; redundancy not required*" → **RAID 0** (striping. RAID 1 = mirroring for redundancy)

> "*Guarantee every new EBS volume in the Region is encrypted*" → **Enable EBS encryption by default** (Region setting; no per-volume opt-out)

> "*Change an in-use volume from gp2 to gp3 and add IOPS without downtime*" → **Elastic Volumes** (modify type/size/IOPS live)

> "*Automate backups of EBS, RDS, and DynamoDB from one place*" → **AWS Backup, not DLM** (DLM = EBS snapshots/AMIs only)

> "*Shared file storage for ECS tasks and on-prem Linux servers, POSIX permissions*" → **EFS**

Pocket card

Keyword	Answer
random I/O / database / boot volume	SSD (gp3 or io1/io2)

Keyword	Answer
large sequential / big data / throughput MB/s	HDD (st1/sc1) — never bootable
≤ 16,000 IOPS	gp3
> 16,000 IOPS	io1/io2
256,000 IOPS / sub-millisecond	io2 Block Express
frequent sequential (logs, ETL)	st1
coldest, cheapest EBS	sc1
gp2 mentioned at all	migrate to gp3 (~20% cheaper)
more IOPS without more storage	gp3 (independent provisioning)
multiple instances, one block volume, same AZ	io1/io2 Multi-Attach (max 16)
root disk survives termination	DeleteOnTermination = false
new extra volume unusable	format + mount + fstab
move volume across AZ/region	snapshot → (copy) → restore
cheap rarely-restored snapshots	Snapshot Archive (75% off, 24–72 hr restore)
recover deleted snapshot	Recycle Bin
instant full performance from snapshot	Fast Snapshot Restore (expensive)
automate snapshot schedules	Data Lifecycle Manager
encrypt existing volume	snapshot → copy-encrypted → restore
shared files, many Linux instances, multi-AZ	EFS (NFS, pay per GB used)
shared files, Windows / SMB / Active Directory	FSx for Windows (never EFS)
HPC parallel file system	FSx for Lustre
thousands of concurrent EFS clients	Max I/O performance mode
EFS throughput default	Elastic mode
cut EFS cost on cold files	lifecycle to IA / Archive
fastest storage / temp / cache / scratch	Instance Store (ephemeral)
instance store + stop/terminate	data gone (reboot survives)
high IOPS + must persist	io2 EBS, not instance store
Cassandra/replicated data, local speed	Instance Store is OK
io1 max IOPS	size (GiB) × 50; up to 64,000 (Nitro)
io2 IOPS ratio	500 IOPS per GiB
> 16,000 IOPS or > 250 MiB/s	Provisioned IOPS SSD (io1/io2)
magnetic "standard" volume	legacy previous-gen, avoid
change type/size/IOPS live	Elastic Volumes
stripe volumes for performance	RAID 0 (no redundancy)
mirror volumes for redundancy	RAID 1
all new volumes encrypted automatically	EBS encryption by default (per Region)

Keyword	Answer
EBS encryption keys	KMS symmetric only, AES-256
I/O during snapshot	continues normally
multi-service centralized backups	AWS Backup (DLM = EBS only)
POSIX shared FS for ECS / on-prem	EFS
lowest-latency storage (persistent)	EBS (EFS higher, S3 not for low-latency file ops)

You now know where an instance keeps its data — next up, S3 takes storage out of the instance entirely and into the world of objects, buckets, and eleven nines of durability.

↑ Back to index

Section 5: Elastic Load Balancing & Auto Scaling

The idea

Imagine a popular restaurant with one front door and a host standing at it. Guests don't wander in and pick their own tables — they walk up to the host, and the host seats them at whichever table is free. If a table breaks (wobbly leg, spilled soup everywhere), the host simply stops seating people there until it's fixed. Guests never know or care which table they got; they just know the restaurant's one address.

That host is a **load balancer**. Your servers (EC2 instances) are the tables. The load balancer gives clients **one single DNS name** (one front door), **spreads incoming traffic** across many servers, and constantly runs **health checks** — little "are you okay?" pings — so it stops sending traffic to any server that's sick. When the server recovers, traffic flows to it again.

Auto Scaling is the restaurant manager who watches how busy it is and **adds or removes tables** to match demand. Put them together and you get the classic AWS architecture: an Elastic Load Balancer (ELB) out front, an Auto Scaling Group (ASG) behind it, growing and shrinking with load, healing itself when instances die.

OSI layers in 60 seconds (you need this to pick the right LB)

Networking is described in "layers" — the OSI model — and the exam expects you to know layers 3, 4, and 7. Think of it as **mail**:

- **Layer 3 (Network)** = the **address on the envelope**. Just an IP address — which building does this go to?
- **Layer 4 (Transport)** = the envelope address **plus the apartment number** — the **port** (like TCP port 443 or UDP port 3000). Which building, AND which door inside it?
- **Layer 7 (Application)** = the mail carrier **opens the envelope and reads the letter**. Now you can see HTTP details: the URL path (`/api/orders`), headers, cookies, hostnames.

The layer a load balancer works at = how deep it looks into traffic = what decisions it CAN make. An L4 balancer never opens the envelope, so it can't route based on URL paths — it's blind to them. But not opening envelopes makes it *fast*. That trade-off is the whole ALB-vs-NLB story.

The four load balancers

	Layer	Protocols	Superpower	Exam smell
ALB (Application)	7	HTTP, HTTPS, gRPC	Smart routing by path/host/header	"route /api to...", microservices, WAF
NLB (Network)	4	TCP, UDP, TLS	Speed + **static IP**	"millions of requests", UDP, "whitelist IPs"
GLB (Gateway)	3	IP packets (GENEVE)	Sends traffic *through* appliance fleets	"third-party firewall/IDS/IPS"

	Layer	Protocols	Superpower	Exam smell
CLB (Classic)	4/7	Old	None — **legacy**	If CLB is an answer option, it's wrong

ALB — the smart host (Layer 7)

The ALB opens every envelope. Because it reads HTTP, it can make **content-based routing** decisions:

- **Path routing**: `/api/*` → the API target group, `/images/*` → the image servers
- **Host routing**: `app.example.com` → one group, `admin.example.com` → another
- **Header / query-string routing**: route by a custom header or `?version=beta`
- **HTTP-method routing**: `GET` vs `POST` / `PUT` to different groups
- **Source-IP routing**: route by the client's **source IP CIDR** (e.g., office range → internal group)

How the routing is wired: a **listener** only checks **protocol + port** (e.g., HTTPS:443). Behind it sit **listener rules** evaluated in **priority order** (lowest number first), with a **default rule** as the catch-all. Each rule's action forwards to a **target group**.

- **Weighted target groups**: one rule can forward to **several target groups with weights** (e.g., 90/10) — the ALB-native answer for **blue/green and canary** releases.
- **Slow start mode**: a newly registered target gets a **gradually increasing share** of requests over a ramp-up period instead of a full blast — for apps that need to warm caches/JIT first.
- The ALB runs **its own health checks** against each target group (HTTP path + expected code) — separate from EC2 status checks.

Key facts to lock in:

- Targets can be **EC2 instances, private IPs, Lambda functions, or containers (ECS)** — grouped into **target groups**.
- The ALB terminates the client connection and opens a new one to the target, so the target sees the ALB's IP. The real client IP is passed in the **X-Forwarded-For** header. "*App needs the client's IP behind an ALB*" → read X-Forwarded-For.
- **AWS WAF attaches to ALB** (a web application firewall inspects HTTP — it needs Layer 7, so it can't attach to an NLB).
- **Sticky sessions**: a cookie pins a user to the same target — for legacy apps that store session state locally. (Better design: externalize session to ElastiCache/DynamoDB, but "sticky sessions" is the quick fix answer.) If session state is **already shared** across instances, sticky sessions add nothing but uneven load — don't use them.
- **Dynamic port mapping** with ECS: multiple containers of the same app on one instance, each on a random port — the ALB tracks and routes to them. This is how you pack containers densely.
- ALB gives you a **DNS name, not a static IP**. THE trap: "*customers must whitelist a fixed IP*" → ALB **cannot** do this → put an **NLB** in front, or use **Global Accelerator**. (You **cannot attach an Elastic IP to an ALB** — the pattern is an **NLB with EIPs → ALB as its target**.)

NLB — the fast, no-questions host (Layer 4)

The NLB never opens envelopes. It sees IP + port and forwards, blazingly fast:

- Handles **millions of requests per second** with **ultra-low latency** — the "extreme performance" keyword answer.

- Speaks **any TCP or UDP** — gaming servers, IoT, MQTT, custom binary protocols. ALB can't do UDP; NLB can.
- **One static IP per AZ**, and you can assign your own **Elastic IPs**. This is THE whitelisting answer. With **BYOIP (Bring Your Own IP)** you can even use your **own already-trusted public IP range** as the NLB's Elastic IPs — partners don't have to update their allow-lists.
- **IP-address targets** can be **any private IP from an ENI** — including targets in a **peered VPC, even cross-Region**, or on-premises over Direct Connect/VPN.
- **No content-based routing** — the NLB (like the CLB) can't read paths, hosts, or headers.
- **Preserves the client source IP** by default — targets see the real client, no header tricks needed.
- **Required as the front of a PrivateLink endpoint service** — when you expose your app privately to other VPCs, an NLB (or ALB behind an NLB) sits in front. (More in Section 6.)

GLB — the security-appliance plumber (Layer 3)

The Gateway Load Balancer has exactly **one job**: transparently push all traffic **through a fleet of third-party security appliances** — intrusion detection/prevention systems (IDS/IPS), next-gen firewalls, deep packet inspectors — before it reaches your app. It's a "bump in the wire": traffic goes in one side, through the appliances, out the other, and nobody has to change IP addresses.

- Uses the **GENEVE protocol on port 6081** (a recognizable exam factoid).
- Keyword mapping is mechanical: "*inspect traffic with third-party / partner security appliances*" → **GLB**. Every time.

CLB — the retired host

Classic Load Balancer is the legacy generation (built for the long-retired **EC2-Classic** network). It has no modern superpower — no content routing and **no SNI** (one certificate per CLB, so many domains would need a multi-name **SAN** certificate). **On the exam, CLB is always the wrong answer.** Done.

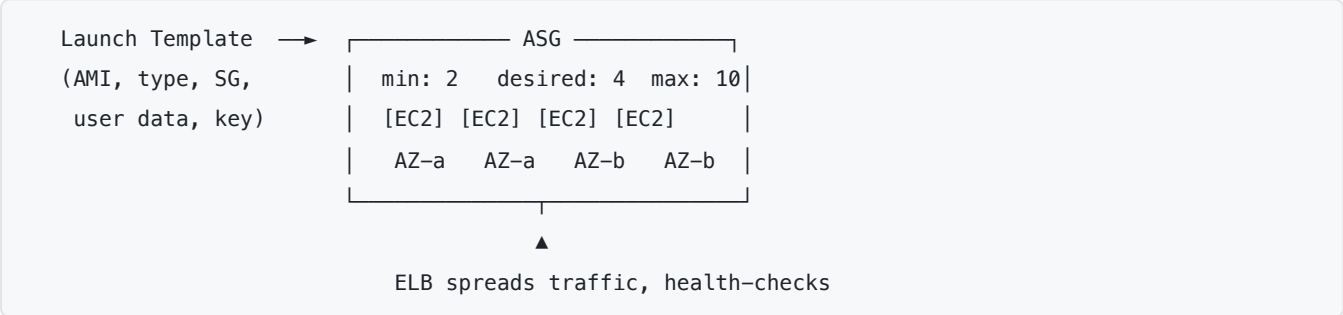
Shared ELB features worth points

- **Cross-zone load balancing**: without it, each LB node only sends traffic to targets in *its own* AZ — if AZ-a has 2 instances and AZ-b has 8, the AZ-a pair gets hammered. Cross-zone spreads evenly across ALL targets in ALL AZs. **ALB: on by default, free. NLB: off by default, inter-AZ data charges apply when enabled.**
- **Connection draining / deregistration delay**: when a target is removed (or ASG scales in), the LB stops NEW requests but lets in-flight requests **finish gracefully** (default 300s). "*Users get errors during scale-in*" → tune deregistration delay.
- **SNI (Server Name Indication)**: lets one load balancer host **multiple TLS certificates** — the client says which hostname it wants during the TLS handshake, and the LB serves the matching cert. "*Host many HTTPS domains on one ALB*" → SNI. ALB and NLB support it; CLB doesn't.
    - **SAN vs SNI**: **SAN (Subject Alternative Name)** = *many domain names inside ONE certificate* — adding a domain means **reissuing** the cert. **SNI** = *many separate certificates behind ONE listener* — add a domain by just adding its cert. "*Add domains without reissuing certificates*" → **SNI**.
- **TLS termination**: the LB decrypts HTTPS itself (certificate lives on the LB, usually from ACM) and can talk plain HTTP to targets — offloading crypto work from your instances. (TLS = Transport Layer Security, the encryption behind the "s" in https.)

- **Access logs**: per-request detail (time, client IP, latencies, request path, response code) — **disabled by default**; when enabled, the LB writes **compressed log files to an S3 bucket you choose**. "*Analyze traffic patterns / troubleshoot requests to the LB*" → enable **ELB access logs**.
- **What an ELB can't do** (popular wrong answers):
  - **Span Regions** — every load balancer lives in **one Region**; multi-Region distribution is **Route 53** or **Global Accelerator**.
  - **Throttle / rate-limit requests** — that's **API Gateway** (or WAF rate-based rules).
  - **Reduce latency from distant clients** — it balances, it doesn't move you closer; that's **CloudFront / Global Accelerator**.

Auto Scaling Groups

An ASG is a rule that says: "keep a fleet of instances alive, sized between these bounds, built from this recipe."



- **Launch template** = the recipe (AMI, instance type, security group, user data). Launch *configurations* are the legacy version — prefer templates:

	Launch Template	Launch Configuration (legacy)
Editable?	**Versioned** — create new versions, pick default/latest	**Immutable** — must create a new one and swap it onto the ASG
Mixed instance types + **On-Demand AND Spot** in one ASG	✅ yes	❌ no
Per ASG	one template (any version)	only **one** at a time
Status	current, recommended	legacy — no new instance types/features

  "*One ASG provisioning across multiple instance types with both On-Demand and Spot*" → **launch template** (mixed instances policy).

- **min / desired / max**: the ASG always steers actual count toward **desired**, clamped between min and max. Scaling policies work by changing desired.
- If an instance dies, the ASG **replaces it automatically** — self-healing for free.
- **What an ASG is NOT**: it **can't span Regions** (it spans AZs within one Region), it **doesn't distribute traffic** (that's the ELB), it **doesn't cut latency** by itself, and it's **too slow for bursts lasting seconds** — new instances take minutes to boot. Second-level spikes → **pre-provision**, buffer with **SQS**, or use **Lambda**.

- **Attaching an existing instance** to an ASG requires that the instance is **running**, its **AMI still exists**, it's **not in another ASG**, it's in **one of the ASG's AZs**, and (if the ASG has a load balancer) it's in the **same VPC** as the LB/target group.

Scaling policies — match the keyword

Policy	What it does	Exam keyword
Target Tracking	"Keep this metric at this value" (e.g., CPU at 40%) — AWS does the math	**Simplest**, "maintain X%"
Step Scaling	Add/remove N instances at metric thresholds you define	Fine-grained thresholds, **multiple adjustments** by alarm severity, sudden spikes
Simple Scaling	One CloudWatch alarm → **one fixed adjustment** (e.g., CPU > 70% → +1), then **waits out the cooldown**	Basic, single adjustment (older; Step/Target preferred)
Scheduled	Scale at known times	"**every Monday 9am**", "month-end batch", predictable
Predictive	Machine learning forecasts load and scales **ahead** of it	Recurring patterns, "proactively"

- **Scheduled actions** set new min/desired/max at a time — e.g., traffic climbs **Wednesday**, stays high **Thursday**, drops **Friday** → one action to scale out before Wednesday, one to scale in Friday. **Predictive scaling assumes a homogeneous ASG** (all instances of equal capacity).
- **Target tracking** is also the fix for an ASG that's **over-provisioned** — it scales in as well as out to hold the target.
- **Suspend / Resume processes**: temporarily pause ASG processes (`Launch`, `Terminate`, `HealthCheck`, `ReplaceUnhealthy`, `AZRebalance`, …) — e.g., to troubleshoot an instance without the ASG replacing it — then resume. (Or move the instance to **Standby**.)
- **Custom metrics**: **memory isn't a default EC2 metric** — to scale on memory, install the **CloudWatch agent** to publish it, then scale on that custom metric. For time-sensitive queue work, CPU is the wrong signal; scale on a queue metric like **`ApproximateAgeOfOldestMessage`** or backlog per instance.
- **Warm-up / cooldown** (default **300 seconds**, configurable): after a scaling action, the ASG waits before launching or terminating more — so it doesn't count a booting instance as "still overloaded" and over-scale. "*ASG launches too many instances in bursts*" or "*scales out and in several times within an hour*" → tune the **cooldown/warm-up**.

Lifecycle hooks: pause a launch or a termination

A **lifecycle hook** holds an instance in a **wait state** (default 1 hour, extendable) so you can run custom actions, then you **complete the lifecycle action** (or the timeout does) and it proceeds:

Transition	States
Scale-out (launch hook `EC2_INSTANCE_LAUNCHING`)	`Pending` → **`Pending:Wait`** → `Pending:Proceed` → `InService`

Transition	States
Scale-in / health-check failure (terminate hook `EC2_INSTANCE_TERMINATING`)	`Terminating` → **`Terminating:Wait`** → `Terminating:Proceed` → `Terminated`
Standby	`InService` → `EnteringStandby` → `Standby` → back via `Pending`
Detach	`InService` → `Detaching` → `Detached`

- **Launch hook** → install software, pull config, register with an external system **before** traffic arrives.
- **Terminate hook** → **download logs, take a final backup, drain work** before the instance is gone.

THE trap: "*Download log files from instances before they're terminated during scale-in*" → **lifecycle hook (terminating)**, NOT a scheduled action — scheduled actions only change capacity, they can't run anything on the instance.

THE health-check trap (this WILL be on your exam)

By default, an ASG uses **EC2 status checks only** — is the VM itself running? But an instance can be perfectly *running* while the app on it is crashed, hung, or returning 500s. The ASG shrugs: "VM's alive, looks fine to me."

THE trap: "*the load balancer marks instances unhealthy but the ASG never replaces them*" → **enable ELB health checks on the ASG**. Then the ASG trusts the LB's application-level verdict and terminates + replaces app-dead instances. Recommended setup when an ALB fronts the ASG: use the **ALB health check for both** routing and replacement.

The ASG + SQS pattern

Classic decoupling architecture: workers in an ASG consume jobs from an SQS queue. Scale on the queue metric — ideally a **custom metric of backlog per instance** (`ApproximateNumberOfMessagesVisible` ÷ instance count) with target tracking. "*Scale workers based on pending jobs*" → **SQS queue depth drives the ASG**, not CPU.

Termination policy (scale-in: who dies first?)

The default termination policy, step by step:

1. Pick the **AZ with the most instances** (to **keep AZs balanced**). If there's a tie, pick an AZ at random among them.
2. Within it, terminate the instance with the **oldest launch template/configuration**.
3. Still tied → the instance **closest to the next billing hour** (a **legacy** rule from hourly billing; with per-second billing it rarely matters).
4. Still tied → **random**.

You mostly just need to know: *default scale-in keeps AZs balanced and retires the oldest config first.* Instances in **Standby** or with **scale-in protection** are skipped.

The layered HA picture

Two different tools survive two different disasters:

```
Route 53 (DNS)  ──►  survives a REGION dying   (DNS failover — minutes, TTL-bound)
     │
     ▼
   ELB          ──►  survives an INSTANCE dying (in-path, near-instant rerouting)
     │
     ▼
   ASG          ──►  REPLACES the dead instance (self-healing capacity)
```

ELB is fast, in-path, within a region. Route 53 failover is DNS-based, slower, across regions. Serious architectures use both — that's the layered high-availability answer the exam loves.

Question patterns

> "*Route `/api/*` to one set of servers and `/images/*` to another*" → **ALB path-based routing** (only L7 sees URLs)

> "*UDP-based multiplayer game needs low latency and a static IP*" → **NLB** (ALB can't do UDP or static IPs)

> "*Inspect all traffic with a third-party firewall/IDS appliance fleet*" → **Gateway Load Balancer** (GENEVE 6081, bump-in-the-wire)

> "*LB marks instances unhealthy, but ASG doesn't replace them*" → **enable ELB health checks on the ASG** (default is EC2 status checks only)

> "*Payroll traffic spikes every month-end at a known time*" → **Scheduled scaling** (predictable = scheduled)

> "*Keep average CPU at 40% with minimal configuration*" → **Target Tracking** ("keep metric at value" + "simplest")

> "*Serve many HTTPS domains with different certificates on one ALB*" → **SNI** (multiple TLS certs, one listener)

> "*Corporate clients must whitelist fixed IP addresses for your load balancer*" → **NLB with Elastic IPs** (or Global Accelerator in front of an ALB)

"*App behind ALB needs the original client IP*" → **X-Forwarded-For header** (NLB would preserve it natively)

"*Scale workers based on the number of unprocessed jobs*" → **ASG scaling on SQS queue depth** (backlog per instance)

"*Millions of TCP requests per second, extreme performance*" → **NLB** ("millions" + "ultra-low latency" = L4)

"*Send 10% of traffic to the new version behind the same ALB*" → **Weighted target groups** on a listener rule

"*Partners allow-list our existing public IP range; front the app with it*" → **NLB with BYOIP Elastic IPs** (ALB can't take an EIP — put the ALB behind the NLB)

"*Add new HTTPS domains without reissuing the certificate*" → **SNI** with separate certs (SAN = one cert, reissue per domain; CLB has no SNI)

"*Capture client IPs, latencies, and paths for every request to the LB*" → **Enable ELB access logs** → **S3** (off by default)

"*Collect logs from instances before scale-in terminates them*" → **Lifecycle hook on termination** (`Terminating:Wait`), not a scheduled action

"*One ASG mixing several instance types with On-Demand and Spot*" → **Launch template** (launch configurations can't)

"*ASG scales out and in repeatedly within the hour*" → **Increase the cooldown** (default 300s)

"*Scale on memory utilization*" → **CloudWatch agent custom metric** + scaling policy (memory isn't a default metric)

"*Distribute traffic across two Regions with one load balancer / one ASG*" → **Not possible** — ELB and ASG are single-Region; use **Route 53** or **Global Accelerator**

> "*Traffic spikes last only a few seconds*" → **ASG is too slow** — pre-provision, buffer with SQS, or go serverless

Pocket card

Keyword	Answer
Path / host / header routing	ALB
HTTP, microservices, containers, Lambda target	ALB
WAF on a load balancer	ALB (L7 only)
UDP, extreme performance, millions req/s	NLB
Static IP / Elastic IP / whitelisting	NLB (or Global Accelerator)
PrivateLink endpoint service front	NLB
Third-party security appliances, GENEVE 6081	GLB
Classic Load Balancer	Wrong answer
Multiple TLS certs, one LB	SNI
Errors during scale-in	Deregistration delay (draining)
Uneven traffic across AZs	Cross-zone LB (ALB free/on, NLB paid/off)
Client IP behind ALB	X-Forwarded-For
"Keep CPU at X%" simply	Target Tracking
Known time spikes	Scheduled scaling
ML-forecasted scaling	Predictive scaling
ASG ignores app-level failures	Enable ELB health checks on ASG
Scale on job backlog	SQS queue depth metric
Instance-level failover	ELB (in-path, instant)
Region-level failover	Route 53 (DNS, slower)
Route by HTTP method / source IP CIDR	ALB listener rules
Rules evaluated how?	Priority order; listener = protocol + port only
Canary / blue-green on one ALB	Weighted target groups
New target needs gradual ramp-up	Slow start mode
Request-level logs	ELB access logs → S3 (off by default)
EIP on an ALB	Not possible → NLB (EIP) in front
Use own trusted IP range on LB	NLB + BYOIP
NLB IP targets	any private IP; peered VPC (even cross-Region)
Many domains, ONE certificate	SAN
Many certificates, ONE listener	SNI (not on CLB)

Keyword	Answer
LB across Regions	Not possible (single-Region)
Throttle / rate-limit requests	API Gateway, not ELB
Multiple types + On-Demand + Spot in ASG	Launch template
Launch configuration	Legacy, immutable
Single fixed adjustment, waits for cooldown	Simple scaling
Pause ASG replacing instances	Suspend processes / Standby
Run actions before launch completes / before terminate	Lifecycle hooks (`Pending:Wait` / `Terminating:Wait`)
Scale in/out several times per hour	Cooldown (300s default)
Scale on memory	CloudWatch agent custom metric
Time-sensitive queue scaling	`ApproximateAgeOfOldestMessage` / backlog per instance
Scale-in victim	AZ with most → oldest template/config → (legacy) billing hour → random
Attach existing instance	running, AMI exists, not in another ASG, same AZs/VPC
ASG across Regions / sub-second bursts	No

One layer down from the load balancer sits the network it all lives in — subnets, gateways, and firewalls — which is exactly where Section 6 (VPC) picks up.

↑ Back to index

Section 6: VPC — Virtual Private Cloud

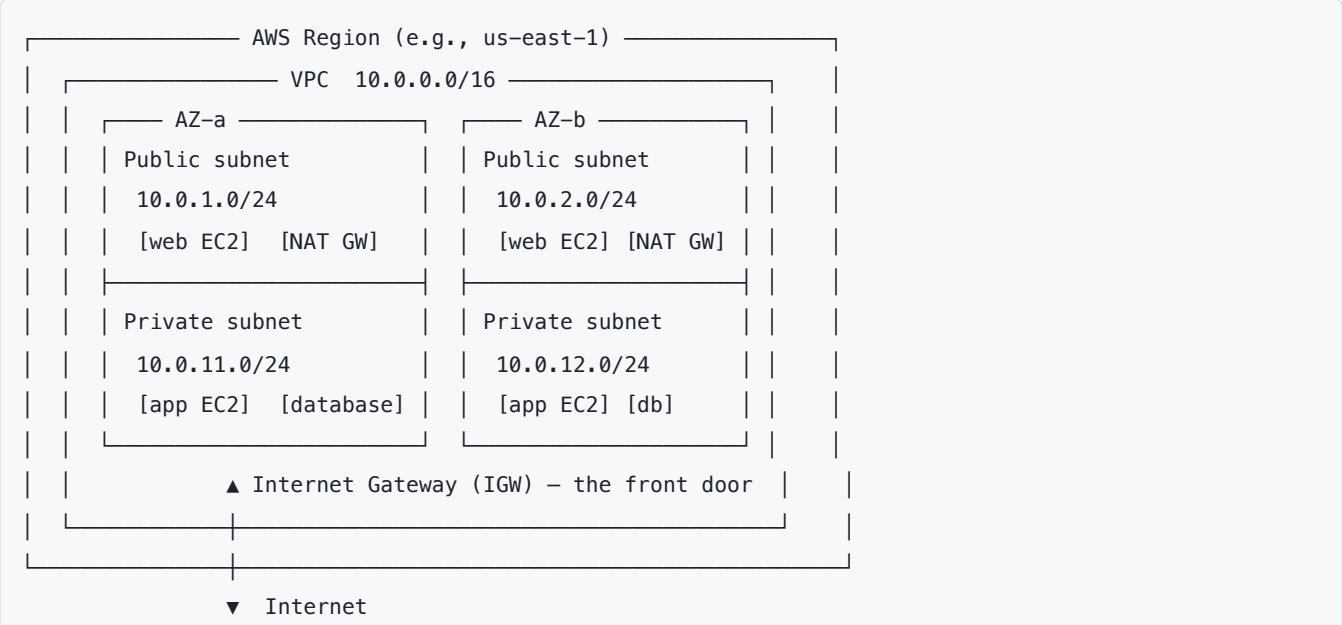
The idea

A VPC is your own **private, fenced-off patch of the AWS cloud** — a network that belongs entirely to you. Think of it as buying a plot of land in a huge city: you decide the street layout (IP ranges), you build rooms (subnets), you decide which rooms have doors to the street (internet access), and you hire guards to check who comes and goes (security groups and NACLs). Nothing gets in or out unless *you* built a door for it.

This is the **biggest topic on the SAA-C03 exam** — networking questions are everywhere, and they're very pattern-matchable once the mental model clicks. So let's build it.

Boxes in boxes

Everything in VPC-land nests:



- **Region > VPC > subnets > instances.** A VPC lives in ONE region; a **subnet lives in ONE Availability Zone** (so multi-AZ = multiple subnets).
- **Route tables are the road signs.** Each subnet has a route table saying "traffic for X goes via Y." Change the signs, change the network. A new subnet you don't explicitly associate is automatically tied to the VPC's **main route table**.

Public vs private: it's all about the route

- An **Internet Gateway (IGW)** is the VPC's **two-way front door** — traffic can go out AND the internet can come in (to public IPs). It has **two jobs**: be a **route-table target** for internet-bound traffic, and perform **NAT for instances with public IPv4 addresses** (the instance only knows its private IP; the IGW maps it to the public one).

- **THE definition** — memorize this exactly: a subnet is **public** if its route table has a route to the IGW (`0.0.0.0/0 → igw-xxx`). No IGW route = **private**. That's it. Not the name, not the IPs — the route.
- **"EC2 can't reach the internet" checklist** — all of these must be true: (1) an **IGW attached** to the VPC, (2) a **route to the IGW** in the subnet's route table, (3) a **public IP or Elastic IP** on the instance (nondefault subnets have auto-assign public IPv4 **off** by default), (4) **SG and NACL** allow the traffic. Troubleshoot in that spirit: *public IP/EIP first, then the route table*. Want `ping` to work? The SG/NACL must allow **ICMP** — ping isn't TCP, so opening port 80/22 doesn't help.
- **No IPv6-only VPC**: IPv4 is mandatory — every VPC needs an IPv4 CIDR, IPv6 is an optional add-on, and you can't remove all IPv4 CIDRs from an existing VPC.

NAT Gateway — the outbound-only door

Private instances still need to download patches and call external APIs. Enter the **NAT Gateway** (Network Address Translation). Picture a **receptionist who mails letters for everyone in the building**: she takes your letter, **swaps the return address for the building's front-desk address**, and **remembers who asked** — so when the reply comes back, she hands it to the right person. But a stranger can't walk up and mail a letter *IN* to you; if nobody inside asked for it, she bins it.

That's NAT: **private instances can initiate outbound to the internet; the internet can never initiate inbound.**

Lock in these facts:

- The NAT Gateway **lives IN a public subnet** — it needs the IGW route itself to reach the internet. Private subnets then route `0.0.0.0/0 → nat-xxx`.
- A NAT Gateway is **per-AZ**. THE trap: one NAT Gateway shared by all AZs = a hidden single point of failure — if its AZ dies, every private subnet loses internet. "*Make NAT highly available*" → **one NAT Gateway per AZ**, each private subnet routing to its local one.
- NAT Gateway is **managed, scales automatically (up to ~45–100 Gbps), no security groups on it**, pay per hour + per GB. It's **redundant inside its AZ** (AWS handles hardware failure within the AZ — the AZ itself is still the failure domain) and supports up to **55,000 simultaneous connections to each unique destination**.
- NAT is a dumb pipe: it **cannot inspect domains/FQDNs or block malicious URLs**. "*Allow outbound only to approved domains*" → **AWS Network Firewall** (see Observability below).
- **NAT Instance** = the legacy DIY version: an EC2 instance doing NAT. It CAN have a security group, supports **port forwarding**, can double as a **bastion host**, but doesn't scale on its own — you manage/patch/size it yourself, and — THE NAT-instance answer — you must **disable the source/destination check** on it (EC2 normally drops traffic not addressed to itself; a NAT forwards other people's traffic, so the check must go).
- IPv6 has no NAT (every IPv6 address is public). The IPv6 equivalent of "outbound-only" is the **egress-only Internet Gateway**. "*IPv6 instances need outbound internet but no inbound*" → egress-only IGW.

CIDR essentials

CIDR (Classless Inter-Domain Routing) notation like `10.0.0.0/16` means "the first 16 bits are fixed; the rest are yours" — /16 ≈ 65,536 addresses, /24 = 256.

- VPC CIDR can be **/16 (largest) to /28 (smallest)**.

- **AWS reserves 5 IP addresses in every subnet** (network, router, DNS, future, broadcast) — a /24 gives you 251 usable, not 256.
- **No-overlap rule**: VPCs you want to connect (peering, TGW, VPN to on-prem) must have **non-overlapping CIDRs**. Plan ranges up front; you can't peer 10.0.0.0/16 with 10.0.0.0/16.

The two guards: Security Groups vs NACLs

Two firewalls, different posts, different memories:

	Security Group (SG)	**Network ACL (NACL)**
Guards	The **instance** (its network interface)	The **subnet** boundary
State	**STATEFUL** — remembers	**STATELESS** — no memory
Rules	**Allow only** (default deny)	Allow **and DENY**
Order	All rules evaluated together	**Numbered**, lowest first, first match wins
Can reference	**Other SGs**	Only CIDR blocks

- **Stateful** = a receptionist **with memory**: if she let your request out, she automatically lets the reply back in. With SGs you never write a rule for return traffic.
- **Stateless** = a gate guard with **no memory**: he checked the outgoing request, but the reply is a brand-new stranger to him — it needs its **own explicit rule**. Replies come back on **ephemeral ports (1024–65535)**, so NACLs need an outbound (and inbound, for the reverse direction) allow for that range. THE trap: "*requests go out but responses never return*" → **NACL is missing the ephemeral-port outbound rule**.
    - The **client's OS picks** the ephemeral range: **Linux 32768–61000**; **Windows Server 2008+ 49152–65535**; Windows Server 2003 1025–5000; **ELB, NAT Gateway and Lambda 1024–65535** — which is why "allow 1024–65535" is the safe NACL answer.
    - Worked example — *SSH from your home IP to an instance*: SG needs **inbound 22** only (stateful); the NACL needs **inbound 22 AND outbound ephemeral ports** back to your IP. (RDP is the same story on **3389**.)
- Neither guard reads HTTP payloads: SGs, NACLs (and Route 53) **cannot stop SQL injection or XSS** — that's **WAF**.
- **SG referencing**: instead of IPs, an SG rule can say "allow port 3306 **from the app-tier's security group**." "*Only app servers may reach the database*" → **DB SG allows inbound from the app SG** — elegant, survives IP changes.
- SGs **cannot deny**. THE trap: "*block traffic from a specific malicious IP*" → **NACL DENY rule** (a SG physically has no deny button).

VPC Endpoints — private hallways to AWS services

Normally, a private instance calling S3 would go out through NAT → internet → S3. Wasteful and exposed. VPC Endpoints let you reach AWS services **without leaving the AWS network**:

	Gateway Endpoint	**Interface Endpoint (PrivateLink)**
Services	**S3 and DynamoDB ONLY**	Nearly **any** AWS service (+ SaaS)
Cost	**FREE**	Paid (per hour + per GB)

	Gateway Endpoint	Interface Endpoint (PrivateLink)
How	A **route-table entry**	An **ENI** (private network card) in your subnet
Reach	**VPC-local only** — NOT usable from peered VPCs or on-prem	**IS reachable** over peering / Transit Gateway / VPN

- "*Private EC2 needs to reach S3, cheapest / most secure way*" → **Gateway Endpoint**, NOT a NAT Gateway. (NAT charges per GB; Gateway Endpoints are free.) This question is practically guaranteed.
- Because Interface Endpoints work across peering/TGW/VPN, the **centralized endpoints pattern** puts them in one shared VPC for many VPCs to use. And yes — an **S3 *interface* endpoint exists** specifically for the "*on-premises servers need private S3 access*" case, since the Gateway one can't be reached from on-prem.
- Gateway endpoint setup = create it **and associate it with the VPC route table** ("*keep DynamoDB traffic off the public internet*" → DynamoDB gateway endpoint + route-table association). Everything else — **SQS, SNS, Kinesis, Secrets Manager, SSM, KMS…** — uses an **interface** endpoint.
- An interface endpoint is literally an **ENI with a private IP from your subnet**.
- **Endpoint policies** are a second lock alongside bucket policies: one endpoint policy can restrict *which* S3 buckets the VPC may reach, while the bucket policy (`aws:sourceVpce`) restricts *who* may reach the bucket.
- Endpoints are **regional** (no inter-region endpoint), connect you to **AWS services — not to other VPCs**, and **can't be created in Wavelength Zones**.

Connecting VPCs: three very different tools

VPC Peering — a 1:1 cable

A direct private link between exactly two VPCs (cross-account and cross-region work fine).

- **NOT transitive**: A↔B and B↔C does NOT give A↔C. Every pair needs its own peering + route-table entries.
- **No CIDR overlap allowed.**
- **Inter-region peering** works — a private connection between VPCs in different Regions over the AWS backbone. (An NLB can even register **IP targets** in a peered VPC across Regions.)
- **No edge-to-edge routing**: a peered VPC **can't borrow the other VPC's "edges"** — its Site-to-Site VPN / Direct Connect to on-prem, its IGW, its NAT device, or its gateway endpoint. Picture it:

  ```
  VPC B 10.0.0.0/16 ──peering── VPC A 172.16.0.0/16 ──VPN or DX── Corporate network
          └──────────────────── ✗ B cannot reach corporate through A ────────────┘
  ```

  So peering is **never the answer for on-prem connectivity**; B needs its own VPN/DX, or put everything behind a **Transit Gateway**.
- Cheap (free to create; you pay data transfer). Great for **2–3 VPCs**.
- The mesh math kills it: full mesh of n VPCs = **n(n−1)/2** connections. **10 VPCs = 45 peerings.** 25 VPCs = 300. Nobody manages that → …

Transit Gateway (TGW) — the hub

A regional **hub-and-spoke router**: attach VPCs, **Site-to-Site VPNs, and Direct Connect**, and everything can talk **transitively** through the hub — one attachment per VPC instead of a mesh.

- Share it **cross-account with AWS RAM** (Resource Access Manager).
- **Peer TGWs across regions** for a global network.
- The **only AWS service supporting multicast**.
- **Per-attachment route tables** let you segment: e.g., prod VPCs can't reach dev VPCs even though both hang off the hub.
- Keyword: "*many VPCs (and/or on-prem) need to communicate, minimal management*" → **Transit Gateway**. "Many" starts around 4+; at 25 it's not even a question.

PrivateLink — a service window, not a network marriage

Peering/TGW **merge networks** — everything can reach everything. PrivateLink instead opens **one service window**: the provider puts an **NLB** in front of their app and creates an **endpoint service**; consumers create an **interface endpoint** in their own VPC and reach *that one service, one-direction, and nothing else*.

- **Overlapping CIDRs are fine** (no routing between networks — it's just an ENI).
- Scales to **thousands of consumer VPCs** without route-table sprawl.
- Keywords: "*SaaS provider exposes a service to many customer VPCs*", "*partner must access ONE API only, not our whole network*" → **PrivateLink**.

Quick chooser: **2–3 VPCs, full network access → Peering. Many VPCs + VPN/DX, transitive → TGW. Expose one service, no network merge, CIDRs overlap → PrivateLink.**

Observability & admin access

- **VPC Flow Logs** capture traffic **metadata** — source/destination IP, port, protocol, **ACCEPT or REJECT** — to CloudWatch Logs or S3. Not packet contents. Use for "*why is this connection failing?*" (look for REJECTs → an SG/NACL is blocking) and security analysis. Need the **full packets**? That's **Traffic Mirroring** (copies inbound/outbound packets from an instance's ENI to another instance's ENI, an **NLB with a UDP listener**, or a **Gateway Load Balancer with a UDP listener**). Memory hook: *network traffic? → Flow Logs; application logs? → CloudWatch Logs.*
- **AWS Network Firewall** = the inline, **stateful** VPC firewall with **deep packet inspection** (Suricata-based IPS): filters traffic through the IGW, NAT GW, VPN or DX, does **FQDN/domain allow- and deny-lists**, and detects protocols like HTTPS regardless of port. It's deployed at the **VPC level (not per subnet)**, can't attach directly to an ALB, and can't by itself restrict S3 bucket access to one VPC (that's an endpoint + bucket policy). Full coverage in Section 22.
- **Network Access Analyzer** only **reports** unintended network access paths against your compliance rules — **no packet inspection, no blocking**.
- **Bastion host** = a DIY **EC2 jump box in a public subnet**: you SSH to it, then hop to private instances. You build, patch, and guard it. The SG chain: bastion's SG allows port 22 **from your corporate IP only**; private instances' SG allows 22 **from the bastion's SG**.
- **SSM Session Manager** = the modern answer: shell access through the Systems Manager agent — **no open port 22, no public IP, no bastion, every session logged** (CloudTrail/S3). THE tie-breaker: "*MOST secure way to administer private instances*" → **Session Manager**, not a bastion.

- **VPC Sharing (via RAM)**: one account owns the VPC, other accounts launch resources into its subnets — **many accounts, one network**, no peering needed. "*Central network team, application accounts deploy into shared subnets*" → VPC sharing. Precisely: the owner shares **subnets** (not the VPC object) with participant accounts **in the same AWS Organization**. RAM also shares **Transit Gateways, Route 53 Resolver rules and License Manager configurations**.
- **Legacy, for recognition only**: the old **VPC wizard** offered four layouts — single public subnet; public + private (NAT); public + private + hardware VPN; private-only + hardware VPN (there was **no "public-only + VPN"** option). **ClassicLink** linked pre-VPC **EC2-Classic** instances to a VPC; EC2-Classic is retired.

Question patterns

> "*Block all traffic from a specific IP address*" → **NACL deny rule** (security groups can't deny)

> "*Only the app tier may connect to the database*" → **DB security group allows inbound from the app tier's SG** (SG-to-SG reference)

> "*Private EC2 must access S3 privately at the lowest cost*" → **Gateway VPC Endpoint** (free — NAT Gateway is the expensive decoy)

> "*25 VPCs plus on-premises via VPN must all communicate*" → **Transit Gateway** (peering mesh = 300 connections; TGW is the hub)

> "*Expose one application to a partner VPC; CIDR ranges overlap*" → **PrivateLink** (NLB + endpoint service; overlap doesn't matter)

> "*Single NAT Gateway — make the design highly available*" → **One NAT Gateway per AZ**, each private subnet routing to its local one

> "*Private instances need to download OS updates*" → **NAT Gateway in the public subnet** + private route `0.0.0.0/0 → NAT`

> "*Most secure way for admins to access private instances*" → **SSM Session Manager** (no port 22, no bastion, fully logged)

> "*Determine why traffic to an instance is being rejected*" → **VPC Flow Logs** (metadata shows ACCEPT/REJECT; full packets = Traffic Mirroring)

> "*IPv6-only instances need outbound internet, no inbound*" → **Egress-only Internet Gateway** (IPv6's "NAT")

> "*NAT instance stopped forwarding traffic*" → **Disable the source/destination check**

> "*Outbound requests succeed but responses never arrive*" → **NACL missing outbound ephemeral-port (1024–65535) rule** (stateless!)

> "*New EC2 in a public subnet can't be reached from the internet*" → **check public IP/EIP first, then the IGW route** (nondefault subnets don't auto-assign public IPs)

> "*Instance is reachable on HTTP but ping fails*" → **allow ICMP** in the SG/NACL

> "*VPC B is peered with VPC A, which has a VPN to on-prem; B can't reach on-prem*" → **expected — no edge-to-edge routing**; give B its own VPN/DX or use a **Transit Gateway**

> "*Restrict private instances' outbound traffic to approved domains only*" → **AWS Network Firewall** (NAT Gateway can't filter FQDNs)

> "*Create an IPv6-only VPC*" → **not possible** — an IPv4 CIDR is always required

> "*Private subnet must reach SQS/SNS/Kinesis privately*" → **Interface endpoint** (gateway endpoints are S3/DynamoDB only)

Pocket card

Keyword	Answer
Public subnet definition	Route to IGW
Outbound-only internet for private subnet	NAT Gateway (in a public subnet)
NAT high availability	One NAT Gateway per AZ
NAT instance fix	Disable source/dest check
IPv6 outbound-only	Egress-only IGW
Reserved IPs per subnet	5
VPC CIDR size	/16 to /28

Keyword	Answer
Block a specific IP	NACL (deny)
Stateful / instance-level	Security Group
Stateless / subnet-level / ephemeral ports	NACL
Tier-to-tier access control	SG referencing another SG
S3/DynamoDB private + free	Gateway Endpoint
Endpoint reachable from on-prem/peered VPC	Interface Endpoint (PrivateLink)
2–3 VPCs, full access	VPC Peering (not transitive!)
Many VPCs + VPN/DX, transitive hub	Transit Gateway (RAM to share, only multicast)
One service, overlapping CIDRs, SaaS	PrivateLink (NLB + endpoint service)
Traffic metadata, accept/reject	Flow Logs
Full packet capture	Traffic Mirroring
Most secure admin access	SSM Session Manager
One network, many accounts	VPC Sharing via RAM
IGW's two jobs	Route target + NAT for public IPv4
Ping fails	Allow ICMP
Linux / Windows 2008+ ephemeral ports	32768–61000 / 49152–65535
Peered VPC using the other's VPN/DX/IGW	✗ No edge-to-edge routing
Peering across Regions	Inter-region VPC peering
Domain/FQDN egress filtering, deep packet inspection	Network Firewall (not NAT GW)
Report unintended access, no inspection	Network Access Analyzer
IPv6-only VPC	Not possible
Restrict which buckets the VPC can reach	VPC endpoint policy

You now own the network — next up are the roads leading into it from the outside world: Route 53, Direct Connect, and Site-to-Site VPN, plus the load balancers from Section 5 that stand at its public doors.

↑ Back to index

Section 7: Route 53

The idea

You type `www.example.com` into your browser. Your computer has no idea what that means — the internet runs on IP addresses, not names. So the first thing that happens, before a single byte of your website loads, is a lookup: "*what IP address is example.com?*"

DNS (Domain Name System) is the internet's phonebook. You give it a name, it gives you back a number, and then it's done. Route 53 is AWS's DNS service — and the single most important thing to understand about it is this:

Route 53 answers the question, then EXITS. It never carries your traffic.

Compare that to a load balancer, which is a **doorman standing in the path of every request** — every packet flows through it, forever. Route 53 is more like the receptionist in the lobby who points you to the right building and never sees you again.

That difference sets the two services at **different zoom levels**:

```
"Which SITE/REGION?"          "Which INSTANCE?"
      Route 53          →      Load Balancer (ELB)
   (phonebook —                   (doorman —
 out of the path)               in the path always)
```

- **Route 53 picks between REGIONS or sites** (us-east-1 vs eu-west-1, primary vs DR).
- **ELB picks between instances** inside a region.

And because clients **cache** DNS answers (controlled by **TTL — Time To Live**, the "how long may you remember this answer" setting), Route 53 failover is **slow** — clients keep using the old cached IP until TTL expires. ELB failover is **instant** because the doorman is in the path and just stops sending to the dead instance. This one fact decides several exam questions.

The standard architecture sandwich you'll see over and over:

```
User → Route 53 → picks a REGION → ALB in that region → picks an INSTANCE
        (DNS)                        (load balancer)
```

Record types

Record	Maps	Notes
A	name → IPv4 address	The bread and butter
AAAA	name → IPv6 address	"Quad-A" = IPv6

Record	Maps	Notes
CNAME	name → **another name**	**CANNOT exist at the root/apex** (`example.com` itself) — only on subdomains like `www.example.com`
Alias	name → **AWS resource**	AWS extension. **WORKS at the apex**, queries are **free**, and it **auto-tracks the resource's changing IPs**

Alias records point at ALBs, CloudFront distributions, S3 static websites, API Gateway, and more. AWS resources like ALBs have IPs that change constantly, so you can never hardcode an A record to one.

- An Alias comes in an **A flavor (IPv4)** and an **AAAA flavor (IPv6)**; a *non-alias* A record is for when you genuinely have a raw, fixed IP (e.g., an Elastic IP). An "alias CNAME at the apex" is still wrong — the apex rule bans CNAMEs, period.
- **MX** = mail exchange record (a priority + a mail server name) — the answer when the scenario is about *email* delivery for the domain.
- **DNSSEC is not a record type** — it's a set of protocols that cryptographically sign DNS answers. If an option lists "DNSSEC" as a record to create, it's a decoy. (Route 53 *does* support DNSSEC **signing** for public hosted zones as a feature.)
- **TTL** is set per record: the number of **seconds recursive resolvers may cache** the answer. Low TTL = faster failover/changes but more queries (cost); high TTL = cheaper, slower to change.

THE trap: "*point the root domain `example.com` at an ALB*" → **Alias record, never CNAME** (CNAME is illegal at the apex) **and never a plain A record** (the ALB's IPs change). Root domain + AWS resource = **Alias**. Every time.

Hosted zones

A **hosted zone** is a container for all the DNS records of one domain.

- **Public hosted zone** — answers queries from the whole internet.
- **Private hosted zone** — answers queries **only from inside VPCs you attach it to**. This is how you get internal DNS names like `db.internal.company.com` that resolve only within your network. Exam signal: "*DNS names resolvable only within the VPC*" → private hosted zone.

S3 static website on your own domain via Route 53 needs three things: a **registered domain**, **Route 53 as its DNS service** (public hosted zone), and an **S3 bucket named exactly like the domain/subdomain** (`www.example.com` bucket for `www.example.com`), pointed at with an Alias record.

Routing policies

This is the heart of the Route 53 exam material. Learn the **signal phrases**:

Policy	What it does	Exam signal phrase
Simple	One record, one answer, no health checks	The default; rarely the answer
Weighted	Split traffic by percentage you assign	"*A/B test*", "*send 10% to the new version*" — **canary**
Latency	Answers with whichever region is **fastest for that user**	"*best performance for global users*", "*lowest latency*"

Policy	What it does	Exam signal phrase
Failover	Primary + secondary; flips when **health check** fails	"*active-passive*", "*disaster recovery site*"
Geolocation	Routes by **where the user IS** — a hard rule	"*EU users must be served from EU*" (compliance), language-specific sites. **Needs a Default record** for unmatched locations
Geoproximity	Draws a map boundary between **YOUR resources**, adjustable with a **BIAS** dial	"*gradually shift/increase the traffic share to a region*", the literal word **"bias"**
Multi-Value	Returns **up to 8 healthy** answers at once	"*simple load balancing via DNS*" — a poor man's load balancer

Latency vs Geolocation — the classic confusion pair:

- **Latency** = a **performance** decision. "What's FASTEST for this user?" (A user in London might get served from us-east-1 if that's genuinely faster right now.)
- **Geolocation** = a **legal/content RULE**. "Users in Germany get the German site, period." Location is the rule, not the speed.

Ask yourself: does the scenario care about *speed* or about *where the user physically is*? Speed → Latency. Rule → Geolocation. Corollary: latency routing **does not guarantee** that users in the same country always land in the same region — network conditions change the answer.

Geoproximity's name tag: if the question says **"bias"** or talks about **expanding one region's share of traffic** by turning a dial, that's Geoproximity. Geolocation has no dial; Geoproximity is all dial. Example: servers in London and Frankfurt — a user in France lands on London by plain proximity; give Frankfurt a **positive bias** and its boundary grows, pulling more mainland-Europe users there (configured through **Traffic Flow**). Negative bias shrinks a region.

Route 53 only answers questions — it has no compute (it can't "run a Lambda" to decide) and it can't block **SQL injection/XSS** (that's WAF).

Active-active vs active-passive failover

	Active-active	Active-passive
Who serves	**All** resources, all the time — no primary, no secondary	**Primary** serves everything; secondary waits
How	Any policy *other than* Failover (e.g., **Weighted**, Latency, Multi-Value) with health checks — all records with the same name/type/policy are returned unless unhealthy	**Failover** policy — only healthy primaries are returned; secondaries only when **all** primaries are unhealthy
Signal	"*use all resources*", "*both regions serve traffic*"	"*standby / backup site waits for failure*"

THE trap: you can't build active-active out of a Failover policy's one-primary-one-secondary pair — the moment you see "primary/secondary", it's active-passive.

Health checks

Route 53 health checks are performed by a fleet of **global checkers** scattered around the internet. That creates one famous limitation:

Global checkers live on the public internet — they CANNOT see private endpoints (instances in private subnets, private IPs). The fix: create a **CloudWatch alarm** that monitors the private resource, then create a Route 53 health check **based on that alarm**.

THE trap: "*health check a private endpoint*" → global checkers can't reach it → **CloudWatch alarm-based health check**.

You can also build **calculated health checks** — combine up to 256 child checks with AND/OR/NOT logic into one parent check ("healthy only if at least 3 of 5 children are healthy").

Failover: who handles what

Failure level	Who fixes it	Speed
Instance dies in a region	**ELB** stops routing to it	Instant (in-path)
Whole region/site dies	**Route 53** failover policy	Slow-ish (DNS TTL caching)

One neat corollary: if your DR design is **a single instance per region**, there's nothing for an ELB to balance — Route 53 Failover pointing at the instances directly is enough, and you can **skip the ELB entirely** (saves money; the exam likes this).

Question patterns

> "*Point the apex/root domain example.com at an Application Load Balancer*" → **Alias record** (CNAME is forbidden at the apex; Alias is the AWS answer, free and auto-tracking).

> "*Active-passive setup: send traffic to the standby region only when the primary fails*" → **Failover routing policy + health check** ("active-passive" is the giveaway).

> "*Test a new app version by sending 10% of users to it*" → **Weighted routing** (canary = percentages = weights).

> "*Users in Germany must see the German-language site*" → **Geolocation** (user's location is a RULE, not a performance choice — and don't forget the Default record).

> "*Serve global users with the best possible performance*" → **Latency routing** ("fastest" = latency).

> "*Gradually increase the share of traffic going to the new region*" → **Geoproximity with bias** ("shift traffic share" / "bias" is Geoproximity's name tag).

> "*Health-check an endpoint that has a private IP inside a VPC*" → **CloudWatch alarm-based health check** (global checkers can't see inside your VPC).

> "*Return several healthy IPs and let clients pick, without a load balancer*" → **Multi-Value answer routing** (up to 8 healthy records).

> "*Both regions should serve traffic all the time, removing any that become unhealthy*" → **Active-active** (Weighted/Latency + health checks — not the Failover policy)

> "*Host a static website from S3 at www.example.com*" → **bucket named** `www.example.com` + Route 53 hosted zone + **Alias** record

> "*Which DNS record should be created for DNSSEC?*" → **none — DNSSEC isn't a record type** (it's a signing protocol)

> "*Multi-region app needs near-INSTANT failover; DNS caching delays are unacceptable*" → **Global Accelerator instead of Route 53** (anything in the request path beats a phonebook when clients cache old answers).

Pocket card

Keyword in question	Answer
Root/apex domain → ALB/CloudFront/S3	**Alias record**
name → name, NOT at apex	CNAME
"A/B test", "10% canary"	**Weighted**
"best performance", "lowest latency"	**Latency**
"active-passive", "DR site"	**Failover** + health check
"users in country X get X content", compliance	**Geolocation** (+ Default record)
"bias", "shift traffic share between regions"	**Geoproximity**
up to 8 healthy answers, DNS-level LB	**Multi-Value**
DNS only inside a VPC	**Private hosted zone**
health check a private endpoint	**CloudWatch alarm-based check**
combine many health checks	Calculated health check

Keyword in question	Answer
instant multi-region failover, no DNS lag	**Global Accelerator**
All resources serve, no primary/secondary	**Active-active** (Weighted/Latency + health checks)
Email for the domain	MX record
S3 website on own domain	Bucket name = domain + Alias
Shift Geoproximity boundary	Bias (via Traffic Flow)

Route 53 hands out addresses and steps aside — but when the exam demands *instant* global failover with fixed IPs, you need something that stays in the path, which is exactly where Section 8's Global Accelerator picks up.

↑ Back to index

Section 8: CloudFront & Global Accelerator

The idea

Here's the physics problem AWS can't wish away: **far away = slow**. Your app lives in us-east-1, your user sits in Singapore, and every request has to cross half the planet — hundreds of milliseconds of lag, packet loss on the choppy public internet, unhappy users.

AWS gives you **two opposite strategies** to fight distance, and the exam loves testing whether you can tell them apart:

1. **CloudFront: bring the CONTENT closer.** Cache copies of your files at **400+ edge locations** worldwide. Think of a **bookstore chain**: instead of every customer flying to one warehouse, the chain stocks the popular books on the shelves of local stores. First customer waits for delivery (cache miss); everyone after grabs it off the shelf (cache hit).
2. **Global Accelerator: make the JOURNEY faster.** Nothing is cached, nothing is copied. Instead, the user's traffic **enters the AWS private backbone at the nearest edge location** and rides AWS's own fiber the rest of the way — a **private highway** instead of the congested public internet. Same warehouse, much better road.

```
CloudFront:            Global Accelerator:
User → edge (CACHED!)  User → nearest edge → AWS backbone → your app
       done.                  (in the path, no caching, every packet)
```

Content closer vs journey faster. Hold on to that, and most questions solve themselves.

CloudFront (CDN — Content Delivery Network)

- **Origins** = where CloudFront fetches the real content: **S3 buckets, ALBs, or any HTTP server** (even outside AWS).
- **TTL (Time To Live)** = how long an edge keeps a cached copy. **Longer TTL → fewer trips to the origin → less origin load** ("reduce load on origin" = increase TTL / improve caching).
- The origin can also control cache duration with `Cache-Control: max-age` and `Expires` headers. Shorter TTLs suit dynamic content; CloudFront shines for **frequently downloaded static files** cached near users.
- **Cache behaviors** = per-**URL path pattern** settings (e.g., `/images/*` cached a day, `/api/*` not cached, each possibly to a different origin).
- **Invalidation** = force-expire cached content early (e.g. `/*` for everything). It **costs money** — the cheap habit is versioned filenames (`app-v2.js`) instead. Versioning also wins because it beats **corporate proxy/browser caches** that never hear about your invalidation, shows the exact version in **access logs**, lets you **serve different versions** at once, and makes **rollback** trivial.
- **Origin Shield** = an extra, centralized **caching layer in front of your origin** (one Region chosen as the shield). Edge caches miss → they ask the shield instead of the origin, so the origin sees far fewer requests. Signal: "*improve cache hit ratio / reduce origin load across many edge locations*".

- **CloudFront also accelerates DYNAMIC content** — even uncacheable requests benefit from riding AWS's network from the edge to the origin. Don't dismiss CloudFront just because the content isn't static.

Locking the bucket: OAC. If users can hit your S3 bucket directly, they bypass CloudFront (and your caching, and your paywall). **OAC (Origin Access Control)** makes the bucket private and grants CloudFront alone the right to read it.

THE trap: "*S3 content must be accessible ONLY via CloudFront*" → **OAC**. You'll see **OAI (Origin Access Identity)** as a distractor — it's the **legacy** mechanism (a special CloudFront identity you grant read access in the bucket policy); OAC is the modern answer.

Paid/private content:

Mechanism	Scope	Signal
Signed URL	**ONE file**	"give a customer access to a single video/file"; also when clients **don't support cookies** (and legacy **RTMP** distributions)
Signed Cookies	**MANY files**	"subscribers can access the whole premium library", HLS video segments, or "*don't want to change the URLs*"
S3 presigned URL	One S3 object, **direct from S3**	"temporary access to an S3 object" — but it **bypasses CloudFront** and needs a unique URL per file (doesn't scale to many files/users)

Keep the two locks apart: **signed URLs/cookies restrict WHO may view content**; **OAC/OAI restricts the PATH** (S3 reachable only through CloudFront). A private CloudFront site usually uses both.

Geo Restriction = allow-list or block-list entire **countries** at the edge. Signal: "*licensing agreements*", "*block users from country X*". (CloudFront isn't in your VPC — this happens at the edge.)

Customizing at the edge — three tools, increasing power (deep dive on the two function types in Section 13):

Requirement	Response headers policy	CloudFront Functions	Lambda@Edge
Add/override response headers (security headers, CORS)	✅ (no code)	✅	✅
Lightweight request manipulation (URL rewrite, redirect, header normalize)	❌	✅ (cheapest, sub-ms)	✅
Inspect **User-Agent** and choose **WebP vs JPEG**, rewrite bodies, call other services	❌	❌	✅

THE trap: a response headers policy **only manages response headers** — it can't make User-Agent-based decisions. Lambda@Edge also runs on the edge locations, so it's the "run code close to users" answer (e.g., Lambda@Edge + Kinesis for global clickstream collection).

THE trap: "*custom SSL certificate for a CloudFront distribution*" → the ACM certificate **must be in us-east-1**, no matter where anything else lives. Free exam point.

Field-level encryption = encrypt specific sensitive form fields (credit card number) **at the edge**, so only your final application — not even your origin infrastructure — can decrypt them.

Global Accelerator

- Gives you **2 static anycast IP addresses** — the same two IPs work worldwide, and they **never change**. Anycast means the internet automatically routes each user to the *nearest* AWS edge announcing those IPs. **Static IPs = the whitelisting answer**: when a client's firewall team demands fixed IPs to allow, CloudFront (thousands of shifting edge IPs) can't help; GA can.
- Handles **ANY TCP or UDP traffic** — not just HTTP. Signal words: **gaming (UDP!), VoIP, IoT, MQTT**.
- **No caching, ever.** GA is a road, not a warehouse.
- **Endpoints**: ALBs, NLBs, EC2 instances and Elastic IPs — in **one or multiple Regions** (signal: "*multi-Region application traffic routing*").
- **Instant failover (~30 seconds)**: GA sits **in the request path** and health-checks your endpoints, so when a region dies it reroutes immediately — **no DNS-cache lag**, because clients keep using the same two IPs the whole time. This is exactly the weakness of Route 53 failover (clients cache old DNS answers) solved.

THE comparison table

	CloudFront	**Global Accelerator**
Strategy	Content closer (cache)	Journey faster (backbone)
Protocol	HTTP/HTTPS	**Any TCP/UDP**
Caching	**Yes** — that's the point	**No** — never
Static IPs	No (many changing edge IPs)	**Yes — 2 static anycast IPs**
Failover	n/a (origin failover exists, but)	**In-path, ~30s, no DNS lag**
Use cases	Websites, video, APIs, downloads	Gaming, VoIP, IoT, whitelisting, instant multi-region failover

The three-way direction trap

The exam loves this triangle — read the **direction and protocol** of the traffic:

```
Global users DOWNLOAD/view content (HTTP)   → CloudFront
Global users UPLOAD to an S3 bucket         → S3 Transfer Acceleration
Global users use a TCP/UDP app (non-HTTP)   → Global Accelerator
```

(S3 Transfer Acceleration uses the same edge network, but for **uploads into S3** — "users worldwide upload large files to a central bucket" is its exact sentence. Pair it with **multipart upload** for big files, >100 MB.)

Transfer Acceleration vs Global Accelerator — both ride edge locations + the AWS backbone, but: Transfer Acceleration is **S3-only, HTTP(S), one bucket endpoint**; Global Accelerator fronts **your application** (ALB/NLB/EC2) for **any TCP/UDP** traffic and gives **static anycast IPs**. "Faster transfers to a bucket" → Transfer Acceleration; "faster/failover-proof access to an app" → GA.

Two easy-to-miss CloudFront features (practice-test lessons)

- **Origin group** = a **primary + secondary origin** pair: when the primary returns errors, CloudFront **automatically retries against the secondary**. THE answer to "configure CloudFront for high availability / origin failover." (Geo restriction next to it is the false twin — that's country blocking, not HA.) Failover triggers on the **HTTP status codes you choose**. An Auto Scaling group can't be an origin by itself (put an ALB in front; EC2 origins should span multiple AZs), and a **Lambda@Edge function can't be a member** of an origin group.
- **Price class** = which **edge locations** your distribution uses (all = best latency, fewer = cheaper). It's a **cost dial** — it has nothing to do with routing to origins. "Reduce CloudFront costs, tolerate higher latency in some regions" → price class.

Question patterns

> "*Serve a static website to users worldwide with low latency; the S3 bucket must not be publicly accessible*" → **CloudFront + S3 origin + OAC** (cache at edges, lock the bucket to CloudFront only).

> "*Paying subscribers should stream any video in the premium catalog*" → **Signed Cookies** (MANY files = cookies; ONE file = signed URL).

> "*Send a customer a link to download a single report*" → **Signed URL** (one file, one URL).

> "*Multiplayer game over UDP has high latency for global players*" → **Global Accelerator** (UDP rules out CloudFront instantly).

> "*Enterprise clients' firewalls require a small set of FIXED IP addresses to whitelist*" → **Global Accelerator** (2 static anycast IPs — the only global service that offers this).

> "*Multi-region app needs failover in seconds, without waiting for DNS caches*" → **Global Accelerator** (in-path health checks, same IPs before and after).

> "*Licensing forbids streaming the content in certain countries*" → **CloudFront Geo Restriction** (country-level block at the edge).

> "*Reduce the load on the origin server behind CloudFront*" → **Increase TTL / improve cache hit ratio** (more hits at the edge = fewer origin fetches).

> "*Users around the world upload large files into one S3 bucket, uploads are slow*" → **S3 Transfer Acceleration** (UPLOAD direction — not CloudFront).

> "*Attach a custom domain certificate to a CloudFront distribution*" → **ACM certificate in us-east-1** (always, regardless of your app's region).

> "*Private S3 videos served through CloudFront; links are S3 presigned URLs*" → **switch to CloudFront signed URLs/cookies** (presigned URLs bypass CloudFront)

> "*Serve WebP or JPEG depending on the browser's User-Agent*" → **Lambda@Edge** (response headers policy only sets headers)

> "*Many edge locations hammer the origin with cache misses*" → **Origin Shield**

> "*Users behind corporate proxies keep getting the old file after a deploy*" → **versioned file names** (proxies ignore CloudFront invalidations)

Pocket card

Keyword in question	Answer
global static content, low latency	**CloudFront**
"S3 only via CloudFront", private bucket	**OAC** (OAI = legacy distractor)
one paid file	**Signed URL**
whole paid library	**Signed Cookies**
block/allow countries	**Geo Restriction**
CloudFront custom cert	**ACM in us-east-1**
encrypt one sensitive field end-to-end	Field-level encryption
reduce origin load	Longer TTL / better caching
force-refresh cached files	Invalidation (`/*` , costs money)
UDP / gaming / VoIP / non-HTTP	**Global Accelerator**
fixed IPs for whitelisting	**Global Accelerator** (2 anycast IPs)
instant failover, no DNS-cache lag	**Global Accelerator**
global UPLOADS to S3	**S3 Transfer Acceleration**
temporary access to one S3 object (no CloudFront)	S3 presigned URL
extra cache layer in front of origin	**Origin Shield**

Keyword in question	Answer
cheap request manipulation at edge	CloudFront Functions
add security headers, no code	Response headers policy
User-Agent logic / WebP vs JPEG	**Lambda@Edge**
different settings per URL path	Cache behavior

Both of these services exist to get users to your data faster — but the data itself usually lives in a database, and how you make *that* fast and failure-proof is Section 9's RDS and Aurora.

↑ Back to index

Section 9: RDS & Aurora

The idea

Running your own database server means patching the OS, applying engine updates, taking backups at 2 a.m., and rebuilding everything when the hardware dies. **RDS (Relational Database Service) = AWS runs the database for you.** You pick an engine — **MySQL, PostgreSQL, MariaDB, Oracle, or SQL Server** — and AWS handles the machinery underneath.

The price of that convenience: **you get NO OS access**. You can't SSH in, can't install custom extensions at the OS level, can't tweak the engine binaries. If a question demands **custom engine configuration or OS-level access**, RDS is out — the answer is **database on EC2** (full control, all the toil) or **RDS Custom** (a halfway house for Oracle/SQL Server).

Now for **THE core distinction — the most-tested database fact on the entire exam.** RDS has two features that both involve "extra copies of your database," and they exist for completely different reasons. Think of your car:

- **Multi-AZ is the spare tire.** It exists for the day something goes wrong. You never drive on it during normal life.
- **Read Replicas are extra checkout lanes at the supermarket.** They exist to serve more customers at once, every ordinary day.

	Multi-AZ (availability)	**Read Replicas (performance)**
Replication	**Synchronous** (standby is always exactly current)	**Asynchronous** (slight lag)
Where	Standby in **another AZ**	**Same AZ, cross-AZ, or cross-REGION**
How many	1 standby	**Up to 15**
Can you read from it?	**NO — ZERO reads from the standby**	**Yes — that's the whole point** (read-only endpoints)
Failover	**Automatic, 60–120 seconds**, via **DNS flip** (same endpoint, new machine behind it)	**None automatic — manual promotion only**
Solves	AZ failure, hardware death	Read-heavy workloads, reporting

THE trap: "*use the Multi-AZ standby to serve read traffic*" → **impossible**. The standby is invisible until failover — the spare tire stays in the trunk.

THE trap (mirror image): "*read replica provides automatic failover*" → **no**. Promotion is a **manual** decision — which also makes a cross-region replica a decent **budget DR** plan, just not an automatic one.

And note: **production uses BOTH** — Multi-AZ for surviving failures, replicas for scaling reads. They're not competitors.

The full three-way: Multi-AZ vs Multi-Region vs Read Replicas

	Multi-AZ	**Multi-Region (e.g. Aurora Global DB)**	**Read Replicas**
Main purpose	**High availability**	**Disaster recovery + local read performance**	**Scalability** (reads)
Replication	**Sync** (non-Aurora); **async** (Aurora)	**Async**	**Async**
Active instances	**Primary only** (non-Aurora); **all active** (Aurora)	**All regions readable**	**All replicas readable**
Backups	Taken **from the standby** (non-Aurora); from **shared storage** (Aurora)	Per region	**None by default**
Span	≥2 AZs in **one region**	Each region can itself be Multi-AZ	Within an AZ, cross-AZ, or cross-Region
Engine upgrades	On the **primary** (non-Aurora); **all instances together** (Aurora)	Independent per region (Aurora: together)	**Independent of the source** (Aurora: together)
Failover	**Automatic** to standby (non-Aurora) / to a replica (Aurora)	Aurora can **promote a secondary region** to master	**Manual promotion** to a standalone DB (non-Aurora) / to primary (Aurora)

- **How Multi-AZ failover actually works:** RDS **flips the endpoint's CNAME** to the standby — the app keeps the same hostname. Automatic triggers: **loss of the primary AZ, loss of network, compute failure, storage failure**.
- **Read replicas** exist for **MySQL, MariaDB, PostgreSQL, Oracle, SQL Server — and Aurora**. Cross-region replication runs over an **encrypted, public-key channel**. Lag: **seconds on RDS**, **milliseconds (<1s) on Aurora**.
- **"Managed", not "fully managed" / not serverless:** RDS still makes **you** pick instance size, **backup window**, and **maintenance (patch) window**. "*No servers to manage at all*" points to DynamoDB / Aurora Serverless, not RDS.

Backups & encryption

- **Automated backups** → enable **PITR (Point-In-Time Recovery)**, retention **maximum 35 days**. Deleted when the instance is deleted.
- **Manual snapshots** → kept **forever**, until you delete them. **"Retain backups for 5 years / 7 years / compliance"** → **manual snapshot**, always — 35 days is the wall automated backups can't cross. The managed alternative for >35 days: an **AWS Backup plan** (e.g. daily backups, retained 90 days / 7 years) does the scheduling and expiry for you.
- **Snapshots can be copied cross-region** (cheap DR). Exporting snapshot data **to S3 is a manual "export" action** — automated snapshots don't land in S3 by themselves.
- **MySQL: use InnoDB, not MyISAM.** PITR and snapshot restore depend on crash-recoverable storage; **MyISAM has no reliable crash recovery**, so restores can be inconsistent. MyISAM only makes sense for heavy full-text search.

Encryption is a birth decision. You can only encrypt an RDS database **at creation**. To encrypt an existing unencrypted database, do the snapshot dance (this question is essentially guaranteed):

```
unencrypted DB → snapshot → COPY the snapshot (enable encryption) → restore from encrypted copy → repoint the app
```

You cannot flip encryption on in place, and you cannot encrypt the original snapshot directly — you encrypt the **copy**.

RDS Proxy

Databases hate being swarmed. Every connection costs memory, and **Lambda** — which can scale to thousands of concurrent executions in seconds — will happily open thousands of connections and knock your database over. **RDS Proxy = a connection pool** that sits in front of RDS, letting thousands of clients share a small, warm set of database connections.

- **Lambda + RDS = RDS Proxy. Always.** If those two appear in the same sentence with "too many connections" or "connection errors," you're done reading.
- Bonus: the proxy also **shrinks failover time** (it holds client connections and re-routes them, instead of everyone re-resolving DNS).
- Keyword generalization: **"database connection exhaustion"** from *any* fleet (Lambda, containers, autoscaled EC2) → **RDS Proxy**. It **preserves application connections during failover** and routes them to the new primary.

Also worth one neuron: **storage autoscaling** — RDS can grow its storage automatically when it runs low ("database keeps running out of disk with unpredictable growth" → enable storage autoscaling).

Security & monitoring

- **Network lockdown:** put the DB in a **private subnet** (app behind ALB + Auto Scaling), and make the **DB security group's inbound rule reference the app/EC2 security group** as its source — never IP ranges (SG referencing, Section 6).
- **IAM database authentication** (MySQL & PostgreSQL): token instead of password, over SSL, managed centrally in IAM — full detail in Section 1. (Raw STS tokens on their own are **not** the RDS answer.)
- **RDS events** (via SNS/EventBridge) cover **operational** events only — instance, parameter group, security group, snapshot changes. Reacting to **row-level INSERT/UPDATE/DELETE** needs engine-native features (triggers / stored procedures calling Lambda), not RDS event subscriptions.

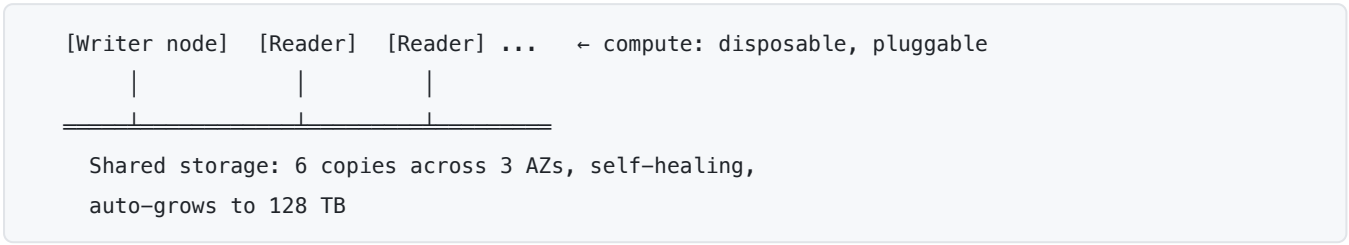
Enhanced Monitoring vs CloudWatch metrics:

	CloudWatch (standard)	**Enhanced Monitoring**
Source	The **hypervisor**	An **agent on the instance**
Sees	CPUUtilization, DB connections, FreeableMemory, Read/Write IOPS	**OS-level** metrics: **per-process CPU% and memory%**, RDS processes, RDS child processes, OS processes

THE trap: "*see how much CPU/memory each process or thread uses*" → **Enhanced Monitoring**. Keywords **"OS", "process", "thread", "child process"** = Enhanced Monitoring; plain CloudWatch can't see inside the instance.

Aurora

Aurora is AWS's own cloud-native engine, **compatible with MySQL and PostgreSQL** (your app connects the same way). The architectural trick: Aurora **separates compute from storage**.



Because every node plugs into the **same shared storage**, a failed writer is replaced by simply promoting a reader that already sees all the data:

- **Failover in under 30 seconds** (vs 60–120s for RDS Multi-AZ)
- **Up to 15 replicas with ~millisecond lag** (vs async seconds on RDS)
- **Storage auto-grows to 128 TB** — no provisioning
- **Two endpoints**: the **writer endpoint** (points at the current writer, survives failover) and the **reader endpoint** (load-balances across all replicas). Apps send writes to one and reads to the other — never hardcode instance addresses.
- Also: **continuous backup to S3 with PITR**, storage across **3 AZs**. Aurora is **not** an in-memory DB, and it handles **OLTP (ACID transactions) plus complex queries**. Aurora is **"provisioned"** (fixed instance sizes, predictable load) unless you choose Serverless (infrequent / intermittent / sporadic load).

Aurora endpoints cheat sheet:

Endpoint	Use it for
Cluster (writer)	All **writes** + DDL (and reads) — always points at the current primary
Reader	**Load-balanced read-only** traffic across replicas
Custom	Route a workload to a **specific subset** of replicas (e.g. bigger instances for analytics)
Instance	Connect to **one exact DB instance** — troubleshooting / fine-grained tuning

- **Aurora failover** = the cluster endpoint's **CNAME is flipped** to a promoted replica (not an A-record change). **No replica?** Aurora tries to create a new primary in the **same AZ first**, then another AZ (slower).
- **Parallel Query** pushes processing down to the storage layer to speed up **one big analytical query** — it does **NOT** load-balance traffic.
- **Babelfish for Aurora PostgreSQL** understands **T-SQL / SQL Server wire protocol** → move a **SQL Server app** to Aurora PostgreSQL with **minimal code changes**.

Aurora's special features — feature → scenario:

Feature	The scenario it answers
Serverless v2	**Spiky, unpredictable, or idle** workloads — dev/test databases used a few hours a day, capacity scales itself, pay for what you use

Feature	The scenario it answers
Global Database	**Cross-region DR with RPO ~1 second, RTO <1 minute**, plus local low-latency reads on other continents. (A plain cross-region read replica = slower replication, slower promotion — the budget distractor)
Cloning	**Copy-on-write instant copy** of a production database — a full-size staging/test copy in minutes without duplicating storage
Backtrack	**Rewind the database in place** (Aurora MySQL) — "*undo an accidental delete/bad migration FAST, without restoring from backup*", no new instance, no restore wait

RPO (Recovery Point Objective) = how much data you may lose; RTO (Recovery Time Objective) = how long until you're back. Aurora Global Database's "~1s / <1min" pair is a memorize-it number.

Question patterns

> "*Database must survive an AZ failure with no manual intervention*" → **Multi-AZ** ("survive failure" + "automatic" = spare tire, 60–120s DNS flip).

> "*Reporting/analytics queries are slowing down the production database*" → **Read Replica** (offload reads to a checkout lane — but if the queries are the *same ones repeatedly*, **ElastiCache** wins; replica is for **diverse/complex** queries).

> "*Lambda functions are exhausting database connections*" → **RDS Proxy** (Lambda + RDS = Proxy, every time).

> "*Encrypt an existing unencrypted RDS database*" → **snapshot** → **copy with encryption** → **restore** (encryption is creation-time only; this is the snapshot dance).

> "*Global application needs cross-region DR with less than 1 minute recovery*" → **Aurora Global Database** (RPO ~1s, RTO <1min — nothing else touches it).

> "*Dev database sits idle nights and weekends; minimize cost*" → **Aurora Serverless v2** (spiky/idle/unpredictable = serverless).

> "*Need a full copy of the production database for testing, quickly and cheaply*" → **Aurora Cloning** (copy-on-write = instant, near-free until it diverges).

> "*Developer ran a bad DELETE; restore the database to 5 minutes ago as fast as possible*" → **Aurora Backtrack** (rewind in place — no restore, no new instance).

> "*Compliance requires database backups kept for 7 years*" → **Manual snapshots** (automated backups cap at 35 days).

> "*Application needs OS-level access / a custom database engine*" → **EC2** (or RDS Custom for Oracle/SQL Server) — plain RDS gives no OS access.

> "*Monitor CPU and memory used by each process/thread on the RDS instance*" → **Enhanced Monitoring** (agent on the instance; CloudWatch only sees hypervisor-level metrics).

> "*Run analytics on a subset of Aurora replicas without touching the rest*" → **Aurora custom endpoint**.

> "*Speed up a single complex analytical query on Aurora MySQL*" → **Parallel Query** (not a load balancer — the reader endpoint does that).

> "*Migrate a SQL Server application to Aurora with minimal code changes*" → **Babelfish for Aurora PostgreSQL** (speaks T-SQL).

> "*Keep application connections alive through a database failover*" → **RDS Proxy** (holds connections, routes to the new primary).

Pocket card

Keyword in question	Answer
survive AZ failure, auto-failover	**Multi-AZ** (sync, 60–120s, DNS flip)
standby serves reads	**TRAP — never**
scale reads, offload reporting	**Read Replica** (async, up to 15, manual promotion)
replica auto-failover	**TRAP — promotion is manual**
same queries over and over	ElastiCache (not a replica)
Lambda + RDS connections	**RDS Proxy**
backups > 35 days / "retain X years"	**Manual snapshot**
restore to any point in time (≤35 days)	Automated backups / PITR
encrypt existing DB	**Snapshot → copy-encrypted → restore**
OS access / custom engine	EC2 or RDS Custom
failover <30s, 15 low-lag replicas, 128TB autogrow	**Aurora**
spiky / idle / unpredictable load	**Aurora Serverless v2**

Keyword in question	Answer
cross-region DR, RPO ~1s, RTO <1min	**Aurora Global Database**
instant prod copy for staging	**Aurora Cloning**
undo mistake fast, no restore	**Aurora Backtrack**
"OS", "process", "thread", "child process" metrics	**Enhanced Monitoring**
route to a subset of Aurora replicas	**Custom endpoint** (one exact node = instance endpoint)
one heavy analytic query faster	**Aurora Parallel Query**
SQL Server app → Aurora, minimal changes	**Babelfish**
MySQL engine for PITR/snapshots	**InnoDB** (not MyISAM)
backups >35 days, managed schedule	**AWS Backup plan** (or manual snapshots)

You now know how to keep the database standing and fast — but when the exam's read traffic is the *same* hot data thousands of times per second, even fifteen replicas is the wrong answer, and that's where caching with ElastiCache takes over.

↑ Back to index

Section 10: DynamoDB

The idea

Picture a coat-check counter at a giant stadium. You hand over your coat, you get ticket **#48291**. Later you hand back the ticket and — instantly — your coat appears. The coat-check clerk never *searches* the racks; the ticket number tells her exactly which hook to walk to. It doesn't matter if the stadium holds 500 people or 5 million: one ticket, one hook, one grab. **That's DynamoDB.**

DynamoDB is AWS's **serverless NoSQL database**. Spell that out:

- **Serverless** = you never see, patch, size, or connect to a server. No instance types, no connection pools, no maintenance windows. You just call an API.
- **NoSQL** = data lives as **items** (like rows) with flexible attributes (like columns, but each item can have different ones). No joins, no rigid schema.

Its superpower is the promise on the tin: **single-digit millisecond latency at ANY scale**. Ten requests per second or ten million — same speed. That's the coat-check trick: every lookup goes straight to a hook via a key, never a search.

DynamoDB vs RDS — the first fork in every question

Scenario says...	Pick
Joins, complex queries, existing SQL app, relational schema	**RDS / Aurora**
Massive scale, serverless, key-value lookups, millisecond latency, unpredictable growth	**DynamoDB**

THE trap: "*migrate with no code changes from MySQL*" → that's **RDS/Aurora**, never DynamoDB. NoSQL means rewriting queries.

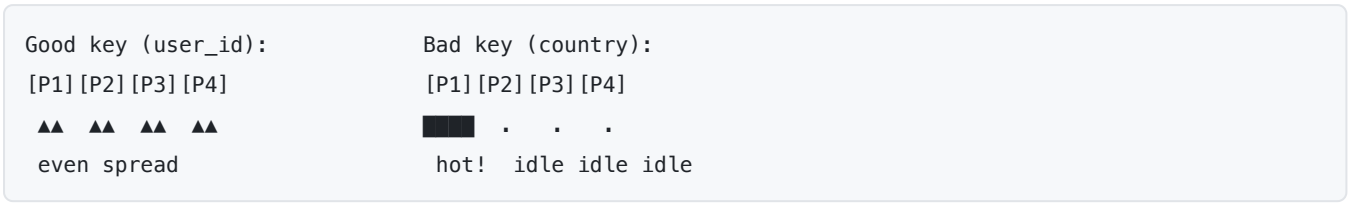
- **Data model: key-value AND document** store (items can hold nested JSON-like attributes). It is **not** an in-memory DB — the in-memory layer is **DAX**.
- **Why it scales where RDS struggles:** RDS pays for normalization, ACID guarantees and joins across tables; DynamoDB has a **flexible schema**, and a **composite primary key** (partition + sort) keeps related items together and spreads data over more partitions.
- **Legacy: SimpleDB** is the old AWS NoSQL store with per-table request/storage limits — if it appears, the modern answer is DynamoDB.

Keys and hot partitions

Every table needs a **partition key** (the coat-check ticket — determines which physical partition stores the item), plus an optional **sort key** (orders items *within* a partition, so one customer can have many orders sorted by date).

DynamoDB spreads data across partitions by hashing the partition key. If everyone's key is `country = "USA"`, all traffic slams one partition — a **hot partition** — and you get throttled while the rest of the table sits idle.

Rule: pick a high-cardinality partition key (many distinct values: `user_id`, `order_id`), not a low-cardinality one (`status`, `country`, `date`).



Capacity modes — same logic as EC2 pricing

- **Provisioned**: you declare read/write capacity up front. **Cheaper for steady, predictable traffic.** Add **auto scaling** to flex within bounds. (This is your Reserved-Instance instinct.) Auto scaling is driven by **Application Auto Scaling**, covers the **table and its GSIs**, and is **not on by default** for tables created via API/CLI — you configure it.
- **On-Demand**: pay per request, no planning. **Pick for spiky, unpredictable, or brand-new workloads** — flash sales, new apps with unknown traffic. (This is On-Demand EC2 instinct.)

THE trap: "*app gets throttled during unpredictable traffic spikes*" → switch to **On-Demand mode**. Don't over-provision.

RCU / WCU — the exam arithmetic

Unit	Buys you
1 RCU	**1 strongly consistent** read/sec, item ≤ 4 KB — or **2 eventually consistent** reads/sec
1 WCU	**1 write**/sec, item ≤ **1 KB**

Eventual consistency is half price because it may serve a copy that's a heartbeat stale. **"Must always read the latest write"** → request a **strongly consistent read** (default is eventual).

Feature zoo → scenario matcher

Feature	The one-liner
DAX (DynamoDB Accelerator)	In-memory cache in front of DynamoDB: **microsecond** reads, **API-compatible = ZERO code changes**, works **only** with DynamoDB. (Need caching for anything else, or need code-level flexibility → ElastiCache.)
Global Tables	**Multi-region ACTIVE-ACTIVE**: users read *and write* locally in every region, DynamoDB replicates. Requires **Streams enabled**. Contrast: **Aurora Global Database = one writer region** (active-passive writes).
Streams	A **change feed**: every insert/update/delete recorded for **24 hours**, typically triggering **Lambda**. Keyword: "*react to item changes.*"
TTL (Time To Live)	Put an expiry timestamp attribute on items → DynamoDB **auto-deletes them for FREE** (no WCU). Deletion happens within **~48 hours** of expiry, not the exact second. Classic: session data, temp tokens.
GSI (Global Secondary Index)	Query the table by a **different attribute** (e.g., by email when the key is user_id). Can be added **ANYTIME**, has its **own capacity**.
LSI (Local Secondary Index)	Alternate *sort* key, same partition key — but can be created **at table creation ONLY**. **THE trap:** if the table already exists, LSI is the wrong answer; pick **GSI**.

Feature	The one-liner
Transactions	**ACID** across multiple items/tables — all-or-nothing. Costs **2x** the capacity. Keyword: "*atomic*", "*all succeed or none.*"
PITR (Point-In-Time Recovery)	Continuous backup, restore to any second in the last **35 days**.
On-demand backup	**Full backups kept until you delete them** — long-term archiving/compliance (PITR stops at 35 days).
Gateway VPC endpoint	Private EC2 → DynamoDB **without the internet or NAT**: gateway endpoint + **route table entry**, free. (Gateway endpoints exist only for **S3 and DynamoDB**; everything else uses interface endpoints.)
Export to S3	Get table data into S3 for analytics — native **export to S3** (from PITR, no RCU) or the **AWS Glue DynamoDB export connector** in an ETL job.

Two more exam staples

The serverless poster-child stack — memorize the chain:

```
User → CloudFront → API Gateway → Lambda → DynamoDB
        (CDN)       (front door)   (code)    (data)
```

Any question asking for a "fully serverless web/API architecture" is assembling this.

More feature details the exam pokes at:

- **DAX** handles **cache population and invalidation for you** and absorbs **hot-key reads** (the same items hammered repeatedly). DAX = **microseconds**; don't confuse with Redshift's "sub-second" analytics.
- **Global Tables** = one **replica table per Region**, managed as a single unit, **multi-master**. **GSIs are not a cross-region feature** — a GSI never replicates data to another region; that's Global Tables' job.
- **Streams** = an **ordered** change feed. Uses: **Lambda triggers, cross-region/table replication, auditing**. Non-Lambda consumers can read Streams with **KCL via the DynamoDB Streams Kinesis Adapter**. KCL shard iterators "expire immediately"? → KCL's **DynamoDB lease table** is short on write capacity → **increase its WCU**.
- **AppSync + DynamoDB** = real-time / collaborative apps (GraphQL subscriptions). **Session state** works in DynamoDB *or* ElastiCache — both are valid answers.

400 KB item limit. An item can't exceed 400 KB. Storing images/documents? **Put the object in S3, store the S3 pointer (key/URL) in DynamoDB.** This S3-plus-pointer pattern is a recurring correct answer.

Question patterns

> "*Flash sale causes unpredictable spikes; table throttles*" → **On-Demand capacity mode** (pay-per-request absorbs spikes) "*Reduce read latency to microseconds without changing application code*" → **DAX** (API-compatible cache, DynamoDB-only) "*Global users need low-latency reads AND writes in every region*" → **Global Tables** (active-active; Aurora Global has only one writer) "*Run custom logic whenever items are added or modified*" → **DynamoDB Streams + Lambda** (24-hour change feed

> triggers functions) "*Automatically remove session records after 30 minutes, at no cost*" → **TTL** (free auto-delete, ~48h window) "*Table keyed on user_id, but app must also query by email*" → **GSI** (new query attribute, addable anytime — LSI is creation-time-only bait) "*Debit one account and credit another — both or neither*" → **DynamoDB Transactions** (ACID, 2x capacity cost) "*Store 2 MB documents per record*" → **S3 for the object, DynamoDB holds the pointer** (400 KB item limit) "*Application must always see the most recent write*" → **Strongly consistent read** (1 RCU vs 2 eventual reads) "*Reporting needs joins across many tables*" → **RDS/Aurora**, not DynamoDB (relational = SQL) "*EC2 in a private subnet must reach DynamoDB without traversing the internet*" → **Gateway VPC endpoint + route table entry** (free; not NAT) "*Export DynamoDB data to S3 for analytics*" → **DynamoDB export to S3 / AWS Glue DynamoDB export connector** "*Keep DynamoDB backups for years*" → **On-demand backups** (or AWS Backup) — PITR only covers 35 days "*KCL app reading DynamoDB Streams: shard iterators expire immediately*" → **Increase WCU on the KCL lease table** "*Real-time collaborative app with live updates*" → **AppSync + DynamoDB**

Pocket card

Keyword	Answer
Serverless NoSQL, key-value, any scale	DynamoDB
Joins / complex SQL	RDS or Aurora instead
Spiky / unknown traffic	On-Demand mode
Steady, predictable traffic	Provisioned (+ auto scaling)
Microseconds, no code change	DAX
Multi-region active-active	Global Tables (needs Streams)
React to item changes	Streams → Lambda
Auto-expire items free	TTL
Query by another attribute, existing table	GSI (LSI = creation-time trap)
Atomic multi-item	Transactions (2x cost)
Restore to any second, 35 days	PITR
Item > 400 KB	S3 + pointer
1 RCU	1 strong or 2 eventual reads ≤ 4 KB
1 WCU	1 write ≤ 1 KB
Private access, no internet	Gateway VPC endpoint
Long-term archive backups	On-demand backup
DynamoDB → S3	Export to S3 / Glue export connector
Hot-key reads	DAX
GSI in another region	TRAP — use Global Tables
Consume Streams outside Lambda	KCL + Streams Kinesis Adapter

DynamoDB gives you millisecond answers forever — but when even milliseconds are too slow, you reach for the in-memory layer, and that's where ElastiCache (Section 11) picks up the story.

↑ Back to index

Section 11: ElastiCache

The idea

Imagine a librarian who keeps getting asked the same question: "*What time do you close?*" The first time, she walks to the back office, checks the policy binder, walks back — thirty seconds. After the hundredth ask, she tapes a sticky note to the front desk: **"We close at 9."** Now every answer takes half a second. She didn't make the binder faster — she stopped consulting it for questions she'd already answered. **That sticky note is a cache.**

A **cache** keeps the answers to repeated questions in **RAM** (memory) instead of making the database work them out again. RAM is roughly **1000x faster** than a disk-based database query — your response times drop from milliseconds to **microseconds**, and your database stops sweating.

ElastiCache is AWS's managed cache service. It runs one of two open-source engines for you — **Redis** or **Memcached** — handling patching, monitoring, and failover. One crucial fact up front: **ElastiCache REQUIRES application code changes.** Your app must be written to check the cache first. (Contrast with **DAX**, which is API-compatible with DynamoDB and needs **zero** code changes — a distinction the exam adores.)

The flow your app implements:

```
App request
    |
    ▼
[Check ElastiCache] ──hit──► return in MICROSECONDS ✔
    |
   miss
    |
    ▼
[Query database] ──► get result ──► write into cache ──► return
                                    (next asker gets the sticky note)
```

Redis vs Memcached — the only table you need

Capability	Redis	Memcached
Persistence (survives restart)	✓	✘
Replication + **Multi-AZ automatic failover**	✓	✘
Backup & restore	✓	✘
Sorted sets (leaderboards)	✓	✘
Pub/sub messaging	✓	✘
Multi-threaded, dead-simple	✘	✓
HIPAA-eligible (classic exam answer)	✓	✘

Capability	Redis	Memcached
Redis AUTH password + in-transit encryption	✓	✘
Cross-region replicas (**Global Datastore**)	✓	✘
Auto Discovery of nodes via one Configuration Endpoint	(cluster-mode endpoint instead)	✓

The rule of thumb: if the question asks for **ANY** grown-up feature — durability, failover, backup, leaderboard, pub/sub — the answer is **Redis**. If it says "*simplest possible cache, data loss is acceptable, multithreaded*" → **Memcached**. Redis wins about 90% of exam appearances.

When to use ElastiCache at all: repeated reads, a **database bottleneck**, or **session storage** — needing **sub-millisecond** latency. Typical use cases: caching, session stores, gaming, geospatial, real-time analytics, lightweight queuing. It *is* a key-value store, but the exam treats it as a **cache in front of a database**, not the primary system of record (for a durable key-value primary store, think DynamoDB).

Redis extras worth a neuron:

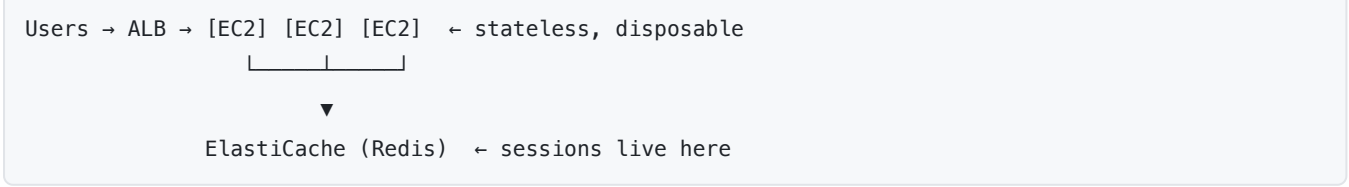
- **Cluster mode** = data **sharded** across multiple node groups for write/memory scale.
- **Global Datastore** = write in one region, read from **up to 2 cross-region replica clusters** — low-latency global reads and a **DR** option (promote a secondary).
- **Securing Redis:** create the cluster with `--transit-encryption-enabled` and `--auth-token` → clients must present the **Redis AUTH** password over TLS. **At-rest encryption** only protects data on disk/backups — it does **not** authenticate clients. (Newer Redis 7+ also supports IAM auth, but **Redis AUTH** is the classic exam answer.)
- **Memcached Auto Discovery:** clients connect to a **single Configuration Endpoint** and automatically learn about nodes being added/removed.
- **Not serverless — classically:** ElastiCache (like RDS and Neptune) was node-based; **ElastiCache Serverless** now exists, but in "fully serverless stack" questions DynamoDB/DAX is still the safer pick.

Free question alert: "*gaming leaderboard with real-time ranking*" → **Redis sorted sets**. Sorted sets keep members ordered by score automatically — top-10 lookups are instant. If you see "leaderboard," you're done reading.

The session store pattern

Picture a web tier behind an **ALB and Auto Scaling Group**. A user logs in; server #3 remembers her session — in its own memory. Then the ASG scales in, terminates server #3, and she's suddenly logged out. That's the exam giveaway phrase: **"users are logged out when instances are terminated/scaled in."**

The fix: make the web tier **stateless** by moving sessions to a shared, fast store outside the instances — **ElastiCache for Redis** (or DynamoDB). Any server can now handle any request.



Caching strategies — when do you write the sticky note?

- **Lazy loading** (cache-aside): populate the cache **on a read miss**. Like writing a cheat sheet *during* the exam — you only jot down answers to questions actually asked. Efficient (only hot data is cached), but data can be **stale** if the database changes underneath, and every miss pays a penalty.
- **Write-through**: update the cache **on every WRITE** to the database. Cache is **never stale**, reads always hit — but you do double writes and cache plenty of data nobody ever reads.
- **TTL** (time-to-live) on cached entries is the middle ground: stale data expires automatically after N seconds.

THE trap: "*stale data is unacceptable*" (prices, inventory) → **write-through**, not lazy loading.

The read-offloader triangle

Three services all claim to "reduce database read load" — the scenario tells you which:

Read pattern	Answer
Repeated, identical reads (same product page 10,000x)	**ElastiCache**
Diverse / complex / analytical queries (each one different — caching can't help)	**Read Replica**
DynamoDB + **"no code changes"**	**DAX**

THE trap: "*month-end reporting queries slow down the app*" → those queries are all *different*, so a cache never hits. Route them to a **Read Replica**.

Question patterns

> "*Popular product pages generate thousands of identical reads that overload RDS*" → **ElastiCache** (repeated reads belong in memory) "*Real-time gaming leaderboard with player rankings*" → **Redis sorted sets** (ranking is built in — free points) "*Users get logged out when the Auto Scaling Group terminates instances*" → **store sessions in ElastiCache for Redis** (make the web tier stateless) "*Cache must survive a node failure without data loss*" → **Redis** (replication + Multi-AZ failover; Memcached has neither) "*Simplest cache, multithreaded, losing cached data is fine*" → **Memcached** (the only time it wins) "*Cached prices must never be out of date*" → **write-through** (cache updated on every write; lazy loading can serve stale) "*Long analytical/reporting queries slow the production database*" → **Read Replica**, not a cache (diverse queries never hit cache) "*Speed up DynamoDB reads without modifying the application*" → **DAX** (ElastiCache would require code changes) "*Publish/subscribe messaging between app components using the cache layer*" → **Redis** (pub/sub is Redis-only) "*Highly available in-memory database for HIPAA-regulated data*" → **ElastiCache for Redis** (HIPAA-eligible + Multi-AZ) "*Require a password for clients connecting to the cache, encrypted in transit*" → **Redis AUTH** (`--transit-encryption-enabled` + `--auth-token`; at-rest encryption doesn't authenticate) "*Low-latency cache reads in another region, plus cache DR*" → **Redis Global Datastore** "*Clients must find Memcached nodes automatically as the cluster scales*" → **Auto Discovery** (Configuration Endpoint)

Pocket card

Keyword	Answer
Repeated identical reads hammering DB	ElastiCache
Leaderboard / ranking	Redis sorted sets
Sessions / stateless web tier	Redis (or DynamoDB)
Durability, failover, backup, pub/sub	Redis
Simplest, multithreaded, loss OK	Memcached
Never-stale cache	Write-through
Stale-tolerant, cache only hot data	Lazy loading (+ TTL)
Diverse/analytical read offload	Read Replica
DynamoDB + no code change	DAX
Microseconds	Any in-memory cache (ElastiCache/DAX)
HIPAA in-memory store	Redis
Cache password / AUTH	Redis AUTH + in-transit encryption
Cross-region cache replicas / DR	Redis Global Datastore
Memcached node discovery	Auto Discovery (Configuration Endpoint)

You now own the "make reads fast" toolbox — next, Section 12 hands you the keyword zoo of specialty databases, where each engine exists to win exactly one kind of question.

↑ Back to index

Section 12: The Other Databases (keyword zoo)

The idea

Think of a hardware store wall: fifty screwdrivers, and each one exists for exactly one screw head. You don't need to *master* any of them — you need to glance at the screw and grab the right handle. AWS's specialty databases are that wall. **Each engine has ONE identifying keyword**, and the exam question always hands you the screw. This section is pure pattern-matching: keyword in, service out.

But first, one concept from zero, because it anchors half the zoo:

OLTP vs OLAP

- **OLTP** — OnLine **Transaction** Processing: **many small operations**. "*Fetch order #123.*" "*Update this user's address.*" Millions per day, each touching a few rows. This is your day-to-day app database: RDS, Aurora, DynamoDB. Row-based storage (grab the whole record at once).
- **OLAP** — OnLine **Analytical** Processing: **few huge scans**. "*Average revenue per region per month across five years.*" One query reads billions of rows but only 3 columns. Purpose-built warehouses use **columnar storage** — data stored column-by-column, so scanning `revenue` doesn't drag every other field off disk. Massively faster for analytics, plus columns compress beautifully.

THE trap (the cardinal sin): "*analysts run big reporting queries against the production database and the app slows down.*" Never run analytics on your prod OLTP database. The answer is always: move analytics to **Redshift** (or a Read Replica / Athena, per the scenario).

The zoo — one call each

Service	The ONE identifying call
Redshift	**Data warehouse.** Keywords: *OLAP, BI, analytics, petabyte-scale, columnar.* Load data in, point Tableau/QuickSight at it. **Redshift Spectrum** = query data **still sitting in S3** from Redshift **without loading it**. **Redshift Serverless** = same warehouse, no cluster to manage, pay per use.
Athena	**SQL directly on files in S3.** Fully **serverless**, pay **$5 per TB scanned**. Keyword: "*analyze logs in S3 with SQL, no infrastructure.*"
Neptune	**Graph database.** Data that is mostly *relationships*: **social networks, recommendation engines, fraud rings, knowledge graphs.** "Friends-of-friends" → Neptune.
QLDB	**Ledger. Immutable, cryptographically verifiable** history — nothing can be changed or deleted, ever, and you can *prove* it. **Centralized** (one owner).
Managed Blockchain	Like QLDB but **decentralized** — **multiple parties** who don't trust each other transact **without a central authority**. QLDB vs Blockchain = one owner vs many parties.
Timestream	**Time-series** database. **IoT sensor data, telemetry, DevOps metrics** — timestamped measurements arriving forever, queried by time window.
DocumentDB	The keyword is literally **"MongoDB."** MongoDB-compatible managed document DB. See MongoDB → circle DocumentDB → move on.
Keyspaces	The keyword is literally **"Cassandra."** Apache Cassandra-compatible, serverless.

Service	The ONE identifying call
OpenSearch	**Full-text / fuzzy search** + **log analytics with dashboards** (the old Elasticsearch/Kibana). Search "runing shoes" and still find running shoes.

The two comparisons worth 30 extra seconds

Athena vs Redshift — both do SQL analytics, so which?

```
Data stays in S3, ad-hoc, occasional, zero infra  →  ATHENA
Data loaded into a warehouse, heavy repeated BI,
complex joins, dashboards all day                 →  REDSHIFT
(Middle child: query S3 FROM your existing
 Redshift cluster without loading                 →  SPECTRUM)
```

- **Spectrum vs Athena in one line:** Athena queries **only S3**; Spectrum **joins S3 data with tables already inside Redshift** in one query. Both handle **structured and semi-structured** files in S3.
- **Redshift is not for OLTP or session state** — it gives **sub-second-to-seconds** analytic answers, not millisecond row lookups (that's DynamoDB/ElastiCache, and DAX is microseconds).
- **Redshift DR:** enable **cross-region snapshot copy** on the cluster (pick a destination region + retention) — snapshots are then copied automatically.

Athena cost optimization — this appears almost **verbatim**: "*reduce Athena query costs / improve performance.*" Athena bills per TB **scanned**, so scan less:

1. Convert files to **columnar formats — Parquet or ORC** (read only needed columns),
2. **Compress** the data,
3. **Partition** it (e.g., by date, so queries skip irrelevant folders).

All three shrink bytes scanned = smaller bill. Memorize the trio.

OpenSearch combo pattern: DynamoDB **cannot do fuzzy or full-text search** (it's a coat-check, not a librarian). The exam's fix: **DynamoDB + Streams → Lambda → OpenSearch** — writes land in DynamoDB, the change feed copies them into OpenSearch, and search queries go to OpenSearch.

DocumentDB vs DynamoDB — both store JSON-ish documents, so which?

	DocumentDB	DynamoDB
Compatibility	**MongoDB** APIs/drivers	AWS-native API
Queries	**Rich ad-hoc queries + secondary indexes** on any field, aggregations	**Key-based** access (partition/sort key, GSIs)
Infrastructure	**Instance/cluster-based** (you size nodes)	**Serverless**, any scale
Pick when	Existing MongoDB app, flexible document queries	Massive scale, predictable key lookups, zero ops

Zoo footnotes from the field:

- **Neptune** classic: build a **social network from user profiles and interactions** (who-knows-whom, likes, follows).

- **Timestream** is **serverless**, ingests **trillions of events per day**, and AWS pitches it as up to **1000x faster** (and cheaper) than relational DBs for time-series.
- **QLDB** = a **transparent, immutable, cryptographically verifiable transaction log**. **Legacy note:** AWS has announced **end of support for QLDB (July 2025)** and points customers to **Aurora PostgreSQL** (with audit tables) — expect it only as the "immutable ledger" keyword on older-style questions.

Database selection cheat sheet:

Need	Pick
Transactional SQL, existing engine	**RDS**
High-performance relational, cloud-native	**Aurora**
Serverless NoSQL key-value	**DynamoDB**
Analytics / warehouse	**Redshift**
Global multi-region NoSQL (multi-writer)	**DynamoDB Global Tables**
Global relational (one writer, fast DR)	**Aurora Global Database**

Question patterns

> "*BI team needs a petabyte-scale data warehouse for complex analytical queries*" → **Redshift** (OLAP, columnar, built for BI) "*Query data in S3 from the existing Redshift cluster without loading it*" → **Redshift Spectrum** (extends the warehouse into S3) "*Run occasional SQL queries on ALB/VPC logs stored in S3, minimal setup*" → **Athena** (serverless SQL-on-S3) "*Reduce Athena costs and speed up queries*" → **Parquet/ORC + compression + partitioning** (bill = TB scanned, so scan less) "*Social app needs friend-of-friend recommendations / detect fraud rings*" → **Neptune** (relationships = graph) "*Complete, immutable, cryptographically verifiable history of financial records, single organization*" → **QLDB** (centralized ledger) "*Multiple companies transact without a trusted central authority*" → **Managed Blockchain** (decentralized — the QLDB foil) "*Millions of IoT sensor readings per minute, analyzed over time windows*" → **Timestream** (time-series) "*Migrate a MongoDB workload to a managed AWS service*" → **DocumentDB** (keyword match, done) "*Migrate a Cassandra workload*" → **Keyspaces** (keyword match, done) "*Product catalog needs fuzzy, typo-tolerant full-text search; data is in DynamoDB*" → **OpenSearch** (via DynamoDB Streams → Lambda; DynamoDB can't fuzzy-search) "*Analysts' reports are slowing the production OLTP database*" → **move analytics off prod** — to Redshift (the cardinal sin, corrected) "*Join historical data in S3 with current tables in the Redshift warehouse*" → **Redshift Spectrum** (Athena can't see Redshift tables) "*Redshift cluster must be recoverable in another region*" → **cross-region snapshot copy** (destination region + retention) "*MongoDB-compatible JSON documents with rich queries on many fields*" → **DocumentDB**; "*serverless key-based access at any scale*" → **DynamoDB** "*Build a social network from user profiles and their interactions*" → **Neptune**

Pocket card

Keyword	Answer
Data warehouse / OLAP / BI / petabyte	Redshift
Query S3 from Redshift, no loading	Redshift Spectrum
SQL on S3, serverless, ad-hoc / logs	Athena
Cut Athena cost	Parquet/ORC + compress + partition
Graph / social / fraud ring / recommendations	Neptune
Immutable verifiable ledger, centralized	QLDB
Decentralized, multi-party, no central authority	Managed Blockchain
Time-series / IoT / telemetry / metrics	Timestream
"MongoDB"	DocumentDB
"Cassandra"	Keyspaces
Full-text / fuzzy search, log dashboards	OpenSearch
Analytics choking prod OLTP DB	Move it — Redshift
S3 data + Redshift tables in one query	Spectrum (Athena = S3 only)
Redshift DR	Cross-region snapshot copy
MongoDB queries vs key lookups	DocumentDB vs DynamoDB
Session state / OLTP	Not Redshift — DynamoDB/ElastiCache
QLDB	Legacy (end of support 2025)

That's every database in the toolbox sorted by screw head — now Section 13 turns to the thing that glues all these services together with code: Lambda.

↑ Back to index

Section 13: Lambda

The idea

Imagine a contractor who **materializes on your doorstep the instant the doorbell rings**, does exactly one job, vanishes — and bills you only for the minutes he existed. No salary, no idle time, no office. Ring the bell a thousand times at once? A thousand contractors materialize in parallel. **That's Lambda: code without a computer.**

Spell it out: Lambda is **serverless compute**. You upload a **function** (your code), tell AWS what **event** triggers it (an S3 upload, an API call, a schedule), and AWS runs it — provisioning, scaling, and patching invisible to you. You pay **per millisecond of execution**, and it **scales automatically per event**. No traffic = no cost. This is *event-driven* architecture: things happen, code reacts.

Hard limits — where half the traps live

Limit	Value
Max execution time	**15 MINUTES**
Memory	**128 MB – 10 GB**
CPU	**scales WITH memory** (no separate CPU knob)
Ephemeral `/tmp` storage	up to **10 GB**
Deployment package	**50 MB zipped / 250 MB unzipped**
Container image	up to **10 GB**
Default concurrency per region	**1,000** (raisable)

THE trap: "*a 2-hour video processing job*" → **NOT Lambda** — 15-minute ceiling. Answer: **AWS Batch** or **ECS/Fargate**. **Step Functions** is only right if the work can be **divided into chunks under 15 minutes each**.

THE trap (the counterintuitive favorite): "*CPU-bound Lambda function is slow — how to speed it up?*" → **increase the memory**. There's no CPU setting; **more memory = more CPU**. It reads like a wrong answer. It's the right one.

Three ways a Lambda gets invoked

Think of how mail reaches you: someone **hands it to you and waits** for a reply, someone **drops it through the slot** and leaves, or **you walk to the PO box** and collect a batch.

1. **Synchronous** (hand-delivered): the caller **waits** for the result. Classic: **API Gateway** → Lambda. Errors go straight back to the caller.
2. **Asynchronous** (through the slot): the source **PUSHES** the event and walks away — **S3, SNS, EventBridge**. Lambda queues it, and on failure **retries twice**, then the event is **gone** — unless you configure a **DLQ (dead-letter queue) or failure destination**. **THE trap:** "*asynchronously invoked events are lost on failure*" → **add a DLQ / on-failure destination**.

3. **Event Source Mapping** (PO box): for **SQS, Kinesis, and DynamoDB Streams**, the source doesn't knock — **Lambda PULLS**, polling the stream/queue and grabbing records **in batches**. Failed batches go **back to the queue** and eventually to the **queue's own DLQ**. Push vs pull: knockers (S3/SNS/EventBridge) vs buckets you must visit (SQS/Kinesis/Streams).

Cold starts and the two "concurrencies"

First invocation after idle, AWS must materialize the contractor — load runtime + code. That pause is a **cold start**, and it shows up as "*sporadic latency spikes after periods of inactivity.*"

Two similarly-named settings, opposite jobs — hook: **P = Pre-warmed, R = Restricted**:

- **Provisioned Concurrency** = **Pre-warmed** instances always ready → **kills cold starts**. Answer to latency questions.
- **Reserved Concurrency** = a **Restricted cap** on how many copies can run at once → **protects downstream systems** (e.g., a fragile legacy database that dies past 50 connections).

Lambda in a VPC

By default Lambda runs outside your VPC and can reach the internet but not your private resources. Put it **in the VPC** (it gets an **ENI** — Elastic Network Interface — in your subnet) and it can reach private RDS... but **it LOSES internet access**. Need both? **NAT Gateway** (or VPC endpoints for AWS services).

And whenever **Lambda talks to RDS**: hundreds of short-lived functions each opening their own DB connection will exhaust the database. The answer is **RDS Proxy** — a connection pooler that shares a warm pool of connections. Lambda + RDS in the same sentence → **RDS Proxy**, always.

Odds and ends the exam sprinkles in

- **Execution role**: the **IAM role attached to the function** IS its permissions. "Lambda can't write to DynamoDB" → fix the execution role.
- **Execution role vs resource-based policy** — two directions, two documents:
  - **Resource-based (function) policy** = **who can invoke ME** (S3, SNS, API Gateway, another account). "*S3 can't trigger the function*" / "*cross-account invoke*" → add permission to the **function policy**.
  - **Execution role** = **what I can access**. When the function calls S3, DynamoDB, SQS, or **KMS**, AWS evaluates the **execution role** (the principal is the assumed role session, not the function ARN). So a **KMS key policy** or bucket policy must grant the **execution role**, not "the Lambda function."
- **Layers**: package **shared libraries/dependencies** once, reuse across functions.
- **Container images**: deploy Lambda as a container up to **10 GB** — the answer when dependencies blow past the 250 MB zip limit.
- **Environment variables**: config outside code; encrypt secrets with **KMS**. They're **encrypted at rest by default**, but anyone with console/ `GetFunctionConfiguration` access **sees them in plaintext** — real secrets (DB passwords, API keys) belong in **Secrets Manager / Parameter Store**, fetched at runtime.
- **EventBridge scheduled rule** → **Lambda** = **serverless cron**. Any "*run a task every night at 2 AM without servers*" → this.

Code at the edge: Lambda@Edge vs CloudFront Functions

Both run code at CloudFront edge locations; the split is muscle vs speed:

	Lambda@Edge	**CloudFront Functions**
Runtime	Node.js/Python, up to seconds	JavaScript only, **sub-millisecond**
Network calls	**YES**	**NO**
Hooks	**viewer + origin** request/response	**viewer** request/response only
Cost	baseline	**~1/6 the price**
Use	**JWT/auth validation, origin selection**, real logic	**header manipulation, URL rewrites/redirects** — trivial tweaks at massive scale

Rule: *needs external calls or real logic* → **Lambda@Edge**; *trivial header/URL tweak on every request* → **CloudFront Functions**.

Question patterns

> *"90-minute video transcoding job — can it run on Lambda?"* → **No — AWS Batch / Fargate** (15-minute hard limit; Step Functions only if divisible into <15-min steps) *"Generate a thumbnail whenever an image lands in S3"* → **S3 event notification** → **Lambda** (the canonical async trigger) *"API is fast normally but has latency spikes after idle periods"* → **Provisioned Concurrency** (Pre-warmed = no cold starts) *"Lambda scaling overwhelms a legacy database that handles only 50 connections"* → **Reserved Concurrency** cap (Restricted) — and mention **RDS Proxy** *"Lambda must query an RDS database in a private subnet"* → **attach Lambda to the VPC + RDS Proxy** (add NAT if it still needs internet) *"CPU-intensive function is too slow"* → **increase memory** (CPU scales with memory — the counterintuitive one) *"Run a cleanup script every night at 2 AM, no servers"* → **EventBridge schedule** → **Lambda** (serverless cron) *"Validate JWT tokens / call an external auth service at the edge"* → **Lambda@Edge** (network calls allowed; CF Functions can't) *"Rewrite URLs or add security headers on millions of requests, cheapest"* → **CloudFront Functions** (sub-ms, ~1/6 price, no network) *"Asynchronously invoked function fails and events disappear"* → **DLQ / on-failure destination** (async retries twice, then drops) *"Lambda should process SQS messages"* → **Event Source Mapping** (Lambda pulls in batches; failures return to the queue) *"S3 in another account must invoke the function"* → **Resource-based (function) policy** (who can invoke me) *"Lambda gets AccessDenied decrypting with a KMS key"* → **Grant the execution role in the key policy / role policy** (the role is the principal, not the function) *"Store a DB password for Lambda so console users can't read it"* → **Secrets Manager / Parameter Store SecureString**, not plain env vars

Pocket card

Keyword	**Answer**
Job > 15 minutes	NOT Lambda → Batch / Fargate
Divisible into <15-min steps	Step Functions

Keyword	Answer
CPU-bound slow function	Increase memory
Cold start latency	Provisioned Concurrency (Pre-warmed)
Protect downstream / cap invocations	Reserved Concurrency (Restricted)
Lambda + RDS	RDS Proxy (+ VPC config)
VPC Lambda lost internet	NAT Gateway / VPC endpoints
Async failures lost	DLQ / failure destination
SQS / Kinesis / DDB Streams	Event Source Mapping (pull, batches)
S3 / SNS / EventBridge	Async push (2 retries)
Serverless cron	EventBridge schedule → Lambda
Shared dependencies	Layers
Deps > 250 MB	Container image (10 GB)
Function permissions	Execution role
Who can invoke the function	Resource-based (function) policy
Lambda → KMS/S3/DynamoDB access checked against	Execution role (not function ARN)
Secrets visible in env vars	Secrets Manager / Parameter Store
Edge + network calls / JWT	Lambda@Edge
Edge header/URL tweak, cheapest	CloudFront Functions
Max runtime / memory / concurrency	15 min / 10 GB / 1,000 default

Lambda is the glue of every serverless diagram you've built so far — CloudFront to API Gateway to Lambda to DynamoDB — and with these four sections you now own that entire stack end to end.

↑ Back to index

Section 14: ECS, EKS & Fargate

The idea

Before containers, shipping software was chaos. Your app worked on your laptop, then exploded on the server because the server had a different Python version, a missing library, a different OS. The dreaded phrase: **"but it works on my machine!"**

Containers fix this the same way shipping containers fixed global trade. Before standardized steel boxes, dock workers hand-loaded barrels, crates, and sacks — slow, fragile, different for every ship. Then someone said: *put everything in an identical steel box, and every crane, ship, and truck in the world handles it the same way.* A software container is that steel box: your **app plus its entire environment** (runtime, libraries, config, OS dependencies) packaged into one **image**. If it runs in the container on your laptop, it runs identically anywhere.

On the compute spectrum, containers are the **middle ground between Lambda and EC2**: more control and longer-running than Lambda (no 15-minute limit), less babysitting than raw EC2.

But now a new problem: real applications run *hundreds* of containers. Who restarts one when it crashes? Who spreads them across machines? Who wires them to the load balancer? Doing this by hand is like being a harbor master directing every crane manually. You need an **orchestrator** — a robotic harbor master. AWS gives you two: **ECS** (Elastic Container Service — AWS's own, simple) and **EKS** (Elastic Kubernetes Service — managed Kubernetes, the open-source industry standard).

ECS vocabulary — learn these four words cold

ECS term	What it is	Analogy
Task Definition	JSON blueprint: which image, CPU/RAM, ports, env vars, IAM roles	The recipe
Task	One running copy of that definition	A dish cooked from the recipe
Service	Keeps N tasks running, replaces crashed ones, wires them to an ALB	The babysitter
Cluster	Logical grouping of the infrastructure the tasks run on	The kitchen

Launch types: EC2 vs Fargate

```
            ECS or EKS (the orchestrator brain)
                   /              \
EC2 launch type          Fargate launch type
---------------          --------------------
YOU manage the           NO instances visible.
instances (patch,        Declare CPU + RAM per
scale, choose type)      task. AWS runs it.
+ GPUs possible          + zero server mgmt
+ Spot instances         + Fargate Spot for
  for cost tricks          interruptible work
                         - NO GPUs (!)
```

- **EC2 launch type**: containers run on EC2 instances *you* manage. You pick instance types, you patch, you can use **Spot instances** for cost savings, and — crucially — you can attach **GPUs**.
- **Fargate**: **serverless containers**. You declare CPU and memory per task; AWS provisions invisible compute. The moment an exam question says **"no server management"** or "without managing infrastructure" for containers → **Fargate**.

THE trap: Fargate does NOT support GPUs. "Containerized ML inference needing GPUs" → **EC2 launch type**, never Fargate. This nuance is tested.

Misconception to kill: Fargate is **not a third orchestrator**. It's a compute layer that plugs into **BOTH ECS and EKS**. ECS/EKS decide *what* runs; Fargate is one option for *where* it runs.

When EKS? Only two signals

Default to **ECS** unless the question says:

1. **"Already using Kubernetes"** (on-prem k8s migration, existing k8s tooling/manifests), or
2. **"Portability / multi-cloud / avoid vendor lock-in"** (Kubernetes runs anywhere; ECS is AWS-only).

No k8s keyword? **ECS.** It's simpler and AWS-native. ("*Docker containers, no Kubernetes needed*" → ECS; "*Kubernetes + serverless / no nodes*" → **EKS on Fargate**.)

EKS fine print the exam pulls from

- **Control plane**: in EKS, **AWS runs the Kubernetes control plane across multiple AZs** (HA, patched for you); you run (or Fargate runs) the worker nodes.
- **On-premises flavors**: **EKS Anywhere** = run EKS-distribution Kubernetes on **your own hardware** — **you** manage the clusters (control plane included). **ECS Anywhere** = register on-prem servers as ECS capacity, managed from the ECS control plane. **EKS Connector** = only **registers/views** an external Kubernetes cluster (any provider) in the EKS console — it does not manage it.
- **Node autoscaling**: **Cluster Autoscaler** scales nodes through **Auto Scaling groups** (fixed instance types per node group). **Karpenter** provisions nodes **directly**, choosing the right instance type/size per pending pods dynamically — faster, better bin-packing, **less operational overhead** → the modern answer for "*efficient EKS node scaling, minimal ops.*"

- **Access control**: **authentication = IAM** (AWS IAM Authenticator; IAM principals mapped via the `aws-auth` **ConfigMap**, auto-created with managed node groups / `eksctl`, now also **EKS access entries**); **authorization = Kubernetes RBAC** (Roles/RoleBindings). "*Give an IAM role read-only access to one namespace*" → map it in aws-auth/access entries **+** a Kubernetes RBAC Role.

ECR — the image warehouse

ECR (Elastic Container Registry) stores your container images (like Docker Hub, but private and IAM-integrated). Bonus exam fact: ECR does **image vulnerability scanning** — "scan container images for CVEs" → **ECR scanning**.

THE role trap pair (guaranteed points)

Two IAM roles per task, constantly confused:

Role	Used by	For	Symptom when wrong
Task Execution Role	ECS agent (plumbing)	**LAUNCHING**: pull image from ECR, write logs to CloudWatch	"Task fails to start / **can't pull image**"
Task Role	Your app code inside	What the **app does once running**: S3, DynamoDB, SQS calls	"**App gets AccessDenied** calling DynamoDB"

Hook: **Execution = getting the container up. Task Role = what the app does once it's up.**

Both are **IAM roles** (trusted by `ecs-tasks.amazonaws.com`) — not resource-based policies. Secrets go in the task definition's `secrets` parameter (Secrets Manager / Parameter Store), read by the **execution role** — see Section 21.

Networking and scaling facts

- **awsvpc network mode**: every task gets its **own ENI** (Elastic Network Interface — its own private IP) and therefore its **own security group**. "Per-container security group" → awsvpc. (Required mode on Fargate.)
- **ALB dynamic port mapping**: on the EC2 launch type, an Application Load Balancer can route to **multiple tasks of the same service on one instance**, each on a random host port. No manual port juggling.
- **Service Auto Scaling**: scale task count on **CPU, memory, or SQS queue depth**. Target-tracking predefined metrics: `ECSServiceAverageCPUUtilization`, `ECSServiceAverageMemoryUtilization`, `ALBRequestCountPerTarget`.
- **Fargate Spot**: discounted Fargate for **interruption-tolerant** workloads.
- **Pricing**: **Fargate** bills for the **vCPU + memory requested** per task (per second); the **EC2 launch type** has **no extra ECS charge** — you pay for the EC2 instances (and EBS) whether tasks fill them or not.
- **Shared persistent storage** across tasks (EC2 or Fargate, multi-AZ) → mount **EFS** in the task definition.
- **App Runner**: even simpler than ECS/Fargate for a containerized web app/API — see Section 37.

The compute ladder (which service for which job)

Signal in question	Answer
Event-driven, runs **< 15 min**	Lambda
Containers, **no infrastructure management**	Fargate
Containers needing **GPU / deep instance control / Spot**	ECS on EC2
Kubernetes / multi-cloud portability	EKS
Classic web app, "just deploy my code"	Elastic Beanstalk
Full OS control, custom everything	EC2

Question patterns

> "*Run containers without managing any servers or clusters of instances*" → **Fargate** (the phrase "no server management" is the trigger) "*Company runs Kubernetes on-premises and wants to migrate to AWS with minimal changes*" → **EKS** ("already Kubernetes" is one of only two EKS signals) "*ECS task fails to start; error pulling image from ECR*" → **Task Execution Role** (launch plumbing = execution role) "*Application inside the container gets AccessDenied calling DynamoDB*" → **Task Role** (app-level permissions once running) "*Containerized GPU-based ML inference, minimize management*" → **ECS on EC2 launch type** (Fargate has NO GPUs) "*Automatically scan container images for vulnerabilities*" → **ECR image scanning** (built into the registry) "*Assign a dedicated security group to each container/task*" → **awsvpc network mode** (per-task ENI = per-task SG) "*Avoid vendor lock-in / portable across clouds*" → **EKS** (Kubernetes is the portability play) "*Cost-optimize fault-tolerant containerized batch jobs on Fargate*" → **Fargate Spot** (interruptible = Spot) "*Scale container count based on messages waiting in a queue*" → **ECS Service Auto Scaling on SQS queue depth** "*Run Kubernetes on our own data-center hardware with EKS tooling*" → **EKS Anywhere** (you manage the cluster) "*View on-prem/other-cloud Kubernetes clusters in the EKS console*" → **EKS Connector** (view only) "*Scale EKS worker nodes efficiently, right-sized instance types, least ops*" → **Karpenter** (not Cluster Autoscaler + ASGs) "*Grant an IAM role access to specific Kubernetes namespaces*" → **aws-auth ConfigMap / access entries (authN) + Kubernetes RBAC (authZ)** "*Containers in several AZs need a shared file system*" → **EFS volume** in the task definition "*Kubernetes workloads with no nodes to manage*" → **EKS on Fargate**

Pocket card

Keyword	Answer
"No server management" + containers	Fargate
"Already using Kubernetes" / "multi-cloud"	EKS
GPUs for containers	EC2 launch type (never Fargate)
Can't pull image / can't write logs	Task Execution Role
App AccessDenied to AWS service	Task Role

Keyword	Answer
Recipe / blueprint	Task Definition
Keeps N copies running + ALB	Service
Image storage + CVE scanning	ECR
Per-task ENI + security group	awsvpc mode
Cheap interruptible Fargate	Fargate Spot
Multiple same-service tasks, one instance	ALB dynamic port mapping
EKS control plane	AWS-managed, multi-AZ
Kubernetes on own hardware	EKS Anywhere (you manage)
On-prem servers as ECS capacity	ECS Anywhere
See external k8s clusters in console	EKS Connector
Dynamic right-sized EKS nodes	Karpenter (vs Cluster Autoscaler → ASGs)
EKS authN / authZ	IAM (aws-auth) / Kubernetes RBAC
Fargate pricing	vCPU + memory requested
ECS on EC2 pricing	No extra charge — pay for instances
Shared storage across tasks	EFS
ECS target-tracking metrics	ECSServiceAverageCPU/MemoryUtilization, ALBRequestCountPerTarget

Once your containers are running, the next question is: what if you don't even want to think about containers — you just want to hand AWS your code? That's Elastic Beanstalk, up next.

↑ Back to index

Section 15: Elastic Beanstalk

The idea

Elastic Beanstalk is AWS saying: **"here's my code, you figure out the rest."**

Think of it as moving into a **fully furnished apartment**. You could buy land, pour a foundation, and wire the electricity yourself (raw EC2 + ASG + ALB by hand) — or you could just show up with your suitcase (your code) and everything is already set up. But here's the key difference from a hotel: **you still get the keys to every room**. You can rearrange furniture, swap appliances — nothing is locked away.

Concretely: you upload your code — **Java, Python, Node.js, .NET, Go, Ruby, PHP, or Docker** — and Beanstalk automatically provisions **EC2 instances, an Auto Scaling Group, a Load Balancer, and CloudWatch monitoring**. You didn't configure any of them, but you *can* see and tune all of them.

Two exam-critical identity facts:

- **EB is an orchestrator, NOT a black box.** You retain **full control of the underlying resources**. The identity phrase the exam loves: "*deploy a web application quickly without managing infrastructure BUT retain control over the resources.*" That sentence = Beanstalk.
- **EB itself is free** — you pay only for the resources it creates (EC2, ALB, etc.).
- **Where your files go**: Beanstalk stores your **application versions (source bundles)** and, optionally, **server/instance logs** in an **S3 bucket** it creates per Region (`elasticbeanstalk-<region>-<account>`).

Config tweaks live in `.ebextensions` — YAML/JSON files in your code bundle that customize the environment (packages, env vars, resources).

There's also a **Worker environment**: instead of serving web traffic, it pulls jobs from an **SQS queue** — the background-processing tier of your app.

THE tested table: deployment policies

This table is the single most-tested Beanstalk fact. Learn to match constraint → row.

Policy	How it works	Downtime?	Capacity during deploy	Extra cost?	Rollback	Pick when...
All at once	Update every instance simultaneously	**YES**	Drops to zero briefly	No	Redeploy old version (slow)	Fastest; **dev/test, downtime OK**
Rolling	Update in batches, batch by batch	No	**Reduced** (a batch is always down)	**No**	Slow (roll batches back)	**No downtime + no extra cost**

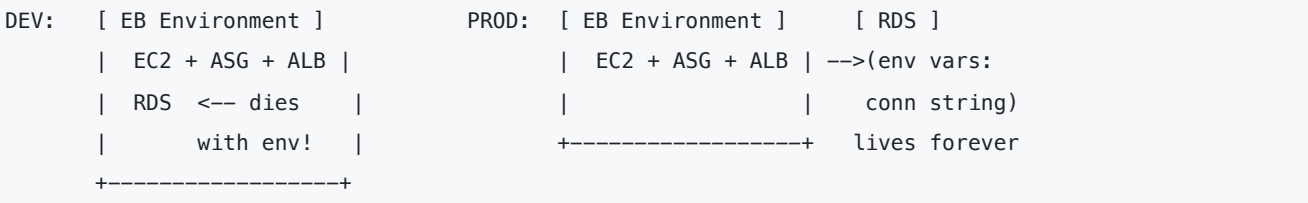
Policy	How it works	Downtime?	Capacity during deploy	Extra cost?	Rollback	Pick when...
Rolling with additional batch	Spin up one extra batch first, then roll	No	**FULL capacity maintained**	Small (one extra batch)	Slow	Can't afford reduced capacity
Immutable	Build a **whole new fleet** alongside the old, swap when healthy	No	Full	Double, briefly	**Safest: just delete new fleet**	Production, **safest rollback**
Blue/Green	Clone the entire **environment**, test it, then **swap DNS CNAMEs**	No	Full	Double while both exist	**Instant: swap CNAME back**	Test new version with real URL, instant switch

Constraint-matching cheats:

- "Fastest, downtime acceptable" → **All at once**
- "No downtime, no additional cost" → **Rolling**
- "Must maintain full capacity" → **Rolling with additional batch**
- "Safest / easiest rollback if instances fail" → **Immutable**
- "Test new version, then instant cutover / instant rollback via DNS" → **Blue/Green (CNAME swap)**

THE RDS trap

If you create an RDS database **inside** a Beanstalk environment, its lifecycle is **tied to the environment** — terminate or rebuild the environment, and **the database dies with it**.



Production rule: RDS lives OUTSIDE the environment, and the app gets the **connection string via environment variables**.

Migration path if you already made the mistake: take an **RDS snapshot** → **restore as a standalone RDS instance** → point the environment at it via env vars → enable deletion protection.

When Beanstalk is the distractor

EB is often a wrong answer planted next to the right one:

Question really wants	Correct answer
Fine-grained, repeatable **infrastructure as code**	**CloudFormation**
Microservices at scale / container orchestration	**ECS / EKS**

Question really wants	Correct answer
Event-driven, sub-15-minute functions	**Lambda**

EB is for: *one classic web app, developers who want speed, control retained.*

Question patterns

> *"Developers want to deploy code quickly without managing infrastructure but must retain full control of the underlying resources"* → **Elastic Beanstalk** (that's its identity phrase verbatim) *"Deploy new version as fast as possible; brief downtime is acceptable (dev environment)"* → **All at once** (speed + downtime-OK = the only reason to pick it) *"Deploy with no downtime and no additional cost"* → **Rolling** (batches reuse existing instances, capacity dips but nothing new is billed) *"Deploy with no downtime while maintaining full capacity"* → **Rolling with additional batch** (the extra batch covers the gap) *"Deployment must be safest possible with quick rollback if health checks fail"* → **Immutable** (bad deploy? delete the new fleet, old one untouched) *"Test the new version against a separate URL, then switch all traffic instantly with instant rollback"* → **Blue/Green with CNAME swap** (DNS swap both ways) *"Beanstalk app's database was lost when the environment was terminated — prevent this in production"* → **Create RDS outside the environment; connect via environment variables** (decouple lifecycles) *"Migrate a database out of an existing Beanstalk environment"* → **Snapshot the RDS instance → restore as standalone RDS** (then reconnect via env vars) *"Long-running background jobs from a queue alongside a Beanstalk web app"* → **Worker environment** (the SQS-consuming tier) *"Customize packages and configuration of Beanstalk instances at deploy time"* → **.ebextensions** (config files in the source bundle)

Pocket card

Keyword	Answer
"Quickly deploy, no infra mgmt, KEEP control"	Elastic Beanstalk
Fastest deploy, downtime OK	All at once
No downtime, no extra cost	Rolling
No downtime, full capacity	Rolling + additional batch
Safest rollback (delete new fleet)	Immutable
DNS CNAME swap, instant rollback	Blue/Green
DB died with environment	Create RDS OUTSIDE env (env vars)
Move DB out of env	Snapshot → standalone RDS
SQS-driven background tier	Worker environment
Instance/env customization files	.ebextensions
Beanstalk pricing	Free — pay for resources only
Where EB keeps app versions + logs	S3 bucket
Fine-grained IaC instead	CloudFormation

Beanstalk fronts your app with a load balancer automatically — but when your "app" is an API consumed by other programs, you want a smarter front door: API Gateway, next section.

↑ Back to index

Section 16: API Gateway

The idea

API Gateway is the **front desk of a busy office building**. Nobody wanders straight to your engineers' desks (your Lambda functions, your backends). Everyone checks in at the front desk first, where the receptionist **routes** them to the right floor, **checks their ID** (authentication), **turns away crowds** when the lobby is full (throttling), and **writes every visit in the logbook** (logging) — all *before* your code runs a single line.

That's API Gateway: a fully managed service that sits in front of your backend and handles routing, auth, rate limiting, caching, and monitoring.

Burn in the canonical serverless chain — the exam's favorite architecture:

```
Client --> CloudFront --> API Gateway --> Lambda --> DynamoDB
```

The reflex: **"no servers to manage, pay per request"** → this chain. When a question describes a REST API with zero server management, this is the answer skeleton.

Three API types

Type	Personality	Features	Trigger phrase
REST API	Full-featured flagship	**API keys, usage plans, caching, WAF, request validation**	Needs any of those features
HTTP API	Minimalist, **~70% cheaper**	Simple proxy to Lambda/HTTP, built-in **JWT** auth, low latency	**"Lowest cost"** simple Lambda proxy
WebSocket API	**Persistent two-way** connection	Server can push to clients	**"Real-time chat"**, "live push", live dashboards

THE trap: usage plans, API keys, and caching are **REST API only**. If the question needs them, HTTP API is a distractor no matter how cheap it is.

Endpoint types (where does it live?)

Endpoint	For	Note
Edge-Optimized	**Globally distributed clients**	Routed through CloudFront's edge network automatically (default)
Regional	Clients in the **same region**	Or when you want to bolt on **your own CloudFront** distribution
Private	**VPC-only** access	Reached via an **Interface VPC Endpoint** — never touches the internet

The auth trio (guaranteed question)

Three ways to answer "who's calling?" — the exam WILL make you pick one:

Authorizer	Use when the caller is...	Mechanism
IAM / SigV4	**AWS-native**: internal services, EC2 roles, IAM users	Requests signed with AWS credentials (Signature Version 4)
Cognito User Pool authorizer	**App users who sign in** (mobile/web app accounts)	Cognito issues a **JWT** (JSON Web Token); gateway validates it, zero custom code
Lambda Authorizer	**Custom logic**: third-party identity provider, legacy/**bespoke tokens**, weird rules	Your Lambda inspects the token and returns an IAM policy

Hook: **IAM = machines, Cognito = your users, Lambda Authorizer = anything weird.**

Traffic control & performance

- **Default throttle: 10,000 requests/second** per account per region. Exceed it and clients get **HTTP 429 Too Many Requests** — "clients receiving 429" → you're being throttled.
- Throttling uses a **token bucket** algorithm: a steady **rate** plus a **burst** allowance. You can set your own **per-method / per-stage rate + burst** (e.g. **1,000 req/s with a 2,000 burst**) to protect a fragile backend from spikes.
- **THE trap: ELB can't throttle.** A load balancer just spreads requests; it has no rate limits. "Protect the backend from request floods / cap request rate" → **API Gateway throttling** (or usage plans), not an ALB.
- **Pricing**: pay **per API call received + data transferred out** (plus hourly cache charges if caching is on). No minimums, no servers.
- **Usage Plans + API Keys** (REST only): per-customer request quotas and rate tiers. Trigger: **"SaaS company sells API access in Basic/Pro tiers"** → usage plans with API keys.
- **Caching**: gateway caches responses (**default TTL 300 seconds**), reducing backend load and latency. You provision the cache **per stage with a size in GB** (billed hourly). "Reduce calls hitting the backend for repeated requests" → enable API Gateway caching.
- **Tracing**: turn on **X-Ray** to follow a request from API Gateway through the backend services behind it.

Two mechanical facts (free points)

1. **THE 29-second timeout.** API Gateway waits a **maximum of 29 seconds** for the backend. Lambda can run 15 minutes — **but not behind API Gateway**. This limit is **hard; "increase the timeout" is impossible past 29s** — any answer suggesting it is wrong. Long-running jobs go **asynchronous**: return **202 Accepted** immediately, hand the work to **SQS or Step Functions**, let the client poll or get notified.

```
Client --> API GW --202--> (immediately)
              |
              +--> SQS --> Lambda/worker (takes 2 min, nobody's waiting)
```

2. **CORS (Cross-Origin Resource Sharing).** When **browser JavaScript on one domain** calls your API on another domain and gets blocked with cross-origin errors → **enable CORS** on API Gateway. Browser + cross-domain + JS error = CORS, every time.

Question patterns

> "*Build a REST API with no servers to manage, pay only per request*" → **API Gateway + Lambda + DynamoDB** (the canonical serverless chain) "*SaaS company wants to offer customers different API request limits per pricing tier*" → **Usage Plans + API Keys** (REST API only — per-customer tiers) "*Mobile app users must sign in before calling the API*" → **Cognito User Pool authorizer** (app users = Cognito JWTs) "*Company has an existing custom/third-party token system for API auth*" → **Lambda Authorizer** (custom logic = your Lambda decides) "*Cheapest way to expose a simple Lambda function over HTTP with JWT auth*" → **HTTP API** (~70% cheaper, minimal features) "*Push live updates to a real-time dashboard / chat application*" → **WebSocket API** (persistent two-way connection) "*API must be accessible only from within the VPC, never the internet*" → **Private endpoint + Interface VPC Endpoint** "*Backend process takes 2 minutes; API Gateway requests keep timing out — fix it*" → **Go async: return 202, queue to SQS/Step Functions** (29s is a hard limit; you cannot raise it) "*Browser JavaScript from another domain gets blocked calling the API*" → **Enable CORS** on API Gateway "*Clients suddenly receive HTTP 429 errors*" → **Throttling** — raise the limit or add caching/usage plans "*Limit requests to a legacy backend to 1,000/s while absorbing short bursts*" → **API Gateway throttling (rate + burst, token bucket)** (an ALB cannot throttle)

Pocket card

Keyword	Answer
Serverless REST API	API GW + Lambda + DynamoDB
API keys / usage plans / caching / WAF	REST API (only)
Lowest cost, simple proxy, JWT	HTTP API
Real-time / chat / live push	WebSocket API
Global clients	Edge-Optimized endpoint
VPC-only API	Private endpoint (Interface Endpoint)
AWS-service callers	IAM / SigV4 auth
App users sign in	Cognito User Pool authorizer
Custom/third-party tokens	Lambda Authorizer
429 errors	Throttling (default 10,000 req/s)
Backend > 29 seconds	Async: 202 + SQS/Step Functions
Cache TTL default	300 seconds
Throttle algorithm	Token bucket: rate + burst (per method/stage)
Rate-limit requests (not ELB)	API Gateway throttling
Pricing	Per API call + data transfer (+ cache GB-hours)

Keyword	Answer
Browser cross-domain error	Enable CORS

You just saw the fix for slow backends is "drop the work into a queue and walk away" — that queue, and the whole messaging toolbox around it, is the next section.

↑ Back to index

Section 17: SQS, SNS & Kinesis — the messaging block

The idea

This section is one of the **heaviest question sources** on the exam, so let's build it from zero.

Imagine your web tier calls your order processor **directly**:

```
[Web server] ──(sync call)──> [Order processor]
```

Three ways this hurts you:

1. **The spinner**: processor is slow → the customer stares at a loading wheel.
2. **Lost orders**: processor crashes mid-call → the order evaporates.
3. **Drowning**: Black Friday spike → processor gets 100x traffic and falls over.

Now put a **buffer** in between:

```
[Web server] --> [ QUEUE:  msg | msg | msg ] --> [Order processor]
   (instant "OK!")    (holds everything)        (works at its own pace)
```

One buffer fixes all three: the web tier answers instantly, messages survive crashes because they sit safely in the queue, and spikes just make the queue longer instead of killing the worker. This is **decoupling**, and the vivid analogy for the whole section: SQS is a **mailbox** (letters wait until you collect them), SNS is a **megaphone** (everyone within earshot hears it *now*, then it's gone), and Kinesis is a **river** (many people can stand on the bank reading the same water, and you can wade back upstream to re-read it).

SQS — the mailbox (queue, PULL, one message → ONE worker)

SQS (Simple Queue Service): producers put messages in; consumers **pull** them out. Each message is processed by **exactly one worker** — once someone takes the letter from the mailbox, it's theirs. A distributed queue system has three parts: the **components** (producers/consumers), the **queue** (redundantly stored across AWS servers), and the **messages** in it.

Visibility timeout — the mechanic the exam adores:

- When a worker reads a message, it is **hidden, not deleted** (default **30 seconds**, max **12 hours**).
- Worker finishes → explicitly deletes it (`DeleteMessage` — **SQS never auto-deletes on receive**; forget the call and the message reappears after the timeout. The only automatic deletion is when the **retention period** expires). Worker **crashes** → timeout expires → message **reappears** for another worker. That's your built-in retry.
- **THE trap: "messages are being processed twice."** Cause: processing takes longer than the visibility timeout, so the message reappears while worker #1 is still on it. Fix: **increase the visibility timeout.**

More SQS facts:

- **DLQ (Dead-Letter Queue)**: a "poison message" that keeps failing gets moved to a side queue after **MaxReceiveCount** attempts — inspect failures without clogging the main queue.
- **Long polling** (up to **20 seconds**): consumer waits for messages instead of hammering empty-poll requests. "Reduce cost / empty responses from polling" → **long polling**.
    - **Short polling** is the default (`ReceiveMessageWaitTimeSeconds` = **0**): returns **immediately, even if the queue is empty**, and samples only a **subset of SQS servers** → false empty responses. **Long polling** (`ReceiveMessageWaitTimeSeconds` **> 0**, set on the queue or per receive call) queries **all servers** and waits until a message arrives or the wait time ends.
- **Delay queues**: postpone delivery of *new* messages for **0–15 minutes** (`DelaySeconds`, default 0) — "message must not be processed until N minutes after it's sent". (Visibility timeout hides a message *after* it's received; delay hides it *before* first receipt.)
- **Access/endpoints**: SQS is reached via its **public regional endpoint** (over the internet via IGW/NAT) or privately via an **Interface VPC Endpoint (PrivateLink)** — same for SNS and Kinesis. Only **S3 and DynamoDB** use **Gateway Endpoints**.
- **Max message size 256 KB.** Bigger payloads → **store in S3, send a pointer** in the message.
- **Retention: 4 days default, 14 days max.** Not a database — collect your mail.
- **Standard vs FIFO**:

	Standard	**FIFO**
Throughput	**Unlimited**	**300 msg/s** (3,000 with batching)
Delivery	**At-least-once** (dupes possible)	**Exactly-once**
Order	Best-effort	**Strict order**
Trigger	Default choice	"Order matters" / "no duplicates"

- FIFO queue names **must end in** `.fifo`. You **cannot convert** an existing Standard queue to FIFO — **create a new FIFO queue** and move producers/consumers over.
- Classic pattern: **Auto Scaling Group of workers scaling on queue depth** (CloudWatch metric `ApproximateNumberOfMessagesVisible`) — the queue gets long, more workers spawn.
- **Time-sensitive messages** → watch **`ApproximateAgeOfOldestMessage`** (how long the oldest message has waited): alarm on it or scale workers on it. **THE trap:** scaling workers on **CPU** doesn't tell you messages are going stale — queue metrics do.

SNS — the megaphone (PUSH, one → MANY)

SNS (Simple Notification Service): publish one message to a **topic**, and SNS **pushes** it to ALL **subscribers** simultaneously — **Lambda, SQS, HTTP endpoints, email, SMS, mobile push**.

Critical nature: **SNS stores nothing.** It's a **megaphone, not a mailbox** — if a subscriber is down at shout-time, that subscriber misses the message. **Message filtering** lets each subscriber receive only messages matching a policy (e.g., billing gets only `order_type: refund`).

THE fan-out pattern — the exam's darling:

```
                        +--> [SQS: analytics queue] --> analytics workers
[Order service] --> [SNS topic]
                        +--> [SQS: fulfillment queue] --> fulfillment workers
                        +--> [SQS: fraud queue]      --> fraud checker
```

SNS → **multiple SQS queues**. Why not SNS alone? **Durability**: if the fraud service is down for an hour, its **queue holds the messages** until it recovers. Rule of thumb: **never SNS alone when reliability matters — fan out into SQS.**

- **Route different messages to different queues** → SNS topic + SQS subscribers, each with an **SNS filter policy** (subscribers get every message by default; the filter policy narrows it).
- Direction matters: **SQS queues subscribe to SNS** (SNS pushes into them). You **can't poll SNS**, and SNS can't pull from SQS.
- **S3 event notifications** → **many consumers**: an S3 event notification rule targets a **single** destination, and you can't attach the same event to several queues/topics. To deliver one S3 event to many queues → **S3** → **SNS** → **fan out to SQS** (or route via EventBridge).

SNS vs SES — both "send email", different jobs:

	SNS	**SES (Simple Email Service)**
Purpose	**System notifications / alerts** (pub/sub, many protocols)	**Application email**: transactional, bulk, marketing
Exam cue	"CloudWatch alarm emails the ops team", "monitor EC2 and notify"	"send order receipts / newsletters to customers"

THE trap: **monitoring/alarm notifications use SNS, not SES** — CloudWatch alarms and EventBridge publish to SNS; SES isn't an alarm target.

Kinesis Data Streams — the river

Real-time streaming with two superpowers SQS simply lacks:

1. **MULTIPLE consumers read the SAME data** (in SQS, one worker takes the message and it's gone; in the river, everyone on the bank sees the same water).
2. **REPLAY**: data is retained (**24 hours default, up to 365 days**) — rewind and reprocess.

Capacity unit = the **shard**: **1 MB/s in, 2 MB/s out**, and **ordering is guaranteed per shard** (partition key decides the shard). More throughput = more shards.

Kinesis's kill-shots vs SQS: **"reprocess yesterday's data"** or **"two applications must read the same stream"** → Kinesis Data Streams, never SQS.

More Streams facts:

- Each record gets a **sequence number** (unique per shard) — that's how per-shard order and replay positions work.
- **Resharding**: change capacity with `UpdateShardCount` (or split/merge shards — **SplitShard** adds capacity, **MergeShards** reduces it). Throughput errors (`ProvisionedThroughputExceeded`) → add shards. (Or use **on-demand** capacity mode and let AWS scale.)

- Retention note: older study material says "24h default, **7 days (168h) max**" — **outdated**; the max is now **365 days**.
- **Who reads the stream?**
  - **Lambda** — event source mapping polls the stream; best for simple, short per-record **transform/enrich** jobs. (Lambda reads Kinesis *Data Streams*; Firehose instead *invokes* Lambda as a transform.)
  - **KCL (Kinesis Client Library)** — custom, **long-running** consumer apps on EC2/containers. KCL tracks shard leases and checkpoints in a **DynamoDB table**: if consumers lag or the **shard iterator expires unexpectedly**, that lease table is short of capacity — **increase its write capacity**.
  - **Kinesis Data Analytics** (now **Managed Service for Apache Flink**) — real-time SQL/Flink analytics on the stream.
  - The **Kinesis Adapter** lets a KCL app consume **DynamoDB Streams** too.
- **Lambda@Edge + Kinesis** → collect and analyze **globally distributed clickstream/user activity** in real time.
- Typical fits: **IoT event streams**, clickstreams, logs, real-time metrics.

Kinesis Data Firehose — the delivery truck

Streams is a river; **Firehose** is a delivery truck on a schedule: **near-real-time (~60 seconds buffering)**, **ZERO code, fully managed**, and it delivers to fixed destinations: **S3, Redshift, OpenSearch, Splunk** (plus optional **Lambda transform** en route).

Trigger phrase: **"stream data into S3 with the LEAST operational effort / no code"** → **Firehose**. If the answer needs custom consumers or replay → Streams.

Firehose is **only a loader** into its fixed destinations — no replay, **no ordering guarantee**, no custom consumers, and it's **not a work queue** for batch jobs. Need custom processing → **Data Streams** (+ Lambda/KCL).

SQS vs Kinesis — both **buffer spikes without losing data**, but:

- Kinesis = **real-time streaming, ordering per shard, multiple readers, replay**; you pay for shards provisioned.
- SQS = simple **per-message work queue**, scales automatically, pay per request.
- "Real-time isn't actually needed — **cut cost**" → **replace Kinesis with SQS**. "Order + replay + multiple consumers" → keep Kinesis.

Amazon MQ — the lift-and-shift broker

Existing on-prem app speaking **open broker protocols — MQTT, AMQP, JMS, STOMP** (RabbitMQ / ActiveMQ)? Rewriting for SQS/SNS is work. **Amazon MQ** = managed RabbitMQ/ActiveMQ, so you **lift-and-shift without code changes**. Protocol names in the question = Amazon MQ.

Four-way decision table

Service	Model	Consumers	Storage/Replay	Pick when
SQS	Queue, PULL	ONE per message	4–14 days, no replay after delete	Decouple, buffer spikes, work distribution
SNS	Pub/sub, PUSH	MANY, all at once	**None**	Notify many systems instantly (fan out to SQS for durability)
Kinesis Streams	Stream (river)	MANY read SAME data	24h–365d, **replayable**	Real-time analytics, multiple readers, reprocessing
Firehose	Delivery pipe	Fixed destinations	Buffers ~60s	Dump streams into S3/Redshift/OpenSearch/Splunk, zero code
Amazon MQ	Traditional broker	Protocol-based	Broker semantics	MQTT/AMQP/JMS migration, no code changes

Question patterns

"*Web tier overwhelmed by traffic spikes; orders being lost*" → **SQS + Auto Scaling worker group** (queue buffers the spike, workers scale on depth) "*Messages must be processed in strict order with no duplicates*" → **SQS FIFO** (order + exactly-once; remember 300/3,000 msg/s cap) "*One event must notify three downstream systems, and none may miss it even if temporarily down*" → **SNS fan-out to multiple SQS queues** (queues hold messages for sleeping services) "*Some SQS messages are being processed twice*" → **Increase the visibility timeout** (processing outlasts the hiding window) "*Messages that repeatedly fail are blocking the queue*" → **Dead-Letter Queue with MaxReceiveCount** (quarantine poison messages) "*Clickstream analytics: two applications must consume the same real-time data, with ability to reprocess yesterday*" → **Kinesis Data Streams** (multiple consumers + replay = its two kill-shots) "*Stream IoT telemetry into S3 with the least operational overhead, no custom code*" → **Kinesis Data Firehose** (zero-code delivery truck to S3) "*Reduce cost of consumers polling an often-empty queue*" → **Long polling (20s)** (fewer empty receives) "*Migrate an on-prem RabbitMQ/ActiveMQ app using AMQP/JMS/MQTT without rewriting*" → **Amazon MQ** (protocol names = MQ) "*Message payloads are 1 MB, exceeding the SQS limit*" → **Store in S3, send a pointer** (256 KB cap) "*Each subscriber should only receive certain message types from a topic*" → **SNS message filtering** (filter policy per subscription) "*Messages are processed but keep reappearing in the queue*" → **Consumer must call DeleteMessage** after processing (SQS never auto-deletes on receive) "*Consumers get many empty responses even though messages exist*" → **Long polling** (`ReceiveMessageWaitTimeSeconds` > 0; short polling samples only some servers) "*Orders must not be processed until 10 minutes after they're placed*" → **SQS delay queue** (`DelaySeconds` , max 15 min) "*Existing Standard queue must now preserve order*" → **Create a new FIFO queue (name ends in `.fifo`)** (no in-place conversion) "*Time-sensitive queue: scale workers so no message waits too long*" → **Scale/alarm on** `ApproximateAgeOfOldestMessage` (not CPU) "*One S3 upload event must reach two different SQS queues*" → **S3 event → SNS topic → both SQS queues** (S3 notification has one destination per event) "*Email the ops team when a CloudWatch alarm fires on EC2*" → **SNS email subscription** (SES is for app/marketing email) "*Send transactional receipts and marketing newsletters to customers*" → **SES** "*EC2 in a private subnet with no internet must reach SQS*" → **Interface VPC Endpoint for SQS** (Gateway endpoints are only S3/DynamoDB) "*KCL consumers fail; shard iterator*

> *expires unexpectedly*" → **Increase write capacity of the KCL DynamoDB lease table** "*Kinesis stream hits throughput limits*" → **Add shards (`UpdateShardCount`)** or switch to on-demand mode "*Custom long-running processing of a Kinesis stream on EC2*" → **KCL app on Data Streams** (Firehose can't run custom consumers) "*Kinesis pipeline doesn't need real-time; reduce cost*" → **Replace with SQS**

Pocket card

Keyword	Answer
Decouple / buffer spikes	SQS
One message → one worker, PULL	SQS
Processed twice	Increase visibility timeout (default 30s)
Poison messages	DLQ + MaxReceiveCount
Empty-poll cost	Long polling (20s)
Strict order + exactly-once	FIFO (300 / 3,000 msg/s)
Message > 256 KB	S3 + pointer
Retention	4 days default / 14 max
One → many, PUSH, instant	SNS (stores nothing)
Notify many, durably	SNS → multiple SQS (fan-out)
Multiple consumers, same data, replay	Kinesis Data Streams (24h–365d)
Shard throughput	1 MB/s in, 2 MB/s out, order per shard
Stream → S3, zero code, ~60s	Firehose (S3/Redshift/OpenSearch/Splunk)
MQTT / AMQP / JMS / RabbitMQ	Amazon MQ
Workers scale on queue depth	ASG + CloudWatch queue metric
Messages going stale / time-sensitive	`ApproximateAgeOfOldestMessage`
Visibility timeout max	12 hours
Message reappears after processing	Call DeleteMessage (no auto-delete)
Short vs long polling	`ReceiveMessageWaitTimeSeconds` = 0 vs > 0 (max 20)
Delay delivery of new messages	Delay queue, 0–15 min
FIFO naming / conversion	Ends in `.fifo`; can't convert Standard → recreate
SQS from private subnet	Interface endpoint (PrivateLink)
Route messages to different queues	SNS + SQS subscribers + filter policies
One S3 event → many queues	S3 → SNS → SQS fan-out
Alarm/ops notifications	SNS (not SES)
Customer/marketing email	SES
Resize a stream	`UpdateShardCount` (MergeShards = shrink)
Custom long-running stream consumer	KCL (lease table in DynamoDB)

Keyword	Answer
Iterator expires / KCL lag	Raise lease-table write capacity
No real-time need, cheaper	SQS instead of Kinesis

Every architecture you've built so far — containers, Beanstalk apps, API Gateway backends — becomes bulletproof the moment you slide one of these buffers between its pieces; decoupling is the quiet superpower behind almost every correct SAA-C03 answer.

↑ Back to index

Section 18: EventBridge & Step Functions

The idea

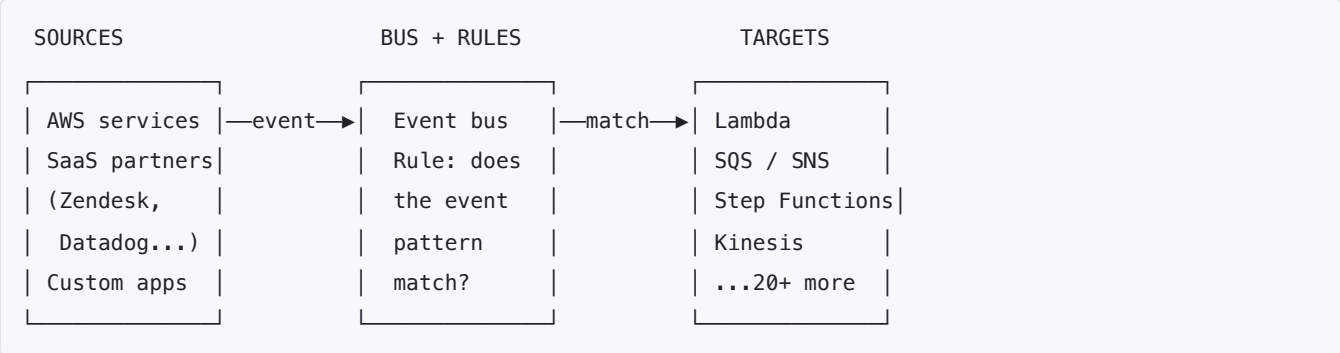
Modern AWS apps are event-driven: something happens somewhere ("an order was placed", "it's 2 AM", "a Zendesk ticket was created"), and something else needs to react. Two services own this space, and the exam loves both.

Picture a **railway control room**. **EventBridge** is the switchboard operator: trains (events) roll in from every direction — AWS services, outside SaaS companies, your own apps — and the operator reads each train's manifest and routes it down the right track to the right destination. **Step Functions** is the station master with a clipboard: it doesn't route random trains, it runs one *planned journey* from start to finish — "stop here, wait there, if the signal is red take the branch line, retry the crossing three times" — and writes down every step in a logbook.

Spell out the jargon: **EventBridge** (formerly **CloudWatch Events** — same engine, bigger brand) is a serverless **event bus**: a pipe that receives structured JSON events and matches them against **rules** to decide where they go. **Step Functions** is a **workflow orchestrator**: you define a **state machine** (a flowchart in JSON) that coordinates Lambda functions and other services step by step.

EventBridge: the event router of AWS

The flow is always the same three-parter:



- **Sources**: nearly every AWS service emits events ("EC2 instance state changed to stopped"), **SaaS partners** like Zendesk, Datadog, Auth0, Shopify deliver events straight onto a **partner event bus**, and your own apps can `PutEvents` custom events.
- **Rules** do **content-based filtering**: match on **any field** of the event JSON — source, detail-type, or deep inside the payload ("only orders where `amount > 1000`" style pattern matching). This rich filtering is EventBridge's signature move.
- **Targets**: Lambda, SQS, SNS, Step Functions, Kinesis, ECS tasks, another event bus... one rule can fan to multiple targets.

Extra features the exam pokes at:

- **Scheduled rules = serverless cron.** "Run a Lambda every night at 2 AM" → EventBridge schedule (cron or rate expression) → Lambda. **No EC2 instance running crontab** — that's the wrong answer every time.
- **Archive + replay**: record events on a bus, then **replay** them later — perfect for reprocessing after a bug fix.
- **Schema registry**: EventBridge discovers event structures and generates code bindings, so developers know what fields to expect.
- **Cross-account event buses**: send events from account A's bus to account B's bus — the standard pattern for centralizing events in a multi-account org.

THE trap: EventBridge vs SNS. Both "send events to targets", so exams bait you. **EventBridge** = **rich content filtering, AWS-service and SaaS event routing, scheduling, archive/replay**. **SNS** = **massive fan-out push** (millions of subscribers, mobile push, SMS, email). "Route events from a SaaS provider" or "filter on event content" → EventBridge. "Fan one message out to huge numbers of subscribers" → SNS.

Step Functions: the workflow orchestrator

A **state machine** is your flowchart; each box is a **state**:

State	What it does	Exam cue
Task	Do work (invoke Lambda, ECS, DynamoDB, etc.)	the worker step
Choice	If/else branching	"different path based on result"
Wait	Pause for time or until a timestamp	"wait 3 days then..."
Parallel	Run branches at the same time	"simultaneously"
Map	For-each over an array of items	"process each item in a list"
Retry/Catch	**Exponential backoff retries built in**	"retry with backoff **without custom code**"

That last row is gold: "*retry failed steps with exponential backoff without writing custom retry code*" → **Step Functions**, full stop.

THE trap: Lambda-chaining. The scenario: a multi-step process where Lambda A calls Lambda B calls Lambda C, and it's fragile, or the whole thing needs **more than 15 minutes** (the Lambda hard timeout), or the business needs **error handling and an audit of every step**. The answer is **never** "Lambdas calling Lambdas" — it's **Step Functions orchestrating the Lambdas**. You get visual workflow, per-step retry, and full execution history for free.

Standard vs Express workflows — memorize this table:

	Standard	Express
Max duration	**1 YEAR**	**5 minutes**
Pricing	per **state transition**	per execution/duration (cheaper at volume)
Execution history	full **audit history** in console	CloudWatch Logs only
Fit	long-running, **human approval**, audit-heavy	**high-volume, short** (IoT ingestion, streaming)

"Workflow includes a **human approval step**" or "runs for days" → **Standard**. "**Hundreds of thousands of short IoT events** per second, cost-sensitive" → **Express**.

Why it survives failures: Step Functions **maintains application state** — it tracks exactly which step each execution is on and logs the data passed between steps, so if a network fails or a component hangs, the workflow **resumes where it left off**. It stitches together Lambda, **Fargate/ECS**, SageMaker, DynamoDB, and more.

SWF — the legacy orchestrator

Amazon SWF (Simple Workflow Service) is the older, **non-serverless** way to coordinate multi-step work across **distributed** components: a managed **state tracker and task coordinator**. You run the workers yourself (typically on **EC2** or on-prem).

Actor	Role
Workflow starter	Kicks off a workflow execution (e.g. the app when an order is placed)
Decider	Holds the **control logic** — decides which task runs next
Activity worker	Does the actual task (can even be a **human** step)

- Guarantee to remember: SWF **assigns each task exactly once and never duplicates it**.
- It manages workflows — it is **not** a monitoring/notification service.
- Classic SWF-flavored scenarios: a **multi-step, multi-decision e-commerce checkout**, orchestrating **distributed business processes** with **EC2 + SWF**, media processing pipelines.

THE trap: on the modern exam, **Step Functions is the default answer** for orchestration (serverless, visual, built-in retries). Pick **SWF** only when the question explicitly needs it — **external signals** into a running workflow, **child workflows**, or an existing SWF/legacy setup. Want serverless *batch jobs* instead of orchestration? That's **AWS Batch**.

Question patterns

> "*Run a cleanup Lambda every night at 2 AM without managing servers.*" → **EventBridge scheduled rule** → **Lambda** (serverless cron, no EC2 crontab).

> "*Ingest events from Zendesk/Datadog into AWS and route them to Lambda.*" → **EventBridge partner event bus** (SaaS integration is EventBridge's exclusive turf).

> "*Order workflow needs retries, error handling, and a manual approval step that may take days.*" → **Step Functions Standard** (up to 1 year + human-in-the-loop).

> "*Multi-step data pipeline of Lambdas exceeds 15 minutes end to end.*" → **Step Functions** (orchestrate, don't chain — Lambda's 15-min cap is per function, the workflow can run far longer).

> "*Process each item in an array of records as a separate step.*" → **Map state** (built-in for-each).

> "*Need an audit trail showing the input/output of every step of each execution.*" → **Step Functions Standard** execution history.

> "*Route only events where a specific JSON field matches a value, to different targets.*" → **EventBridge rule with content-based filtering**.

> "*Millions of short-lived, high-frequency workflow executions at lowest cost.*" → **Step Functions Express**.

> "*Reprocess last week's events after fixing a bug.*" → **EventBridge archive + replay**.

> "*Retry a failing step with exponential backoff, no custom retry code.*" → **Step Functions Retry** (built into the state definition).

> "*Workflow must resume exactly where it left off after a component hangs or the network fails.*" → **Step Functions** (tracks state + logs data between steps).

> "*Coordinate tasks across distributed EC2 workers; each task must be assigned exactly once, never duplicated; uses deciders and activity workers.*" → **Amazon SWF** (legacy; otherwise prefer Step Functions).

> "*New serverless orchestration of multiple services with branching and retries.*" → **Step Functions** (not SWF — SWF isn't serverless).

Pocket card

Keyword	Answer
Serverless cron / "nightly job"	EventBridge scheduled rule
SaaS events (Zendesk, Datadog)	EventBridge partner event bus
Content-based event filtering	EventBridge rule
Replay past events	EventBridge archive + replay
Cross-account event routing	EventBridge cross-account bus
Huge fan-out push / SMS / mobile	SNS (not EventBridge)
Multi-step workflow, retries, audit	Step Functions
Workflow > 15 min	Step Functions (not Lambda chain)

Keyword	Answer
Human approval / up to 1 year	Standard workflow
High-volume short IoT / < 5 min	Express workflow
For-each over items	Map state
Branching logic	Choice state
Backoff retries, no code	Built-in Retry
Resume after failure / app state tracking	Step Functions
Starter / decider / activity worker	SWF (legacy)
Task assigned once, never duplicated	SWF
SWF vs new serverless workflow	Step Functions by default

EventBridge decides *which* work should happen when an event arrives; Step Functions makes sure multi-step work *finishes correctly* — and next we meet the services that watch everything else: CloudWatch, CloudTrail, and Config.

↑ Back to index

Section 19: CloudWatch, CloudTrail & AWS Config

The idea

Three services all "monitor" AWS, and the exam adores swapping them on you — this is **THE most-confused trio** on the test. Here's the split, and once you see it you can't unsee it.

Picture a **hospital**. **CloudWatch** is the bank of bedside monitors: heart rate, blood pressure, alarms when a value crosses a line — "**how is the patient doing?**" **CloudTrail** is the visitor logbook at the front desk: who came in, what they did, when, from where — "**WHO did WHAT?**" **AWS Config** is the patient's medical chart: what the patient's state *was* at any point in history, and whether it matches the doctor's orders — "**what did it look like at time T, and is it compliant?**"

Spell it out: **CloudWatch** = performance and health (**metrics**, **logs**, **alarms**). **CloudTrail** = **API audit** — every API call to AWS recorded. **AWS Config** = **configuration history + compliance** — snapshots of your resources' settings over time, checked against rules.

Read the question's verb: *slow/high/alert* → CloudWatch. *Who/deleted/changed by whom* → CloudTrail. *What did it look like / is it compliant* → Config.

CloudWatch — "how is it doing?"

Metrics. Numbers over time. EC2 sends **CPU, network, and disk-ops by default** — but **MEMORY and disk-space usage are NOT default metrics**. To get those you install the **CloudWatch agent** (custom metrics). This is a classic trap: "alert when instance memory exceeds 80%" → the answer involves the **agent + custom metric**, never a default metric.

- Agent-only metrics to recognize: **memory, disk space, disk swap, page file utilization** — plus the agent **collects instance/application log files** into CloudWatch Logs (it replaces the old CloudWatch Logs agent).
- Once memory is a custom metric, an **Auto Scaling group can scale on it** (target tracking / step scaling on the custom metric).

Alarms. Watch a metric, cross a threshold, take action: notify **SNS**, trigger **Auto Scaling**, or run **EC2 actions** (stop/terminate/**reboot**/recover). "Monitor a metric and **email** the team" → **CloudWatch alarm** → **SNS topic** (email subscription) — SNS, not SES.

- **Recover** action: only for **system status check** failures (AWS-side hardware/host problems), **not instance status check** failures, and only for **EBS-backed** instances (instance store isn't supported). The recovered instance keeps its instance ID, IPs, and EBS volumes.
- Recovery is a **CloudWatch alarm action** — an EventBridge (CloudWatch Events) rule can't directly trigger EC2 recovery.

Composite alarms combine several alarms with AND/OR to **reduce alert noise** — "too many alarms firing at once" → composite alarm.

Logs. Log **groups** (per app) contain log **streams** (per instance/container). Retention configurable **1 day to 10 years** (default: forever). Three power features:

- **Metric filters**: turn a log pattern into a metric — "**count ERROR lines** and alarm when > 100/min" → metric filter + alarm.
- **Logs Insights**: SQL-ish **query language** for interactive log analysis.
- **Subscription filters**: stream log events in **real time** to **Kinesis or Lambda** — "process logs as they arrive" → subscription filter.

Dashboards are **cross-region** — one pane of glass for a global app.

CloudTrail — "WHO did WHAT?"

Every API call — console click, CLI command, SDK call — is recorded: identity, time, source IP, parameters.

- **90-day event history is free** and automatic. For longer retention, create a **Trail** → **S3** (and make it a **multi-region trail** — a best practice and a favorite answer).
- **Management events** (control plane: create/terminate/modify resources) are logged **by default**. **DATA events** (S3 **object-level** reads/writes, Lambda invokes) are **opt-in and cost extra** — THE trap: "*who deleted the object in the bucket?*" → default CloudTrail can't say; you need **data events enabled**.
- **Log file integrity validation**: cryptographically prove logs weren't tampered with — the "forensics/court" keyword.
- **CloudTrail Insights**: detects **unusual API activity** (sudden spike in TerminateInstances).
- Trail log files delivered to S3 are **encrypted with S3 server-side encryption (SSE) by default** — nothing to configure. (Opt for **SSE-KMS** if you need your own key.)
- Scope: CloudTrail logs **AWS account activity (API calls)** for governance, compliance, and operational/risk auditing — **not your application's internals** (that's X-Ray tracing / CloudWatch Logs). Conversely, **X-Ray doesn't record AWS API calls**.
- S3 nuance: CloudTrail gives API-level bucket/object tracking, but **not per-request details like referrer and turn-around time** → that's **S3 server access logs**.

AWS Config — "what did it look like, and is it compliant?"

- A **configuration recorder** snapshots your resources' settings every time they change, giving you a **timeline** per resource — "what did this security group look like **last Tuesday**?" → Config.
- **Config rules** (AWS-**managed** or **custom** Lambda-backed) continuously evaluate compliance: "no security group may allow SSH from 0.0.0.0/0". Results roll up to a **compliance dashboard**.
- **Auto-remediation**: a noncompliant resource can trigger an **SSM Automation document** to fix itself — "automatically close world-open SSH" → **Config rule + remediation action**.
- Example managed rule: `IAM_PASSWORD_POLICY` — checks the account's IAM password policy for compliance.

THE trap: Config does NOT prevent anything. It **detects** and (optionally) remediates *after the fact*. If the question says "**prevent** users from ever doing X" → that's **SCPs** (or IAM), not Config.

Config vs Trusted Advisor: Trusted Advisor only gives **best-practice recommendations** (cost, security, fault tolerance, performance, service limits) — you **can't define your own rules** in it. Custom compliance rules → **Config**.

The "who watches what" lineup

Service	Answers	Cue
CloudWatch	How is it performing?	metrics, logs, alarms
CloudTrail	Who did what (API calls)?	audit, "who deleted/changed"
AWS Config	What did it look like / is it compliant?	config history, rules
Trusted Advisor	Am I following best practices?	recommendations, service limits
IAM Credential Report	Status of every user's credentials	passwords/access keys/MFA age
VPC Flow Logs	What network traffic hit the ENI?	IPs, ports, ACCEPT/REJECT
X-Ray	Which hop in the request is slow?	distributed tracing

CloudWatch Logs vs VPC Flow Logs: CloudWatch Logs holds **application/OS log lines**; Flow Logs capture **network traffic metadata** (and can be *delivered into* CloudWatch Logs or S3).

```
Question verb          →  Service
──────────────────────────────────
"is it slow/unhealthy?"→  CloudWatch
"who changed it?"      →  CloudTrail
"what was it / legal?" →  Config
```

X-Ray one-liner: distributed **tracing** across microservices — follows one request through every hop. "*Which microservice adds the latency?*" → **X-Ray** (not CloudWatch — CloudWatch tells you *a* service is slow; X-Ray tells you *which hop* in the request chain).

Question patterns

> "*Determine who terminated a production EC2 instance last week.*" → **CloudTrail** (API audit = who did what).

> "*Alert when application memory usage exceeds 80% on EC2.*" → **CloudWatch agent + custom metric + alarm** (memory is not a default metric).

> "*Trigger an alarm when the app logs more than 100 ERROR lines in 5 minutes.*" → **Metric filter on CloudWatch Logs + alarm**.

> "*Show the configuration of a security group as of last Tuesday, and flag any SG allowing SSH from the internet.*" → **AWS Config** (timeline + managed rule).

> *"Automatically fix noncompliant security groups without manual work."* → **Config rule + SSM Automation remediation**.

> *"A request spans 10 microservices; find which one is slow."* → **X-Ray** (distributed tracing).

> *"Process log entries in real time as they are written."* → **CloudWatch Logs subscription filter** → **Kinesis/Lambda**.

> *"Identify who downloaded a specific S3 object."* → **CloudTrail data events** (object-level = opt-in, extra cost).

> *"Keep API activity records for 7 years, tamper-evident."* → **Trail** → **S3 + log file integrity validation** (free history is only 90 days).

> *"Prevent anyone in the org from disabling CloudTrail."* → **SCP** (Config detects; SCP prevents).

> *"Scale an Auto Scaling group on EC2 memory utilization."* → **CloudWatch agent custom memory metric + ASG scaling policy**.

> *"Monitor EC2 metrics and email the ops team when a threshold is crossed."* → **CloudWatch alarm** → **SNS** (not SES).

> *"Automatically recover an EC2 instance when the underlying host fails."* → **CloudWatch alarm on StatusCheckFailed_System + recover action** (EBS-backed only; not for instance status check failures).

> *"Encrypt CloudTrail log files in S3 — what must you do?"* → **Nothing** (SSE by default; choose SSE-KMS only for your own key).

> *"Define custom compliance rules, e.g. check the IAM password policy."* → **AWS Config rule (`IAM_PASSWORD_POLICY`)** (Trusted Advisor only recommends).

> *"Get best-practice recommendations on cost, security, and service limits."* → **Trusted Advisor**.

> "*List all IAM users and the status of their passwords, access keys, and MFA.*" → **IAM Credential Report**.

Pocket card

Keyword	Answer
Performance, health, alarms	CloudWatch
Who did what / API audit	CloudTrail
Config history / compliance	AWS Config
EC2 memory or disk-space metric	CloudWatch agent (not default)
Count log pattern → alarm	Metric filter
Query logs interactively	Logs Insights
Real-time log streaming	Subscription filter → Kinesis/Lambda
Too many alarms / noise	Composite alarm
S3 object-level "who"	CloudTrail data events (opt-in)
Logs beyond 90 days	Trail → S3, multi-region
Tamper-proof logs	Log file integrity validation
Unusual API spike	CloudTrail Insights
Auto-fix noncompliance	Config + SSM remediation
PREVENT actions	SCP (never Config)
Trace across microservices	X-Ray
ASG scale on memory	Agent custom metric + scaling policy
Metric threshold → email	CloudWatch alarm → SNS
Alarm EC2 actions	Stop / terminate / reboot / recover
Auto-recover EC2	System status check failure, EBS-backed only
CloudTrail log encryption	SSE by default (SSE-KMS optional)
Custom compliance rules	Config (not Trusted Advisor)
Best-practice recommendations	Trusted Advisor
Per-user credential status	IAM Credential Report
App logs vs network traffic	CloudWatch Logs vs VPC Flow Logs

Now you know who's watching the house — next comes who holds the keys: KMS and CloudHSM.

↑ Back to index

Section 20: KMS & CloudHSM

The idea

Encryption is easy to say and hard to do well — the hard part is never the math, it's **key management**: where do keys live, who can use them, how do you rotate them, and how do you prove who touched them?

Picture a **bank vault with a signing register**. **KMS** (**Key Management Service**) is the bank's shared vault: your keys live in safe-deposit boxes, the bank's staff (AWS) run the vault, almost every AWS service knows how to walk up to it, and **every single time a key is used, it's written in the register** — that register is **CloudTrail**, and that audit trail is **KMS's superpower**. **CloudHSM** is different: it's you renting an **entire private vault room** — dedicated hardware, your rules, and even the bank staff **cannot see inside**.

Spell out the jargon: **KMS** stores and manages **encryption keys** centrally; it integrates with essentially every AWS service (S3, EBS, RDS, Lambda env vars...). **HSM** = **Hardware Security Module**, a tamper-resistant physical device that performs crypto operations.

KMS key types

Type	Cost	Rotation	Control
AWS-managed (`aws/s3` etc.)	free	automatic	none — AWS runs it
Customer-managed key (CMK)	**$1/month**	**optional auto-rotation (yearly)**	full: your key policy, disable/delete
AWS-owned	invisible	—	none; shared across accounts, you never see it

Need control — your own key policy, cross-account grants, rotation you can point to in an audit → **customer-managed key**.

Key policies + IAM: BOTH must allow. Access to use a key = the **key policy** says yes **and** the caller's **IAM policy** says yes. A user with `kms:*` in IAM still gets denied if the key policy doesn't permit them.

KMS permissions follow the principal. When Lambda/ECS/EC2 uses a key, the caller is the **role** (execution role, task role, instance role) — grant *that* role in the key policy, not the function or instance ARN. KMS is IAM-based: there's **no username/password** to "enable encryption".

What KMS is (and isn't): it holds **keys** and does encryption operations for files, strings, S3 objects, EBS, secrets — it does **not store secrets** themselves. **Secrets Manager always encrypts with a KMS key** (your CMK or the default `aws/secretsmanager`), so Secrets Manager depends on KMS. Lambda **environment variables are KMS-encrypted at rest by default**, but anyone with console access to the function can still read them — for real secrets, encrypt with a CMK using **encryption helpers** or, better, use Secrets Manager.

Envelope encryption — the ≤4KB rule

KMS will only directly encrypt data up to 4KB. For anything bigger, use the **GenerateDataKey** pattern (**envelope encryption**):

```
You: "GenerateDataKey" ──► KMS
KMS returns: ┌─ plaintext data key  (use it to encrypt your 100MB locally, then discard)
             └─ encrypted data key  (store it next to the ciphertext)
Decrypt later: send encrypted data key ─► KMS unwraps it ─► decrypt locally
```

The big data never travels to KMS; KMS only wraps/unwraps the small **data key**. "Encrypt a large file with KMS" → **envelope encryption / GenerateDataKey**, never "KMS encrypts it directly".

Deletion, multi-Region, and Bucket Keys

THE trap: there is NO immediate key deletion. Scheduling deletion imposes a **mandatory 7–30 day waiting period** (default 30). Any answer that "deletes the KMS key immediately" is wrong — deleting a key makes everything it encrypted unrecoverable, so AWS forces you to wait. Need to stop use *right now*? **Disable** the key (instant, reversible).

Multi-Region keys: the **same key material** replicated into other Regions with matching key IDs — **encrypt in one Region, decrypt in another**. The use cases: **DynamoDB global tables, cross-Region disaster recovery**, client-side encryption for globally moving data.

S3 Bucket Key recap: instead of one KMS call per object, S3 uses a short-lived bucket-level key → **far fewer KMS API calls** → **cheaper** (KMS request costs drop ~99%), at the price of **coarser CloudTrail audit granularity** (you see bucket-key events, not one event per object).

CloudHSM — when the shared vault isn't enough

CloudHSM = **dedicated, single-tenant** hardware security modules in your VPC.

- **FIPS 140-2 LEVEL 3** validated — **KMS is Level 2**. The moment a question says "Level 3", the answer is CloudHSM. (FIPS 140-2 is the US government's crypto-module security standard; Level 3 adds physical tamper-resistance requirements.)
- **YOU manage the keys; AWS has no access to them** — AWS manages the hardware only.
- Use cases: strict **regulatory/compliance** mandates, **BYOK**-style full key custody, **SSL/TLS offload**, running your own certificate authority.
- **KMS custom key store**: authorize KMS to use **your CloudHSM cluster** as the key backend — you keep KMS's service integrations and CloudTrail audit, while key material lives in single-tenant HSMs you control.
- CloudHSM stores **keys**, not passwords/secrets (that's Secrets Manager).
- **THE trap:** run **at least two HSMs in different AZs**. With a single HSM, keys created since the last **daily backup** can be lost — and if you lose your HSM credentials, **AWS cannot recover your keys**.

Decision rule: "AWS must not be able to access the keys" / "FIPS 140-2 Level 3" / "dedicated hardware" → **CloudHSM**. Everything else — integration, audit, low cost, ease → **KMS**.

Question patterns

> "*Security team must audit every use of an encryption key.*" → **KMS + CloudTrail** (every key use is logged — KMS's superpower).

> "*Encrypt a 100MB file using KMS.*" → **Envelope encryption via GenerateDataKey** (KMS only encrypts ≤4KB directly).

> "*A compromised key must be deleted immediately.*" → **Impossible — 7–30 day mandatory waiting period; disable it now instead**.

> "*Compliance requires FIPS 140-2 Level 3 validated modules.*" → **CloudHSM** (KMS is Level 2).

> "*Keys must be inaccessible even to AWS.*" → **CloudHSM** (single-tenant, customer-controlled).

> "*Encrypt data in us-east-1, decrypt the same ciphertext in eu-west-1 (DR/global tables).*" → **KMS multi-Region keys**.

> "*Keys must rotate automatically every year under company control.*" → **Customer-managed key with auto-rotation enabled**.

> "*KMS request costs exploded from millions of S3 objects with SSE-KMS.*" → **Enable S3 Bucket Key** (fewer KMS calls; accept coarser audit).

> "*User has full IAM KMS permissions but still gets AccessDenied on a key.*" → **Key policy doesn't allow them — BOTH key policy and IAM must permit**.

> "*Want KMS integration with S3/EBS, but key material must live in dedicated HSMs you control.*" → **KMS custom key store backed by CloudHSM**.

> "*CloudHSM must not lose keys if an HSM fails.*" → **Cluster of 2+ HSMs across AZs** (a single HSM risks keys created since the last daily backup).

> "*Lambda needs to decrypt with a CMK — what goes in the key policy?*" → **The Lambda execution role** (permissions follow the principal).

> "*Sensitive values in Lambda environment variables are visible in the console.*" → **Encrypt with a CMK via encryption helpers, or move them to Secrets Manager** (default encryption is at rest only).

Pocket card

Keyword	Answer
Audit every key use	KMS + CloudTrail
Free, AWS-rotated key	AWS-managed key
Your policies, $1/mo, yearly rotation	Customer-managed key
Encrypt > 4KB	Envelope encryption / GenerateDataKey
Delete key now	Impossible — 7–30 day wait (disable instead)
Encrypt/decrypt across Regions	Multi-Region keys
Cut KMS costs on S3	S3 Bucket Key
FIPS 140-2 Level 3	CloudHSM
"AWS cannot see keys" / dedicated hardware	CloudHSM
SSL offload / own CA	CloudHSM
Access needs key policy AND IAM	Both must allow
Lambda/ECS uses a key	Grant the role (permissions follow the principal)
KMS + your own HSMs	Custom key store (CloudHSM)
CloudHSM high availability	2+ HSMs in different AZs
Lost CloudHSM credentials	Keys unrecoverable — AWS can't help
Store passwords?	Not KMS/CloudHSM — Secrets Manager

You now hold the keys — next, the safes where applications keep their passwords: Secrets Manager and Parameter Store.

↑ Back to index

Section 21: Secrets Manager & Parameter Store

The idea

Every app has secrets — database passwords, API keys, license codes — and the worst place for them is where developers love to put them: hardcoded in the source. AWS gives you two lockboxes, and the exam's whole game is making you pick the right one.

Picture two places to keep a spare house key. **Parameter Store** is a **key hook board in a locked closet**: cheap (free, in fact), tidy, labeled hooks arranged in neat rows — grab whatever key you need. **Secrets Manager** is a **hotel keycard system**: it costs money, but it **automatically reissues the keycards on a schedule**, so even a stolen card goes stale on its own.

Spell out the jargon: both are managed stores for configuration values and secrets, both **encrypt with KMS**, both are fetched by apps at runtime via API. **THE differentiator is AUTOMATIC ROTATION** — and it belongs to Secrets Manager alone.

Secrets Manager — rotation is the whole point

- **Native automatic rotation**: for **RDS, Redshift, and DocumentDB** credentials, Secrets Manager rotates the password using a **built-in Lambda** it manages for you — zero custom code. For anything else, you supply a custom rotation Lambda. **The moment a question says "automatically rotate" → Secrets Manager. Always. No exceptions.**
- **$0.40 per secret per month** (+ small per-10k-API-calls fee).
- Secrets up to **64KB**.
- **Cross-account sharing** via resource policies — "share a secret with another AWS account" → Secrets Manager.
- Encrypted with **KMS**, every access logged in **CloudTrail**.
- Rotation happens **without downtime** — the app just fetches the current value at connect time. During rotation Secrets Manager keeps **multiple versions** tagged with staging labels (`AWSCURRENT`, `AWSPENDING`, `AWSPREVIOUS`); Parameter Store has no such rotation versioning.
- It stores and rotates secrets — it does **not mint short-lived tokens** (that's STS/Cognito).

Parameter Store (SSM Parameter Store) — free and hierarchical

- **Standard tier is FREE**: up to **4KB** per parameter, **10,000 parameters**.
- **Hierarchical paths**: `/prod/db/password`, `/dev/db/password` — fetch a whole branch with one `GetParametersByPath` call, and scope IAM permissions per path ("dev can read `/dev/*` only").
- **SecureString** type = **KMS-encrypted** value — yes, Parameter Store can store encrypted secrets too.
- **Advanced tier**: **8KB**, parameter **policies** (e.g., expiration notification), higher throughput — small fee.
- **NO native rotation.** That's the dividing line.

"Store application config / feature flags / license keys / AMI IDs" or "**cheapest** option" → **Parameter Store**. It's built for broad, mostly non-secret config — URLs, AMI IDs, instance IDs, DB connection strings, environment variables — plus SecureString for **secrets that change infrequently and don't need rotation** (cheaper than Secrets Manager when you have lots of static values).

The comparison table (memorize this)

	Secrets Manager	**Parameter Store**
Automatic rotation	**YES — native for RDS/Redshift/DocumentDB**	**NO**
Cost	$0.40/secret/month	**FREE** (standard tier)
Max size	64KB	4KB (8KB advanced)
Hierarchy	flat names	**paths** `/prod/db/...`
Cross-account sharing	**yes**	no (standard)
KMS encryption	always	SecureString type
Built for	secrets that must rotate	config + secrets that don't

Widen the lens — the four "secret-ish" services the exam mixes up:

Service	**Use it for**	**Rotation / renewal**	**Stores passwords/API keys?**
Secrets Manager	DB credentials, API keys, tokens	**Automatic rotation**	Yes
SSM Parameter Store	App config, env vars, static secrets (SecureString)	No native rotation	Yes (SecureString)
KMS	**Encryption keys** + encrypt/decrypt operations	Key rotation only	**No** — not a secret store
ACM	Public/private **SSL/TLS certificates** for ALB, CloudFront, API Gateway	**Automatic certificate renewal**	No

```
"rotate automatically"? ──yes──▶ Secrets Manager
        │no
"cheapest / config / hierarchy"? ──▶ Parameter Store
```

The runtime pattern (the "fix the hardcoded password" answer)

Both integrate with **Lambda and ECS environment injection**, and the exam-blessed architecture is always the same:

```
App (EC2/Lambda/ECS)
  │  IAM role attached — NO credentials in code or env files
  ▼
GetSecretValue / GetParameter (at runtime)
  ▼
KMS decrypts transparently ──► app connects to DB
```

ECS injection specifics: in the task definition, reference a Secrets Manager secret or Parameter Store parameter in the container definition's `secrets` parameter (→ injected as **environment variables**) or `secretOptions` (→ **log driver configuration**). Works on both **EC2 and Fargate** launch types; the **task execution role** needs permission to read the secret/parameter (and the KMS key).

THE trap: any answer where credentials sit in code, an AMI, a config file in the repo, or a plain environment variable is wrong. The fix is **fetch at runtime via IAM role** — and if rotation is mentioned, Secrets Manager, so the password changes **without downtime** and the app never notices.

Question patterns

> "*Automatically rotate the RDS database password every 30 days.*" → **Secrets Manager** (native RDS rotation via built-in Lambda).

> "*Store application configuration values at the lowest possible cost.*" → **Parameter Store standard tier** (free).

> "*Organize config per environment like /prod/db/url and /dev/db/url, with IAM scoped per environment.*" → **Parameter Store hierarchical paths**.

> "*Share a database secret with an application in another AWS account.*" → **Secrets Manager cross-account resource policy**.

> "*Developers hardcoded the DB password in the code — what's the fix?*" → **Secrets Manager + IAM role, app fetches at runtime** (never in code/env files).

> "*Store an encrypted license key for free.*" → **Parameter Store SecureString** (KMS-encrypted, still free).

> "*Rotate credentials without application downtime.*" → **Secrets Manager** — app fetches the current secret at connection time.

> "*Secret is 20KB of JSON credentials.*" → **Secrets Manager** (64KB limit; Parameter Store caps at 4/8KB).

> "*Audit who accessed the database secret.*" → **Either service + CloudTrail** (all API access is logged).

> "*Inject a DB password into ECS containers (EC2 or Fargate) as an environment variable.*" → **`secrets` parameter in the container definition referencing Secrets Manager/Parameter Store** (`secretOptions` for log config).

> "*Store many application parameters that change rarely and need no rotation, cost-effectively.*" → **Parameter Store SecureString** (Secrets Manager is overkill and costs per secret).

> "*Need SSL/TLS certificates with automatic renewal for an ALB.*" → **ACM** (not Secrets Manager, not KMS).

> "*Where should encryption keys be managed?*" → **KMS** — it manages keys, it does not store passwords.

Pocket card

Keyword	Answer
"Automatic rotation" (any phrasing)	Secrets Manager
RDS/Redshift/DocumentDB creds	Secrets Manager (built-in rotation Lambda)
Cheapest / free config store	Parameter Store standard
Hierarchical paths /prod/...	Parameter Store
Cross-account secret sharing	Secrets Manager
Encrypted param, free	Parameter Store SecureString
> 8KB secret	Secrets Manager (64KB)
Hardcoded password fix	Secrets Manager + IAM role runtime fetch
Rotation without downtime	Secrets Manager
$0.40/secret/month	Secrets Manager
Many static params, no rotation, cheap	Parameter Store SecureString
ECS env var from a secret	`secrets` in container definition
ECS log driver secret	`secretOptions`
Encryption keys (not secrets)	KMS

Keyword	Answer
SSL/TLS certs, auto-renew	ACM

Keys from Section 20 encrypt these secrets, CloudTrail from Section 19 audits every peek at them — the monitoring, key, and secret stories all click together, and that's exactly how the exam will test you.

↑ Back to index

Section 22: WAF, Shield & the Security Service Zoo

The idea

Picture your application as a nightclub. Traffic pours in from the internet — most of it friendly, some of it trying to sneak SQL injection through the front door, some of it a mob of 10 million bots trying to trample the entrance. AWS gives you a **layered security crew**: a smart bouncer at the door who reads every ID (WAF), riot police who handle mobs (Shield), a head of security who makes sure *every* club in your chain follows the same door policy (Firewall Manager), and a whole back office of detectives watching camera feeds for suspicious behavior (GuardDuty, Macie, Inspector, and friends).

The exam loves this section because it's mostly a **matching game**: read the scenario, name the right crew member. Let's meet them.

WAF — the Layer 7 bouncer

AWS WAF (Web Application Firewall) works at **Layer 7** — the application layer — meaning it actually *reads HTTP requests*: URLs, headers, query strings, body content. Jargon check: "Layer 7" is the top of the OSI networking model, where HTTP lives; Layers 3/4 are raw IP packets and TCP connections.

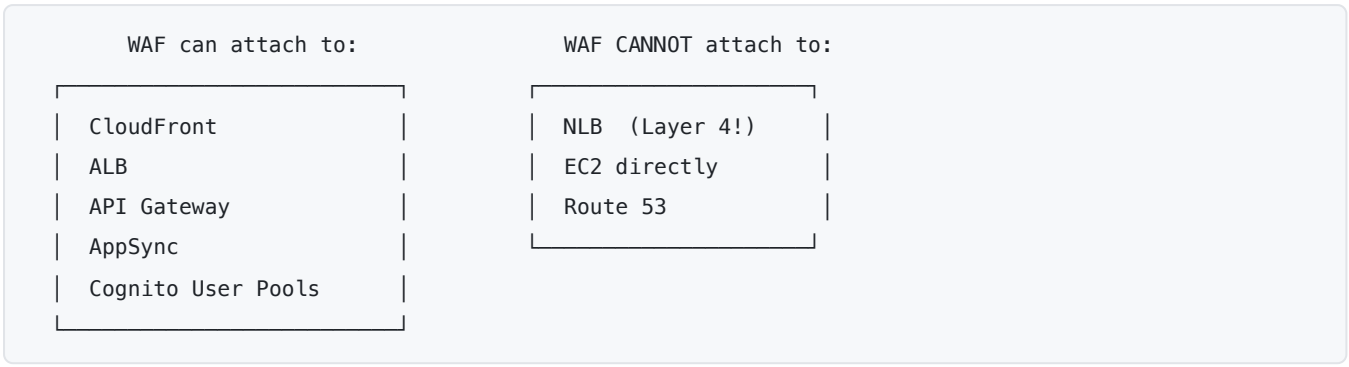
Because WAF reads HTTP, it can catch things a packet firewall never could:

Rule type	What it catches
SQL injection rules	`' OR 1=1 --` hiding in a form field
XSS rules	Cross-Site Scripting — malicious `<script>` tags in input
Rate-based rules	Too many requests **per IP** in 5 minutes → block that IP
Geo-match	Block or allow by country
IP sets	Explicit allow/deny lists of IP addresses
Managed rule groups	Pre-built rule packs from AWS/vendors (e.g., **OWASP Top 10** core rule set)

Exam gold: "*rate limit requests per IP*" → **WAF rate-based rule**. Every time.

And a lovely testing feature: **Count mode** — the rule *counts* matches instead of blocking, so you can test rules against production traffic without breaking real users. "Evaluate a new rule without impacting users" → Count mode.

Where can WAF attach? Only to **Layer 7 fronts**:



Also: **NACLs and Security Groups can't block SQLi/XSS** — they only see IPs/ports. HTTP-content attacks → **WAF**.

THE trap: "*attach WAF to a Network Load Balancer*" → **impossible**. NLB is Layer 4 (TCP) — it never parses HTTP, so a Layer 7 firewall has nothing to read. WAF needs an L7 front: **CloudFront, ALB, API Gateway, AppSync, or Cognito**.

Shield — the riot police (DDoS)

DDoS = Distributed Denial of Service — thousands of machines flooding you with traffic to knock you offline.

- **Shield Standard**: **free, automatic, for everyone**. Protects against common **Layer 3/4** attacks (SYN floods, UDP reflection). You already have it — nothing to enable.
- **Shield Advanced**: **~$3,000/month** (1-year commitment). Adds **Layer 7 DDoS protection**, a **24/7 DDoS Response Team (DRT)** of human experts you can call mid-attack, and **cost protection** — AWS refunds the scaling charges the attack caused.

Memory hook: any question mentioning **L7 DDoS, expert help, or refunds for attack-driven scaling** → **Shield Advanced**. A plain "protect against common DDoS at no cost" → **Shield Standard** (it's already on). Shield Advanced also gives **near-real-time visibility into attacks** and integrates with WAF.

THE trap: WAF alone is not a DDoS answer — rate-based rules help with L7 floods, but large-scale DDoS resilience = **Shield + WAF** together.

DDoS-resilient architecture (the exam's reference answer):

- **CloudFront** (and Route 53) at the edge to absorb traffic
- **ALB + Auto Scaling group** to scale out under load
- Databases (e.g., **RDS**) in **private subnets**
- **CloudWatch alarms** on unusual **NetworkIn / CPU** spikes
- **Shield (Advanced for L7/DRT) + WAF** on CloudFront/ALB

Firewall Manager — the head of security for the whole chain

You have 50 AWS accounts in an **AWS Organization**. You want every ALB in every account — including accounts created *next month* — to automatically get your WAF rules. Manually? Nightmare. **AWS Firewall Manager** is the **central policy manager**: define security policies once (WAF rules, Shield Advanced, Security Group rules, Network Firewall rules) and it **auto-applies them across the organization**, including to **new accounts and new resources as they appear**.

Prerequisites (exam-tested): requires **AWS Organizations** + **AWS Config** enabled.

THE trap: "*every new account automatically gets the security policies*" → that word **automatically across accounts** means **Firewall Manager**, not WAF alone. WAF is the rule; Firewall Manager is the rollout machine.

THE trap: **Firewall Manager and Security Hub don't stop attacks themselves.** Firewall Manager only *administers* WAF/Shield Advanced/SG/Network Firewall policies across accounts; Security Hub only *reports* alerts and posture. The thing that actually blocks is **WAF** (L7) / **Shield** (DDoS) / **Network Firewall** (VPC).

Network Firewall — the VPC-wide inspector

AWS Network Firewall is a **managed firewall for your whole VPC**, inspecting traffic at **Layers 3 through 7** — think intrusion prevention, domain filtering, stateful rules for *all* traffic entering/leaving the VPC, not just HTTP to one ALB. Scenario says "*inspect all traffic in the VPC*" or "*filter outbound traffic to specific domains for the entire VPC*" → **Network Firewall**.

- **Stateful IDS/IPS** built on open-source **Suricata** rules; filters traffic via **IGW, NAT gateway, VPN, or Direct Connect**.
- Can do **deep packet inspection**, allow only known AWS service domains/IPs, block lists of bad **domains (FQDN)**, and detect protocols like HTTPS **regardless of port** (a NAT Gateway can't filter domains).
- Deployed at the **VPC level** (firewall endpoints in dedicated subnets), not per subnet; it **can't attach to an ALB** and can't by itself restrict S3 bucket access to one VPC (use bucket policy + VPC endpoint).

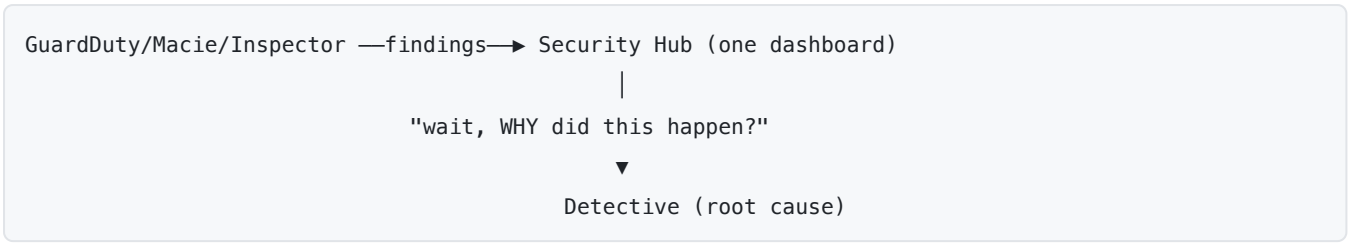
Lookalikes, not firewalls: Network Access Analyzer (VPC feature) *reports* unintended network access paths against your requirements — **no deep packet inspection**. **Traffic Mirroring** *copies* ENI traffic to an appliance/NLB/GWLB for out-of-band analysis. **Detective** can't create a firewall either.

The detection zoo — one-liners you must know cold

These are pure matching questions. Burn in the one-liners:

Service	One-liner
GuardDuty	**ML threat detection** on **CloudTrail, VPC Flow Logs, DNS logs** — no agents. Finds **cryptomining, unusual API calls, compromised credentials**
Macie	**PII / sensitive data discovery in S3** (ML finds credit cards, SSNs) — data classification, shows where sensitive data lives and how it's accessed
Inspector	**Vulnerability scanner** — **CVEs** on **EC2 (via SSM agent), ECR container images, Lambda**; automated security/compliance assessment of apps (agent collects package & config info)
Security Hub	**Aggregation dashboard** — collects findings from all the above + runs compliance standards (CIS, PCI)
Detective	**Investigate AFTER a finding** — builds a graph of relationships to find **root cause**
Artifact	**Download AWS compliance reports** (SOC, PCI, ISO) to hand to auditors

Mental model of the flow:



THE trap: "*GuardDuty vs Inspector*" — GuardDuty watches **behavior in logs** (threats happening now); Inspector scans **software for vulnerabilities** (holes that *could* be exploited). Threat = GuardDuty; CVE = Inspector.

THE trap: "*Macie for EC2 or RDS*" → no. **Macie is S3-only.**

Question patterns

> "*Block SQL injection attacks against an application behind an ALB*" → **WAF on the ALB** (L7 rules read HTTP).

> "*Limit each client IP to 2,000 requests per 5 minutes*" → **WAF rate-based rule** ("per IP rate limit" is WAF's signature move).

> "*Protection against common DDoS attacks at no additional cost*" → **Shield Standard** (free, automatic, L3/L4).

> "*Company suffered a large DDoS, wants expert support during attacks and refunds for attack-related scaling costs*" → **Shield Advanced** (DRT + cost protection = the $3k tier).

> "*Ensure WAF rules are applied to all ALBs across 50 accounts, including future accounts*" → **Firewall Manager** (org-wide auto-apply; needs Organizations + Config).

> "*Identify S3 buckets containing personally identifiable information*" → **Macie** (PII in S3, full stop).

> "*Alert when EC2 instances are used for cryptocurrency mining or credentials are compromised*" → **GuardDuty** (ML on CloudTrail/Flow/DNS logs, agentless).

> "*Continuously scan EC2 instances and container images for software vulnerabilities (CVEs)*" → **Inspector** (SSM agent on EC2, ECR, Lambda).

> "*Single pane of glass for security findings across all accounts and services*" → **Security Hub** (the aggregation dashboard).

> "*After a GuardDuty finding, analyze and identify the root cause of the incident*" → **Detective** (post-finding investigation graph).

> "*Auditor requests AWS's SOC 2 / PCI compliance reports*" → **Artifact** (self-service report downloads).

> "*Block SQL injection — options include NACL rules*" → **WAF** (NACLs/SGs only see IPs and ports).

> "*Design a web app resilient to DDoS*" → **CloudFront + Shield/WAF in front of ALB + ASG, DB in private subnets, CloudWatch alarms on NetworkIn/CPU** (WAF alone isn't enough).

> "*Which service prevents attacks: WAF, Security Hub, or Firewall Manager?*" → **WAF** (Security Hub reports; Firewall Manager administers).

> "*Deep packet inspection and domain-based filtering for all traffic leaving the VPC*" → **Network Firewall** (Suricata IPS; not Network Access Analyzer, not NAT Gateway).

> "*Identify unintended network access paths to resources against compliance requirements*" → **Network Access Analyzer** (reports only, no packet inspection).

Pocket card

Keyword	Answer
SQLi / XSS / HTTP filtering	WAF
Rate limit per IP	WAF rate-based rule
Test rule without blocking	WAF Count mode
WAF attach points	CloudFront, ALB, API GW, AppSync, Cognito (never NLB/EC2)
Free automatic DDoS (L3/L4)	Shield Standard
L7 DDoS / DRT / cost protection	Shield Advanced ($3k/mo)
Security policies across org, auto for new accounts	Firewall Manager (needs Organizations + Config)
Inspect ALL VPC traffic L3–L7	Network Firewall
Cryptomining / odd API calls / no agents	GuardDuty

Keyword	Answer
PII in S3	Macie
CVEs on EC2/ECR/Lambda	Inspector
One findings dashboard + compliance checks	Security Hub
Root cause after a finding	Detective
SOC/PCI/ISO reports for auditors	Artifact
SQLi via NACL/SG?	No — WAF
Large-scale DDoS resilience	Shield + WAF + CloudFront + ASG (not WAF alone)
Manages but doesn't block	Firewall Manager / Security Hub
Suricata IPS, deep packet inspection, FQDN filtering	Network Firewall
Report unintended network access, no DPI	Network Access Analyzer
Copy ENI traffic for analysis	Traffic Mirroring

You've now got the security crew sorted — next up, the service that decides who your app's *users* even are: Cognito.

↑ Back to index

Section 23: Cognito

The idea

Your mobile app has a million *customers* — not employees, customers. They need to sign up, log in, maybe with Google or Facebook, and then... upload photos to S3. Two very different problems just walked in: **"who are you?"** and **"what AWS stuff may you touch?"** Amazon Cognito answers both, with two deliberately different pieces.

Here's the analogy that carries the whole section: think of an office building. At the front desk there's a **login box** — you prove who you are and get a **visitor badge** (that's the **User Pool**, and the badge is a **JWT token**). But the badge alone doesn't open doors. You take it to the **exchange desk**, which swaps your badge for a **physical keycard** that actually opens specific rooms (that's the **Identity Pool**, and the keycard is a set of **temporary AWS credentials**).

Jargon check before we dive in:

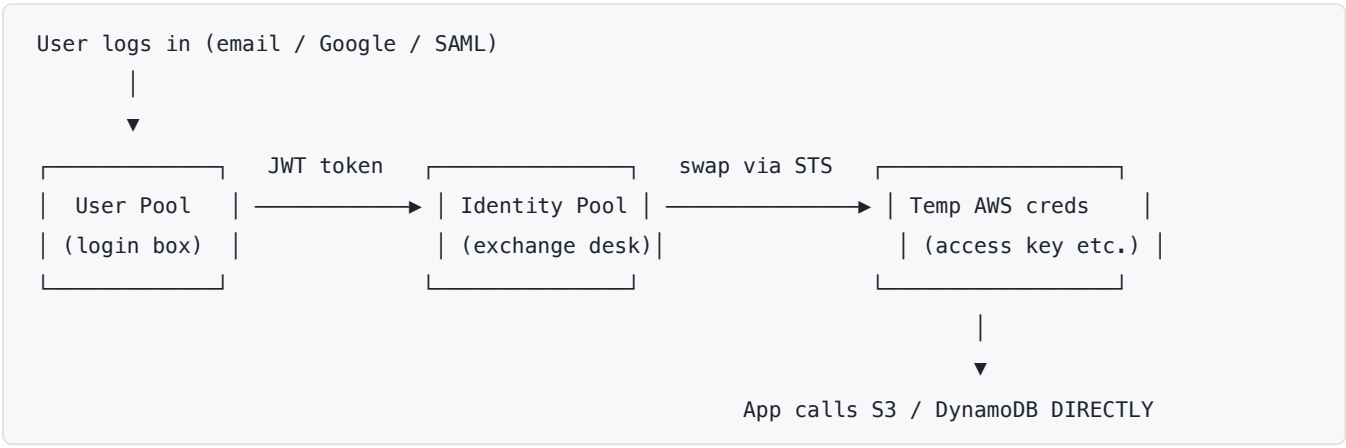
- **JWT** = JSON Web Token — a signed blob of text proving "this user authenticated, here's who they are."
- **STS** = AWS Security Token Service — the service that mints short-lived AWS credentials.
- **IdP** = Identity Provider — anything that vouches for identity (Google, Facebook, a corporate SAML server).

The core distinction (THE thing to know)

	User Pools	**Identity Pools**
Question answered	**AUTHENTICATION** — *who are you?*	**AUTHORIZATION for AWS** — *what AWS resources may you touch?*
What it is	A **user directory**: sign-up, sign-in, password resets	A **credential exchange desk**
What it returns	**JWT tokens**	**Temporary AWS credentials** (via **STS**)
Extras	**Hosted UI** (pre-built login pages), **MFA**, **social login** (Google/Facebook/Apple), **SAML** enterprise IdPs	**Guest / unauthenticated access**, fine-grained per-user IAM permissions
Plugs into	**API Gateway** and **ALB** as an **authorizer**	S3, DynamoDB, any AWS API — directly from the app

THE trap: the exam says "*users need to access S3 directly from the mobile app*" and offers "Cognito User Pools" as a tempting answer. A JWT can't call S3 — **only AWS credentials can**. Direct AWS resource access → **Identity Pools**.

The classic flow (draw this in your head)



Note the Identity Pool is generous about what it accepts: tokens from a **User Pool**, from **social providers** directly, or from **SAML** — it's an exchange desk for *any* recognized badge.

The clever bits worth exam points

- **Authorizers**: a **Cognito User Pool authorizer on API Gateway** validates the JWT before your backend ever runs — "authenticate API users without writing auth code" → User Pool authorizer. ALB can also authenticate through Cognito before forwarding requests.
- **Guest access**: Identity Pools support **unauthenticated identities** — "let users browse content *before* signing up" → Identity Pool guest access.
- **Guests need Identity Pools specifically**: plain STS has no notion of an anonymous guest — Identity Pools mint guest credentials. But Identity Pools **don't manage authentication** (no sign-up/sign-in/passwords) — that's always the User Pool or an external IdP.
- **Web identity federation** (Login with Amazon, Facebook, Google, any **OIDC** IdP) is today done through Cognito — no custom sign-in code, no identities to manage yourself. (Legacy **AWS Mobile Hub** used to wire this up for mobile apps; it's retired in favor of **Amplify**.)
- **No direct CloudFront integration**: User Pools plug into **API Gateway and ALB**, but **not natively into a CloudFront distribution** — protecting CloudFront content needs **signed URLs/cookies** or **Lambda@Edge** validating the JWT.
- **Per-user permissions with policy variables**: an IAM policy attached via the Identity Pool can use `${cognito-identity.amazonaws.com:sub}` so each user can only reach `s3://bucket/${their-own-id}/*`. **"Each user accesses only their own folder"** → Identity Pool + **IAM policy variables**. One policy, a million users, zero per-user setup.

Cognito vs IAM Identity Center — don't mix up the audiences

THE trap: "*employees need single sign-on to AWS accounts*" → that's your **workforce**, and the answer is **IAM Identity Center** (the successor to AWS SSO), *not* Cognito. The rule:

- **Customers of your app** (millions of external users) → **Cognito**
- **Employees / workforce** signing into AWS accounts and business apps → **IAM Identity Center**

If the humans in the question get a paycheck from the company, Cognito is the wrong answer.

Question patterns

> "*Mobile app needs user sign-up, sign-in, and login with Google and Facebook*" → **Cognito User Pools** (directory + social login + hosted UI).

> "*Authenticated app users must upload files directly to S3 from the device*" → **Cognito Identity Pools** (JWT → STS → temporary AWS credentials; only creds can call S3).

> "*Allow users to browse limited content without creating an account*" → **Identity Pool unauthenticated (guest) access**.

> "*Authenticate users of a REST API on API Gateway without custom code*" → **Cognito User Pool authorizer** (API Gateway validates the JWT for you).

> "*Each user may only read and write objects in their own S3 prefix*" → **Identity Pool + IAM policy variables** (`${cognito-identity...:sub}` scopes one policy per user).

> "*Company employees need SSO access to multiple AWS accounts*" → **IAM Identity Center, NOT Cognito** (workforce = Identity Center; customers = Cognito).

> "*Enterprise customers must log into your SaaS app with their corporate SAML identity provider*" → **User Pools with SAML federation** (still authentication — still the login box).

> "*Add MFA to your application's customer logins*" → **User Pools** (MFA lives where authentication lives).

> "*Use Cognito Identity Pools to handle user sign-up and password management*" → **Wrong — Identity Pools don't authenticate; use User Pools** (Identity Pools only hand out AWS creds).

> "*Authenticate users with Cognito directly on a CloudFront distribution*" → **Not natively supported — use ALB/API Gateway with a User Pool, or Lambda@Edge / signed cookies** in front of CloudFront.

> "*Mobile app users log in with Amazon/Facebook/Google (OIDC) without custom sign-in code*" → **Web identity federation via Cognito**.

Pocket card

Keyword	Answer
Sign-up / sign-in / user directory	User Pools
JWT tokens	User Pools
Social login (Google/Facebook/Apple), SAML	User Pools (federation)
Hosted UI, MFA	User Pools
API Gateway / ALB authorizer	User Pool authorizer
Temporary AWS credentials for app users	Identity Pools (via STS)
Direct app access to S3/DynamoDB	Identity Pools
Guest / unauthenticated access	Identity Pools
Each user only their own folder	Identity Pool + IAM policy variables
Employees SSO to AWS accounts	IAM Identity Center (never Cognito)
Identity Pools for sign-in/passwords?	No — User Pools authenticate
Cognito + CloudFront natively?	No — ALB/API GW, or Lambda@Edge
Social/OIDC login, no custom code	Web identity federation (Cognito)
AWS Mobile Hub	Legacy — now Amplify
Memory hook	User Pool = login box (JWT); Identity Pool = exchange desk (creds)

Cognito handles your app's *customers* — but when the identities live in a corporate Active Directory, you're in the next section's territory: Directory Services.

↑ Back to index

Section 24: Directory Services

The idea

First, what *is* Active Directory? If you've never worked in a Windows shop: **Active Directory (AD)** is Microsoft's on-premises system that acts as the **corporate phonebook plus the master keyring**. It stores every **user, computer, and group** in the company, and when Alice logs into her laptop, AD is what checks her password and decides which shared drives and printers she can use. Users log into a **domain** (like `corp.example.com`), and nearly every large enterprise on Earth runs it. Related jargon: **LDAP** is the standard protocol for talking to directories like AD; **domain join** means enrolling a machine into the AD domain so it obeys its rules.

Now the cloud problem: your company moves workloads to AWS, but the phonebook still lives in the on-prem server room. Windows services in AWS — **FSx for Windows File Server, Amazon WorkSpaces (virtual desktops), RDS for SQL Server** — all *need* a directory to authenticate against. AWS gives you **three options**, and the exam tests whether you can pick the right one from a scenario.

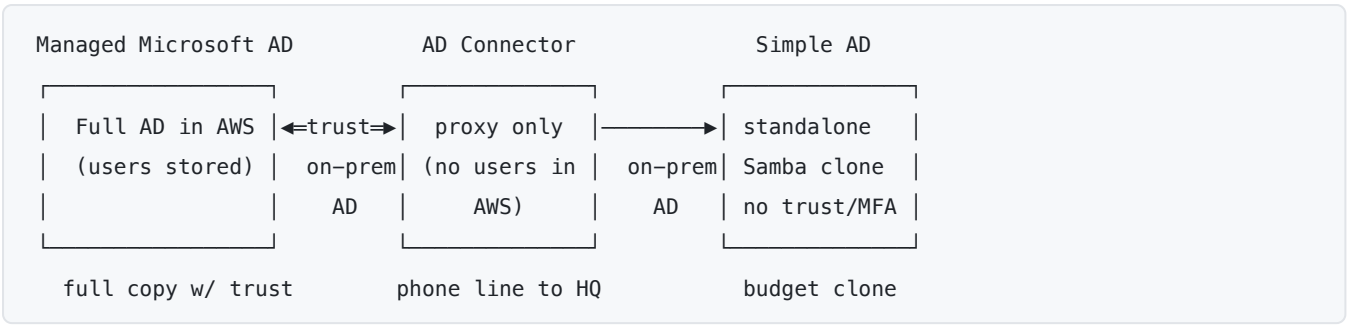
The analogy: your on-prem AD is the head office's filing room.

- **AWS Managed Microsoft AD** = build a **full branch-office filing room** in AWS, and optionally connect it to head office with a trusted courier route.
- **AD Connector** = install only a **phone line**: every lookup rings head office; nothing is stored in the branch.
- **Simple AD** = a **budget photocopy** of a filing room — works for small stuff, but it's not the real Microsoft thing.

The three options (the table to memorize)

	AWS Managed Microsoft AD	AD Connector	Simple AD
What it really is	**REAL Microsoft AD** running in AWS	A **PROXY** — just forwards auth requests	**Samba-based** AD-compatible clone
Users stored in AWS?	Yes	**No — nothing stored in AWS**	Yes
Works with on-prem AD?	Yes — **TRUST relationship** (two-way trust; use users from **both** sides)	Yes — it *only* redirects to on-prem	**NO trust** — standalone only
MFA support	**Yes**	Yes (via on-prem RADIUS)	**No MFA**
Best for	**Full AD features in the cloud**; EC2 domain join; RDS/FSx integration; hybrid via trust	**"Keep all identities on-prem"**, let AWS services authenticate against them	**Cheap basic LDAP**, small orgs, **< 5,000 users**

Jargon check: a **trust relationship** means two separate AD forests agree to honor each other's logins — cloud AD trusts on-prem AD and vice versa (**two-way trust**), so users from *either* directory can access resources on *either* side, without syncing passwords into the cloud.



How the exam distinguishes them

- **Managed Microsoft AD** is the answer when the scenario needs *actual AD machinery in AWS*: **EC2 Windows domain join**, **MFA**, seamless integration with **RDS for SQL Server / FSx for Windows / WorkSpaces**, or a **trust with on-prem** so both user sets work. Phrase to spot: "*full AD features in the cloud*" or "*trust with on-prem*".
- **AD Connector** is the answer when compliance or policy says **identities must never be stored in the cloud**. It's a pipe, not a directory — every authentication is redirected to the on-prem domain controllers. (Corollary: if the VPN/Direct Connect link to on-prem dies, authentication dies with it.)
- **Simple AD** is the answer when the words are **"lowest cost"**, **"basic LDAP"**, **"small organization"** — and *nothing* in the scenario mentions trust, MFA, or on-prem integration. It offers a **subset** of Managed AD: user accounts and group memberships, **group policies**, secure connections to (domain-joined) EC2, and **Kerberos-based SSO**.
- **AD Connector** is AWS's **"directory gateway"** — and for a company already running a corporate AD, it's usually the **easiest integration**. Bonus: once connected, you can **assign IAM roles to AD users/groups** so they reach the AWS console with their AD logins (no IAM users).
- Where it sits in the identity map: **Directory Service** = directory integration for **Windows/AD workloads and domain-joined EC2**; **IAM Identity Center** = workforce SSO to AWS accounts; **Cognito** = app customers. (Directory Service historically also offered **Amazon Cloud Directory**, a non-AD graph directory — legacy, rarely tested.)
- **Amazon WorkSpaces** = managed cloud **virtual desktops (VDI)**, Windows or Linux, scaling to thousands of workers — "full desktop" (vs **AppStream 2.0** = stream a single app). It authenticates against one of the three directories.

THE trap: "*connect Simple AD to on-prem AD with a trust*" → **can't**. Simple AD supports **no trust and no MFA** — the moment either word appears, Simple AD is eliminated.

THE trap: "*AD Connector as a standalone directory*" → also can't. A phone line with no head office on the other end connects to nothing — **AD Connector requires an existing on-prem AD**.

The common pairing pattern

Remember this trio, because it fuels most directory questions: **FSx for Windows File Server, Amazon WorkSpaces, and RDS for SQL Server (Windows Authentication) all require a directory.** The question is rarely "do I need a directory?" — it's *which* of the three options fits the constraints given. FSx + "use our existing on-prem identities" → Managed AD with a trust, or AD Connector. FSx + "need MFA and cloud-resident AD" → Managed Microsoft AD.

Memory hook: Managed AD = full copy with trust; Connector = phone line to on-prem; Simple = budget clone.

Question patterns

> "*Deploy FSx for Windows File Server; users must authenticate with existing on-prem AD identities*" → **Managed Microsoft AD with a trust, or AD Connector** (both let on-prem identities work; pick whichever the options offer).

> "*Security policy requires that no user identities are stored in the cloud*" → **AD Connector** (proxy only — all auth redirected on-prem, nothing stored in AWS).

> "*Small company needs a low-cost, basic LDAP-compatible directory for one application*" → **Simple AD** (cheap Samba standalone, fine under ~5,000 users).

> "*EC2 Windows instances must join a domain, and MFA is required*" → **AWS Managed Microsoft AD** (real AD: domain join + MFA; Simple AD has neither MFA nor trust).

> "*Establish a two-way trust between AWS and the on-premises domain so users on both sides access resources*" → **Managed Microsoft AD** (the only option supporting trust relationships).

> "*RDS for SQL Server with Windows Authentication for corporate users*" → **Managed Microsoft AD** (RDS integrates with it directly; add a trust for on-prem users).

> "*WorkSpaces desktops authenticating against on-prem AD without replicating the directory*" → **AD Connector** (phone line: auth flows back on-prem).

> "*Employees should sign into the AWS console with their existing corporate AD credentials, with the least effort*" → **AD Connector + IAM roles mapped to AD users/groups** (or Identity Center with AD as the source).

> "*Provide thousands of workers with secure Windows/Linux cloud desktops*" → **Amazon WorkSpaces** (VDI; needs a directory).

Pocket card

Keyword	Answer
Full AD features in AWS	Managed Microsoft AD
Trust relationship with on-prem	Managed Microsoft AD (two-way trust)

Keyword	Answer
MFA + domain join	Managed Microsoft AD
No identities stored in cloud	AD Connector
Proxy / redirect auth to on-prem	AD Connector
Low cost, basic LDAP, small org	Simple AD
No trust, no MFA	Simple AD's limits
FSx Windows / WorkSpaces / RDS SQL Server	All need a directory — pick from the three
Existing corporate AD, easiest integration	AD Connector ("directory gateway")
AD users into AWS console	AD Connector + IAM roles for AD users/groups
Simple AD features	Users/groups, group policies, EC2 domain join, Kerberos SSO
Cloud virtual desktops (VDI)	Amazon WorkSpaces
Memory hook	Full copy w/ trust / phone line / budget clone

Directories solve *who* on-prem users are in the cloud — next we solve where their *files* live, with the hybrid storage bridge crew: Storage Gateway, DataSync, and Transfer Family.

↑ Back to index

Section 25: Storage Gateway, DataSync & Transfer Family

The idea

Here's the hybrid-storage problem: your on-prem servers speak the old languages — **NFS and SMB file shares** (network drives), **iSCSI block volumes** (raw disks over the network), **tape backups**. Meanwhile the cheap, infinite storage lives in AWS. The servers can't be rewritten overnight, so you need cloud storage that **looks local** to them.

The analogy: your office has a cramped filing room, and you've rented an infinite warehouse across town (S3). Three different companies help you use it:

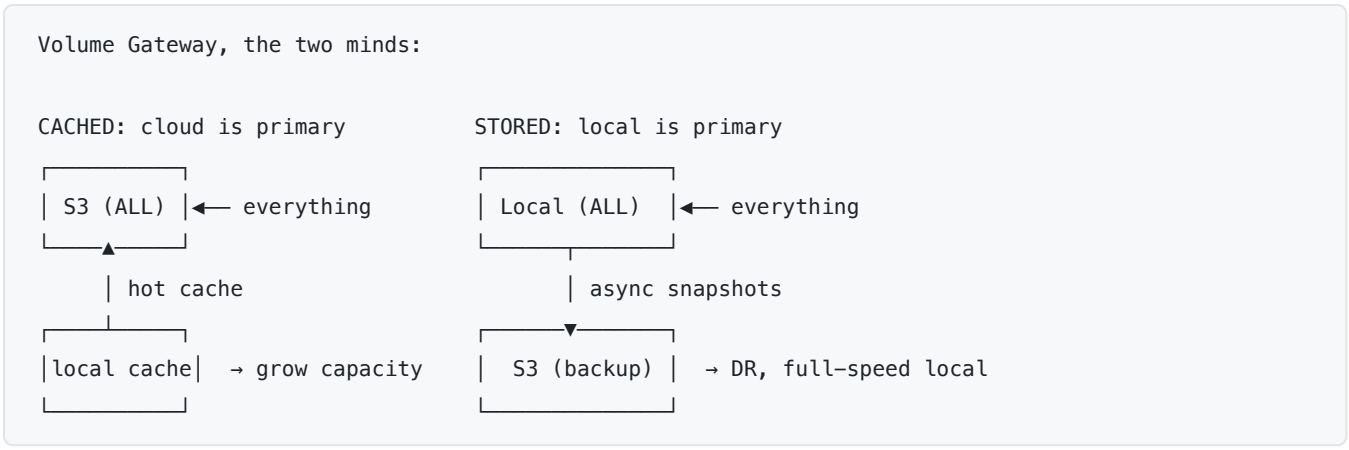
- **Storage Gateway** = a **magic filing cabinet** installed in your office. It looks and behaves like a normal cabinet, but its drawers secretly extend into the warehouse — frequently used folders stay in the drawer (**local cache**), everything else lives across town. This is an **ONGOING bridge** — it stays forever.
- **DataSync** = a **professional moving company**. They show up, load the truck, move your files to the warehouse fast and carefully (labels and permissions intact), and leave. **Migration**, one-time or on a schedule.
- **Transfer Family** = a **staffed loading dock** on the warehouse itself, so outside partners can keep delivering packages the old way (**SFTP**) while everything lands directly in S3.

Jargon check: **NFS** (Network File System, the Linux share protocol), **SMB** (Server Message Block, the Windows share protocol), **iSCSI** (block storage — raw disk — over the network), **SFTP/FTPS/FTP** (classic file-transfer protocols).

Storage Gateway — the ongoing hybrid bridge

Deployed as a **VM (or hardware appliance) on-premises**, backed by cloud storage, with a **local cache** for low-latency access to hot data. Four flavors — the exam tests which flavor fits:

Type	Protocol	Backed by	The exam phrase
S3 File Gateway	NFS / SMB	**S3** (real objects!)	"*Replace the NAS; files in S3; keep local file access*"
FSx File Gateway	SMB	**FSx for Windows**	Low-latency on-prem access to FSx Windows shares
Volume Gateway — Cached	iSCSI	**Primary data in S3**, hot cache local	"*Expand on-prem storage capacity*" — the disk is bigger than your building
Volume Gateway — Stored	iSCSI	**Primary data LOCAL**, async **snapshots to S3**	"*Low-latency access to the ENTIRE dataset*" + cloud backup / DR
Tape Gateway	iSCSI VTL	S3 → **Glacier**	"*Replace physical tape backup infrastructure*" — near-verbatim question



Memory hook: Cached = **capacity** (cloud holds it all); Stored = **speed + safety** (local holds it all, cloud holds the backup).

THE trap: "*low-latency access to the FULL dataset*" → **Stored**, not Cached — a cache only keeps the hot slice; if latency must be low for *everything*, everything must live locally.

Also worth a point: **Tape Gateway** presents a **Virtual Tape Library (VTL)** so existing backup software (Veeam, NetBackup) keeps working unchanged — the tapes are just... S3 and Glacier now. Any scenario with the word **"tape"** ends at Tape Gateway. Archived virtual tapes can go to **S3 Glacier Flexible Retrieval or S3 Glacier Deep Archive**. "*Lowest-cost long-term tape archive*" → **Deep Archive**.

More gateway facts:

- Storage Gateway exists to **connect existing on-prem storage and apps to cloud storage**. It's **not an application migration tool** (for servers, use Application Migration Service).
- It's **bandwidth-efficient**: it transfers **only changed data** and **compresses** it on the way.
- **Volume Gateway** backs volumes up as **point-in-time EBS snapshots**, so you can restore them as EBS volumes in AWS. That's handy for **backup, DR, and migration** of block data.

DataSync — the moving company

AWS DataSync is the **migration / scheduled-transfer engine**: **one-time or scheduled** sync from on-prem **NFS/SMB** into **S3, EFS, or FSx**. You install an **agent** on-prem, and DataSync moves data fast (it's built for TB-scale over the network) while **preserving file metadata and permissions**, with **bandwidth throttling** so you don't flatten the office internet during business hours. Bonus fact the exam likes: DataSync also moves data **between AWS storage services** — S3 ↔ EFS ↔ FSx — no agent needed for that. DataSync can also **write directly into any S3 storage class, including S3 Glacier Deep Archive**. "*Move on-prem archive files straight to Deep Archive over the network*" → **DataSync**, with no lifecycle hop needed. (DataSync moves **files**. To migrate whole **servers**, use the **replication agent of Application Migration Service (MGN)**.)

THE distinction of this whole section:

- **DataSync = MOVE the data** (one-time migration or scheduled copy — the truck leaves).
- **Storage Gateway = ongoing hybrid ACCESS** (the bridge stays; on-prem apps keep reading/writing after — or instead of — migrating).

THE trap: "*migrate 50 TB from on-prem NAS to S3, preserving permissions*" offers "S3 File Gateway" as a shiny wrong answer. A gateway is for *continuing access*, not bulk moves — a **migration** with **metadata**

preserved is **DataSync**, every time. Flip side: "*keep using on-prem apps against files that must live in S3*" → gateway, not DataSync.

Transfer Family — the managed SFTP dock

AWS Transfer Family = a fully managed **SFTP / FTPS / FTP endpoint** in front of **S3 or EFS**. Your partners keep their crusty-but-beloved SFTP scripts; the files land straight in your bucket. "*Partners upload via SFTP, must keep their existing workflow*" → **Transfer Family** — and **never** the distractor "run an SFTP server on EC2" (that's undifferentiated heavy lifting the exam always wants you to reject).

One pointer

Everything above assumes a network path. If the scenario is **offline, or the data is so huge the network math doesn't work** (weeks of transfer time, remote sites, limited bandwidth), that's the **Snow Family** — Section 27's story.

A newer option sits in between: **AWS Data Transfer Terminal**. These are **physical AWS locations** (in select cities) where you **bring your own storage devices** and plug them into a high-throughput connection to upload data to AWS, for example S3. You carry the data in instead of having AWS ship a device to you.

Option	How the data travels	Signal
S3 Transfer Acceleration	Over the internet, via the nearest **edge location**	Long-distance uploads to one bucket
DataSync	Over your network (agent)	Online migration/sync, metadata kept
Data Transfer Terminal	**You physically bring your drives** to an AWS facility	Large dataset, drives in hand, need it uploaded fast without ordering a device
Snow Family	**AWS ships you a device**, which you ship back	Offline / remote / network would take > 1 week

Question patterns

> "*Company wants to eliminate its physical tape backup infrastructure but keep existing backup software*" → **Tape Gateway** (virtual tape library → S3/Glacier; software sees tapes).

> "*Replace an aging NAS; store files in S3 but keep low-latency local access over SMB/NFS*" → **S3 File Gateway** (cloud-backed share with local cache).

> "*Migrate 50 TB from an on-prem NAS to S3, preserving file permissions and metadata, minimal effort*" → **DataSync** (migration engine; preserves metadata; gateway is for ongoing access, not moves).

> "*Business partners upload files via SFTP and cannot change their workflow; files must land in S3*" → **Transfer Family** (managed SFTP endpoint — never build SFTP on EC2).

> "*On-prem app needs low-latency access to the ENTIRE dataset, with backups to AWS for DR*" → **Volume Gateway — Stored** (full data local, async snapshots to S3).

> "*On-prem storage is running out of space; extend capacity while keeping frequently used data fast*" → **Volume Gateway — Cached** (primary in S3, hot cache local).

> "*Copy files nightly from on-prem SMB shares to EFS, with bandwidth throttling*" → **DataSync** (scheduled transfers + throttling are its signature features).

> "*Low-latency on-prem access to file shares stored on FSx for Windows File Server*" → **FSx File Gateway**.

> "*Transfer data between S3 and EFS within AWS*" → **DataSync** (works AWS-to-AWS too, agentless).

> "*Archive virtual tapes at the lowest possible cost*" → **Tape Gateway** → **S3 Glacier Deep Archive**.

> "*Copy on-prem archive files directly into S3 Glacier Deep Archive over the network*" → **DataSync** (writes straight to the Deep Archive storage class).

> "*Team has hard drives full of data and wants to upload them at high speed at an AWS facility*" → **AWS Data Transfer Terminal** (bring your own device; Snow = AWS ships a device to you).

Pocket card

Keyword	Answer
Replace tape backups	Tape Gateway (VTL → S3/Glacier)
NAS replacement, files in S3, local access	S3 File Gateway
SMB cache for FSx Windows	FSx File Gateway
Extend on-prem capacity, primary in cloud	Volume Gateway — Cached
Full dataset local + snapshots to cloud	Volume Gateway — Stored
Migrate / scheduled sync, preserve permissions	DataSync
Bandwidth throttling, on-prem agent	DataSync
S3 ↔ EFS ↔ FSx transfers	DataSync
Partners upload via SFTP/FTPS/FTP	Transfer Family (never SFTP-on-EC2)
Ongoing hybrid access vs. one-time move	Storage Gateway vs. DataSync

Keyword	Answer
Offline / massive data, no bandwidth	Snow Family (Section 27)
Cheapest tape archive tier	Tape Gateway → Glacier Deep Archive
Gateway sends only changed, compressed data	Storage Gateway (bandwidth-efficient)
Volume Gateway backups restorable in AWS	EBS snapshots
Online copy straight to Deep Archive	DataSync
Migrate servers (not files)	Application Migration Service, not DataSync/Gateway
Bring your own drives to an AWS facility	Data Transfer Terminal

That's the online bridge to the cloud sorted — when the pipe is too small for the payload, you put the data on a truck, and that's where the Snow Family rolls in.

↑ Back to index

Section 26: FSx Family

The idea

You already know EFS: the generic, elastic, shared filesystem for Linux. But some workloads don't want "generic" — they were born and raised on a *specific* filesystem technology, and they refuse to run on anything else. A .NET app expects Windows file shares. A supercomputing cluster expects Lustre. A company that spent ten years on NetApp hardware expects NetApp.

FSx is AWS's answer: **fully managed versions of famous third-party filesystems**. Think of it like a food court. EFS is the everyday cafeteria — fine for everyone eating plain Linux food. FSx is the row of specialty restaurants next to it: one only serves Windows, one only serves high-performance computing, one only serves NetApp loyalists, one only serves ZFS fans. You don't pick a restaurant by taste — you pick it because **the exam question names the technology**, and you match the name.

Key vocab spelled out: SMB = Server Message Block (the Windows file-sharing protocol). NFS = Network File System (the Linux one). HPC = High-Performance Computing. NTFS = the Windows filesystem format.

The four specialists

FSx flavor	Trigger words	What it is
FSx for Windows File Server	Windows, SMB, NTFS, Active Directory, .NET, "user home directories"	Managed Windows file share
FSx for Lustre	HPC, ML training, video rendering, financial simulation, "process S3 data fast"	Parallel filesystem, hundreds of GB/s throughput
FSx for NetApp ONTAP	NetApp, SnapMirror, multi-protocol	Managed NetApp; speaks NFS + SMB + iSCSI at once
FSx for OpenZFS	ZFS, "Linux NFS file server migration"	Managed ZFS with snapshots/clones

FSx for Windows File Server

- Native **SMB protocol and NTFS**, integrates with **Active Directory** (both AWS Managed AD and your on-prem AD).
- Supports **Multi-AZ** deployment for high availability.
- THE trap: **EFS is Linux-only — it cannot serve Windows/SMB clients.** The moment you see "Windows," EFS is eliminated.
- What it **can't** do: it can't present S3 objects as files or write results back to S3, and it isn't a parallel HPC file system. Those are **Lustre** jobs.
- Its Active Directory configuration is set **at creation and can't be changed afterwards**. Switching AD means creating a new file system and migrating the data.

FSx for Lustre

- "Lustre" = Linux + cluster. Built for **massively parallel access**: machine learning training, video rendering farms, genomics, financial modeling. Scales to **hundreds of GB/s** and millions of IOPS.
- **Unique superpower: native S3 integration.** Point Lustre at an S3 bucket and it presents the objects as files; results can be written back to S3. "Process data *in S3* with a high-performance filesystem" → Lustre, every time. The classic pattern is **hot data processed in parallel on Lustre, cold data left in S3**. Rolling your own (EMR, or a self-managed parallel file system on EC2) doesn't match it for speed or effort.
- Two deployment types:

Type	Use	Durability
Scratch	Short-term, temporary processing	No replication — cheap, data lost if hardware fails
Persistent	Long-term storage	Replicated within AZ

FSx for NetApp ONTAP

- The migration landing pad for on-prem **NetApp** appliances. Supports **SnapMirror** replication, so you can mirror on-prem NetApp straight into AWS.
- **Multi-protocol**: NFS, SMB, *and* iSCSI from one filesystem — the only FSx that serves Linux and Windows clients simultaneously.
- Supports **Multi-AZ** deployment, with consistent **sub-millisecond latency** on its SSD tier.

FSx for OpenZFS

- For migrating **ZFS-based** or generic NFS Linux file servers, with ZFS goodies: instant **snapshots and clones**, low-latency performance.

The decision algorithm: scan the question for a named technology. Windows/SMB/AD → FSx for Windows. HPC or S3-as-a-filesystem → Lustre. NetApp → ONTAP. ZFS → OpenZFS. No named tech, just "shared storage for Linux EC2 instances" → plain EFS.

Question patterns

> "*Windows applications need shared storage with Active Directory authentication*" → **FSx for Windows File Server** (SMB + AD = Windows FSx; EFS can't do Windows)

> "*ML training job needs high-throughput access to a dataset stored in S3*" → **FSx for Lustre** (only Lustre mounts S3 as a filesystem at HPC speed)

> "*Migrate on-premises NetApp storage to AWS with minimal changes*" → **FSx for NetApp ONTAP** (the word "NetApp" is the whole answer)

> "*Short-term, cost-optimized high-performance storage for a batch processing job*" → **FSx for Lustre Scratch** (temporary + cheap + no replication = Scratch)

> "*File share accessed by both Linux (NFS) and Windows (SMB) clients*" → **FSx for NetApp ONTAP** (the only multi-protocol option)

> "*.NET application storing user home directories, must survive AZ failure*" → **FSx for Windows, Multi-AZ** (Windows workload + HA = Multi-AZ Windows FSx)

> "*Replace an on-prem ZFS file server serving NFS to Linux clients*" → **FSx for OpenZFS** (ZFS named → OpenZFS)

> "*HPC job must process hot data in parallel while cold data stays cheaply in S3*" → **FSx for Lustre linked to S3** (not FSx for Windows, not EMR)

> "*Point an existing FSx for Windows file system at a different Active Directory*" → **Create a new file system** (the AD configuration can't be changed after creation)

Pocket card

Keyword	Answer
Windows / SMB / NTFS / AD / .NET	FSx for Windows File Server
HPC / ML training / rendering / GB/s	FSx for Lustre
Process S3 data as files	FSx for Lustre (S3 integration)
Temporary + cheapest Lustre	Scratch deployment
Long-term Lustre	Persistent deployment
NetApp / SnapMirror	FSx for NetApp ONTAP
NFS + SMB + iSCSI together	FSx for NetApp ONTAP
ZFS	FSx for OpenZFS
Plain Linux shared storage	EFS (not FSx at all)
Hot data parallel, cold data in S3	FSx for Lustre + S3
FSx Windows reading S3 as files?	No (Lustre only)
Change AD on FSx for Windows	Not possible; recreate
Multi-AZ, sub-ms latency, NetApp features	FSx for NetApp ONTAP

FSx moves files at specialty speed — but when the data is too big to move over a wire at all, you put it on a truck, which is exactly where the Snow Family comes in.

↑ Back to index

Section 27: Snow Family

The idea

Here's a physics problem AWS can't engineer away: **bandwidth**. Try to upload 100 TB over a 100 Mbps connection and the math says roughly **100+ days** — and that's if nobody else touches the network. Amazon's solution is delightfully low-tech: *never underestimate the bandwidth of a truck full of hard drives*. AWS ships you a ruggedized, encrypted storage box; you copy your data onto it locally at LAN speed; you ship it back; AWS loads it into S3.

Rule of thumb the exam loves: if transferring over the network would take **more than about a week**, use a Snow device instead.

The second superpower: Snow devices carry **compute**. They can run EC2 instances and Lambda functions on-board, which means you can process data at places with no internet at all — ships at sea, mines, military bases, remote research stations. That's "edge computing."

The device lineup

Device	Capacity	When
Snowcone	8–14 TB	Tiny, rugged, ships anywhere (even drone-deliverable); small edge sites
Snowball Edge Storage Optimized	~80 TB usable	**The workhorse** — migrations of ~50–500 TB (order several)
Snowball Edge Compute Optimized	Less storage, more CPU + **optional GPU**	Edge processing: run EC2/Lambda where there's no connectivity
Snowmobile	Up to **100 PB**	An actual **45-foot shipping-container** truck for **exabyte-scale** moves (several trucks); consider at **>10 PB**

Snowball Edge Storage Optimized specs (worth recognizing): **80 TB usable HDD**, **40 vCPUs**, **1 TB SATA SSD**, and up to **40 Gb** networking. Snowball Edge was offered in three configurations: **Storage Optimized**, **Compute Optimized**, and **Compute Optimized with GPU**.

Extra flavor points:

- **Snowcone** comes with the **DataSync agent preinstalled** — copy data on it, and it can sync back online later, or you ship it.
- Multiple Snowballs can be ordered in parallel for big jobs (e.g., 200 TB ≈ 3 × 80 TB Snowball Edge Storage Optimized).
- **AWS OpsHub** = the **GUI application** for managing Snow devices (no CLI wrestling required).

The process

1. **Order** the device in the AWS console.

2. AWS **ships** it to you.
3. You connect it locally and **copy data — encrypted automatically with KMS** (Key Management Service). Keys never live on the device in usable form.
4. **Ship it back** (E Ink label updates itself).
5. AWS **loads the data into S3**, then securely wipes the device.

THE trap: **Snowball cannot import directly into S3 Glacier.** Data always lands in **S3 first**; if you want Glacier, you attach an **S3 lifecycle rule** that transitions the objects afterward. Any answer choice saying "import from Snowball straight to Glacier" is wrong — reliably tested. (By contrast, **DataSync** can write online **directly to S3 Glacier Deep Archive**; see Section 25. Shipped data = S3 first; network copy = straight into the archive class.)

Edge computing angle: Snowball Edge Compute Optimized (with optional GPU) runs EC2 AMIs and Lambda locally. Scenario smells: "vessel collecting sensor data with intermittent connectivity," "remote facility must process video before transfer." Process on the device, ship or sync results later.

Question patterns

> "*Migrate 200 TB, the site has a 100 Mbps link, deadline in 3 weeks*" → **Snowball Edge Storage Optimized** (network math = months, so ship devices — a few 80 TB units)

> "*Decommission an entire datacenter: multiple petabytes (>10 PB) to AWS*" → **Snowmobile** (past ~10 PB, send the truck)

> "*Research ship must run analysis on collected data with no internet connectivity*" → **Snowball Edge Compute Optimized** (compute at the disconnected edge; GPU if ML is mentioned)

> "*Company wants to archive 80 TB directly into S3 Glacier using Snowball*" → **Import to S3, then lifecycle rule to Glacier** (Snowball can't write to Glacier directly — THE trap)

> "*Small remote clinic needs to transfer ~8–10 TB from a space- and power-constrained site*" → **Snowcone** (tiny footprint, tiny capacity)

> "*Transfer 40 TB once; would take 6 weeks over the existing connection*" → **Snowball Edge** (>1 week over the wire → Snow family)

> "*Manage Snow devices with a graphical interface*" → **AWS OpsHub** (the Snow GUI)

Pocket card

Keyword	Answer
Transfer would take > 1 week	Snow family
8–14 TB, tiny/rugged/edge	Snowcone
50–500 TB migration	Snowball Edge Storage Optimized
Process data offline / GPU at edge	Snowball Edge Compute Optimized
> 10 PB, up to 100 PB	Snowmobile
Straight to Glacier?	No — S3 first + lifecycle rule
Encryption on device	KMS, automatic
GUI for Snow devices	OpsHub
Preinstalled DataSync agent	Snowcone
Snowball Edge Storage Optimized specs	80 TB HDD, 40 vCPU, 1 TB SSD, 40 Gb
Exabyte-scale, 45-ft container	Snowmobile
Straight to Deep Archive, online	DataSync (not Snow)

Once your data (and everything else) is in AWS, you'll want to build environments the same way twice without clicking — that's CloudFormation's whole reason to exist.

↑ Back to index

Section 28: CloudFormation

The idea

Imagine you bake a spectacular cake once by improvising — a pinch of this, a swirl of that. Now bake an identical one next week. You can't; you never wrote it down. Clicking around the AWS console is improvised baking. **CloudFormation is the written recipe**: you describe your entire infrastructure — VPCs, EC2, RDS, IAM roles, everything — in a **YAML or JSON template**, and AWS "cooks" it for you. A running instance of a recipe is called a **stack**.

This is **Infrastructure as Code (IaC)** — AWS's native answer to Terraform, and it can model third-party resources too: templates live in version control, deploy identically to dev/test/prod or to a new region, and delete cleanly in one command. If creation fails partway, CloudFormation **rolls back automatically by default** — no half-baked cakes left in the oven.

Template anatomy (know the section names)

Section	Job
Parameters	Inputs at deploy time (instance size, env name)
Mappings	Lookup tables, e.g. the right AMI ID per region
Conditions	"Only create this in prod" logic
Resources	**The only REQUIRED section** — what actually gets built
Outputs	Values to surface after creation; **Export** them and another stack reads them with **Fn::ImportValue** (cross-stack references)

Intrinsic function one-liners: **!Ref** = reference a parameter/resource, **!GetAtt** = grab an attribute of a resource (like an instance's DNS name), **!Sub** = substitute variables into a string.

Features → scenarios (this is the exam meat)

- **Change Sets** — a **preview** of exactly what an update would create/modify/delete *before* you apply it. "Team wants to see what a stack update will do first" → Change Set.
- **Stack Policies** — protect specific resources within a stack **from being modified by stack updates**.
- **Drift Detection** — answers "did someone hand-edit our resources in the console?" Compares live reality to the template. "Detect manual changes" → Drift Detection.
- **StackSets** — deploy **one template to MANY accounts and regions** in one operation. "Roll out a security baseline across the whole organization" → StackSets.
- **Nested Stacks** — stacks inside stacks; package common patterns (a standard VPC, say) as reusable components.
- **DeletionPolicy** — per-resource instruction for stack deletion: **Retain** (keep the resource) or **Snapshot** (snapshot it first — works for RDS, EBS, etc.). "Keep the database when the stack is deleted" → DeletionPolicy: Retain.

Resource attributes (the "policy" family):

Attribute	When it acts	What it does	Cue
CreationPolicy (+ **cfn-signal**)	**CREATE**	Stack **waits** for N success signals (with a timeout) before marking the resource complete; the instance runs the `cfn-signal` **helper script** once software is installed/configured	"Don't continue until the app on EC2 is actually running" — memory hook: **CREATE + WAIT**
DependsOn	Create/delete order	Forces resource B to be created **after** resource A	"Resource must exist before another" — ordering only
UpdatePolicy	**UPDATE** of an ASG (etc.)	How updates roll out: **rolling update** in batches or replacing the ASG	"Update ASG instances in batches without downtime"
UpdateReplacePolicy	**UPDATE that replaces** a resource	**Retain/Snapshot** the *old* physical resource instead of deleting it	"Keep the old DB when an update forces replacement"
DeletionPolicy	**Stack DELETE** (or resource removal)	Retain / Snapshot / Delete	"Keep data when stack is deleted"

THE trap: **DependsOn only orders creation — it doesn't know whether the software inside the instance works.** "Wait until the application is installed and running before stack creation proceeds" → **CreationPolicy + cfn-signal**, not DependsOn.

THE trap: manual console edits to stack-managed resources don't update the template — they create *drift*, and the next stack update can stomp them. The cure is Drift Detection (to find it) and discipline (to stop it).

CDK in one line: the Cloud Development Kit lets you write infrastructure in **Python/TypeScript/Java**, which *compiles down to CloudFormation templates*. "Developers want to define infrastructure in a real programming language with loops and logic" → **CDK**.

Elastic Beanstalk contrast: Beanstalk is web-app-shaped automation — hand it code, it builds the standard web stack (and uses CloudFormation underneath). CloudFormation itself can build **any** infrastructure in any shape. Beanstalk = specific meal deal; CloudFormation = whole grocery store.

Question patterns

> "*Team must review what changes a template update will make before applying it*" → **Change Sets** (preview = change set, always)

> "*Deploy standardized security/compliance resources to all accounts in an organization, across regions*" → **CloudFormation StackSets** (one template → many accounts/regions)

> "*Ensure the RDS database is not deleted when the stack is torn down*" → **DeletionPolicy: Retain** (or

Snapshot if they want a final copy)

"*Determine whether anyone modified stack resources manually via the console*" → **Drift Detection** (template vs. reality diff)

"*Developers prefer defining infrastructure in TypeScript/Python rather than YAML*" → **AWS CDK** (code that compiles to CloudFormation)

"*Deploy identical environments repeatedly across regions with no manual steps*" → **CloudFormation templates** (the core IaC pitch)

"*Prevent accidental updates to a critical resource during stack updates*" → **Stack Policy** (update-protection inside a stack)

"*Reuse a common VPC pattern across many templates*" → **Nested Stacks** (componentize the recipe)

"*Stack must not proceed until software on the EC2 instance is installed and running*" → **CreationPolicy + cfn-signal** (DependsOn only orders resources)

"*Make sure the database is created before the app server resource*" → **DependsOn**

"*Roll out a new launch template to an ASG in batches during a stack update*" → **UpdatePolicy (AutoScalingRollingUpdate)**

"*An update replaces the RDS instance; keep the old one*" → **UpdateReplacePolicy: Retain/Snapshot**

Pocket card

Keyword	Answer
Preview changes before update	Change Sets
Many accounts + regions, one template	StackSets
Keep data on stack delete	DeletionPolicy Retain / Snapshot
Detect manual console edits	Drift Detection
Infra in Python/TypeScript	CDK
Protect resource during updates	Stack Policy

Keyword	Answer
Reusable template components	Nested Stacks
Share values between stacks	Outputs + Export / ImportValue
Only required template section	Resources
Failed creation	Automatic rollback (default)
Wait for app install during CREATE	CreationPolicy + cfn-signal
Resource ordering	DependsOn
Rolling ASG update	UpdatePolicy
Keep old resource on replacement	UpdateReplacePolicy

You now know how to stamp out infrastructure across many accounts — next comes the question of who governs all those accounts, which is Organizations and Control Tower territory.

↑ Back to index

Section 29: AWS Organizations & Control Tower

The idea

Real companies don't run one AWS account — they run dozens: one per team, per environment, per project, because accounts are the strongest isolation boundary AWS offers. **AWS Organizations** is how you herd them: one **management account** at the top, and every other account (a **member account**) hangs off a tree: **Root → OUs (Organizational Units) → accounts**.

Think of it as an office building. The management account is the building owner. Each OU is a floor (Production floor, Sandbox floor), each account is an office on that floor. The owner can post rules on any floor — "no open flames on floor 3" — and every office on that floor must obey, *no matter what their own office policy says*. Those building rules are **SCPs**.

Core concepts

Consolidated billing — the whole organization gets **one bill**, paid by the management account. Better yet, usage is **pooled for volume discounts**, and **Reserved Instances and Savings Plans are shared across accounts** by default: if account A bought an RI it isn't using, account B's matching instance gets the discount. "Multiple accounts, want one invoice and shared discounts" → Organizations consolidated billing. You still get a **combined view plus a detailed per-account cost report**.

Organizations also lets you **create accounts programmatically (automate account creation)** and **group them into OUs**. What it does **not** do is authenticate people: there's **no external-IdP login on Organizations itself** — workforce sign-in is **IAM Identity Center**.

SCPs (Service Control Policies) — the exam's favorite. Three facts to burn in:

- SCPs are **permission CEILINGS**, not grants. They define the *maximum* of what identities in an account **can possibly do** — they **never give anyone permission**. A user still needs an IAM policy that allows the action.
- **Effective permission = SCP allows it AND IAM allows it.** Either one denying = denied.
- They attach to the **Root, OUs, or individual accounts** and cascade down the tree. Classic use: explicit-deny guardrails — "prevent every account from using regions outside eu-west-1," "prevent anyone from disabling CloudTrail."

THE trap: **SCPs do NOT apply to the management account.** The building owner is exempt from the building rules. If a question says "the restriction must also bind the management account," an SCP alone can't do it — and that's the point being tested. (Corollary best practice: keep workloads out of the management account.)

A second sneaky pattern: "*A user has AdministratorAccess in a member account but gets Access Denied.*" Nothing is broken — **check the SCP** on their account or OU.

One-line contrast: **IAM policy = what a user CAN do; SCP = the maximum an account can EVER do.** SCPs restrict accounts — they **don't authenticate or grant access to teams**.

Who-does-what in the identity/governance map:

Need	Service
Manage/group many AWS accounts, one bill	**Organizations**
Guardrails across accounts	**SCPs**
Employees' single login across accounts (incl. corporate directory)	**IAM Identity Center**
Windows AD / domain-joined EC2	**Directory Service**
Mobile/web app customers	**Cognito**
Pre-built governed landing zone	**Control Tower**

Control Tower — Organizations gives you the raw tree; **Control Tower is the automated, opinionated setup on top of it**. It builds a **landing zone** (a pre-architected multi-account environment with logging, audit accounts, SSO), and provides:

Piece	What it does
Account Factory	Self-service creation of **new, standardized, pre-configured accounts**
Preventive guardrails	Implemented as **SCPs** — *block* disallowed actions
Detective guardrails	Implemented as **AWS Config rules** — *detect and flag* violations after the fact

Signal phrase: "set up / automate a **governed, secure multi-account environment** with best practices" → **Control Tower**. If the question is just about grouping accounts or billing, plain Organizations suffices.

RAM (Resource Access Manager) — share actual **resources** across accounts without duplicating them: **VPC subnets** (multiple accounts launching into one shared VPC), **Transit Gateways**, **Route 53 Resolver rules**, License Manager configs. "Avoid building the same networking in every account" → RAM. RAM can share **within your Organization (OUs/accounts) or with any individual AWS account**; for **VPC sharing**, the owner shares **subnets** (not the VPC object) with participant accounts **in the same Organization**.

Question patterns

> "*Prevent all accounts in the organization from launching resources outside approved regions*" → **SCP on the root/OU** (org-wide ceiling = SCP)

> "*An SCP denies an action, yet the management account can still perform it — why?*" → **SCPs don't apply to the management account** (the exemption trap)

> "*New teams need AWS accounts that come pre-configured with security baselines, via self-service*" → **Control Tower Account Factory** (standardized account vending)

> "*Company wants a single bill and to share Reserved Instance discounts across accounts*" → **Organizations consolidated billing** (pooled usage, shared RI/SP)

> "*Multiple accounts must use the same VPC subnets / a central Transit Gateway*" → **AWS RAM** (share, don't duplicate)

> "*IAM user has full admin policy in a member account but is denied — what to check?*" → **The SCP** (ceiling overrides IAM allow)

> "*Automatically set up a multi-account environment following AWS best practices*" → **Control Tower** (landing zone + guardrails)

> "*Guardrail that flags noncompliant resources but doesn't block creation*" → **Detective guardrail** (Config rule; preventive = SCP = block)

> "*Teams must log in with corporate credentials across all accounts — use SCPs?*" → **No — IAM Identity Center** (SCPs restrict, they don't authenticate; Organizations has no external login)

> "*Automate creation of new accounts and group them by business unit*" → **AWS Organizations** (account creation API + OUs; Control Tower Account Factory if baselines are needed)

Pocket card

Keyword	Answer
Block actions org-wide / by OU	SCP
SCP grants permissions?	Never — ceiling only (needs IAM allow too)
SCP vs management account	Doesn't apply — exempt
One bill, shared RI/SP discounts	Consolidated billing
Automated governed multi-account setup	Control Tower
Self-service standardized new accounts	Account Factory
Preventive guardrail	SCP (blocks)
Detective guardrail	Config rule (flags)
Share subnets / TGW / resolver rules	RAM
Admin denied in member account	Check the SCP
Account managed under Organizations	Member account
SCP for authentication/login?	No — IAM Identity Center
VPC sharing via RAM	Share subnets, same Organization
Per-account cost detail within one bill	Consolidated billing report

Governance says which accounts may do what — the next layer down is letting teams safely launch only pre-approved infrastructure, which is Service Catalog's job.

↑ Back to index

Section 30: Service Catalog & Trusted Advisor

The idea

Two small services, one shared theme: keeping humans out of trouble.

AWS Service Catalog is a **vending machine of pre-approved infrastructure**. The cloud team stocks the machine; everyone else just presses a button and gets a compliant, working stack — no keys to the kitchen required. Concretely: administrators define **products** (which are really **CloudFormation templates**) and group them into **portfolios**, then grant end users access to launch them. The magic trick: users can launch these stacks **without holding broad IAM permissions themselves** — Service Catalog launches on their behalf using its own constrained role. So a data scientist can spin up "the approved EMR cluster" without ever having permission to create EMR clusters directly.

Signal phrase: "allow teams to deploy **only approved, standardized, compliant configurations** via self-service" → **Service Catalog**.

Trusted Advisor is your **automated AWS consultant** — a robot auditor that continuously scans your account and nags you about the exact things a human consultant would bill you for.

Trusted Advisor's five pillars

Pillar	Example checks
Cost Optimization	Idle EC2 instances, unattached Elastic IPs, underused RDS
Performance	Overutilized instances, high-latency configs
Security	Security groups open to 0.0.0.0/0, **MFA missing on root**, public S3 buckets
Fault Tolerance	Missing Multi-AZ, snapshots not taken, single-AZ exposure
Service Limits	You're approaching a **service quota** (e.g., near your VPC or EC2 limit)

THE trap (tested nuance): **the free tier of Trusted Advisor gives only a handful of core checks** (basic security + service limits). To unlock the **full set of checks — including all Cost Optimization checks — you need a Business or Enterprise support plan.** If a question asks "why can't the team see the cost optimization checks?", the answer is their support plan, not a permissions problem.

Also worth one neuron each:

- Trusted Advisor **recommends**; it doesn't fix. Pair it with EventBridge/CloudWatch for alerting on check status changes.
- Trusted Advisor **can't define custom rules** — need your own compliance rule (e.g., "IAM password policy must require X" via the managed rule `iam-password-policy`)? → **AWS Config**.
- The **Service Limits** check flags a quota once usage passes **80%**; Performance/Fault Tolerance checks also surface **over-utilized resources**.
- Distinguish from neighbors: **Config** = "is this resource *compliant with my rules*?"; **Inspector** = software vulnerabilities on workloads; **Trusted Advisor** = broad best-practice checks across the five

pillars, including cost and limits.

The governance look-alikes (the exam lines them up as options):

Service	Primary purpose
CloudTrail	"**Who did what?**" — API auditing
AWS Config	"**What changed, and is it compliant?**" — config tracking + custom rules
Trusted Advisor	Best-practice **recommendations** (cost, security, limits...)
IAM credential report	Status of IAM users' **credentials**
Control Tower	**Landing zone** for a governed multi-account setup
Service Catalog	Catalog of **approved products/templates**
Security Hub	Aggregated **security findings** dashboard (doesn't prevent attacks)
RAM	**Share resources** across accounts
Systems Manager OpsCenter	Central place for **operational issues** (OpsItems)
Well-Architected Tool	Reviews whether a **workload's architecture** follows best practices
Personal Health Dashboard	AWS events affecting **your** resources (vs Service Health Dashboard = public, all customers)

Question patterns

> "*Non-technical teams must deploy infrastructure, but only pre-approved configurations, without granting them broad IAM permissions*" → **Service Catalog** (vending machine: products in portfolios, launch-on-behalf)

> "*Identify idle instances and unattached Elastic IPs to reduce the bill*" → **Trusted Advisor** (Cost Optimization pillar)

> "*Team can only see a few Trusted Advisor checks — why?*" → **Support plan** (full checks need Business/Enterprise)

> "*Get warned before hitting an AWS service quota*" → **Trusted Advisor Service Limits check** (limits = its own pillar)

> "*Flag security groups allowing unrestricted access and missing root MFA*" → **Trusted Advisor** (Security pillar)

> "*Central IT curates standard CloudFormation-based products for business units*" → **Service Catalog portfolios** (products live in portfolios, shared to users/accounts)

> "*Continuously check that the IAM password policy meets company rules*" → **AWS Config rule** (`iam-password-policy`) — Trusted Advisor can't define custom rules

> "*Is AWS EC2 having an outage affecting MY instances?*" → **AWS Health / Personal Health Dashboard** (Service Health Dashboard shows only public, general events)

Pocket card

Keyword	Answer
Self-service approved stacks, minimal IAM	Service Catalog
Product / portfolio	Service Catalog terms (CFN template / group of products)
Idle / unused / unattached resources	Trusted Advisor (Cost)
Approaching quota warning	Trusted Advisor (Service Limits)
Open SGs, root MFA check	Trusted Advisor (Security)
Full checks locked	Business/Enterprise support plan
Five pillars	Cost, Performance, Security, Fault Tolerance, Service Limits
Custom compliance rule (e.g., password policy)	AWS Config, not Trusted Advisor
Service Limits alert threshold	80% of quota
Who did it? / What changed? / Recommendations?	CloudTrail / Config / Trusted Advisor
Events affecting my resources	Personal Health Dashboard

Trusted Advisor tells you the bill *could* be smaller — the cost management tool zoo in the next section is how you actually see, alert on, and shrink it.

↑ Back to index

Section 31: Cost Management Tools

The idea

AWS billing tools all sound alike, so think of your AWS bill as a household budget. One tool is the **bank app** where you browse where the money went (Cost Explorer). One is the **spending alarm** on your phone (Budgets). One is the **shoebox of every raw receipt** (Cost & Usage Report). One is your **suspicious spouse** noticing you spent triple the usual on takeout (Anomaly Detection). One is the **personal trainer** telling you your gym membership is too big for your usage (Compute Optimizer). The exam never asks you to master any of them — it asks you to **pick the right one from a verb**.

The tool zoo

Tool	One-liner	Signal words
Cost Explorer	Visualize and filter **historical** spend, **forecast ~12 months** ahead, get **RI/Savings Plan purchase recommendations**	"analyze," "visualize," "which service drives the bill"
AWS Budgets	Set thresholds on cost or usage; **alert** (or trigger actions) when actual/forecast crosses them	"notify," "alert at 80%," "exceeds"
Cost & Usage Report (CUR)	The **most detailed** line-item billing data, delivered **to S3**, queried with **Athena** (or QuickSight)	"granular," "most detailed," "line items," "SQL"
Cost Anomaly Detection	**Machine learning** watches spend and alerts on **unusual spikes** — no thresholds needed	"unexpected," "unusual," "anomaly"
Compute Optimizer	ML **rightsizing recommendations** for EC2, EBS volumes, Lambda, ASGs	"overprovisioned," "rightsize," "recommend instance size"
Cost Allocation Tags	Tag resources (team/project), then slice costs by tag — must be **activated in the Billing console**	"cost per team/department/project," "chargeback"

Two footnotes:

- **Savings Plans / Reserved Instances** are the *commitment discounts* themselves (covered with EC2 pricing) — Cost Explorer is where AWS **recommends** buying them based on your history.
- **Billing Conductor** — niche one-liner: build *customized* (pro forma) billing views/rates for showing costs to internal groups or customers.

THE trap: **Budgets alerts, Explorer analyzes.** If the scenario wants a *notification when spending hits X%*, Cost Explorer is the decoy — it has forecasts but doesn't page you. And in the other direction, Budgets won't give you pretty historical breakdowns.

Second nuance worth a mark: tags don't help billing **retroactively or automatically** — a cost allocation tag only starts appearing in billing data **after you activate it**, and only for usage from then on.

Third lookalike: **Compute Optimizer vs Well-Architected Tool.** Compute Optimizer asks "*are my **resources** the right **size**?*" (EC2/EBS/Lambda/ASG metrics → rightsizing). The WA Tool asks "*is my*

architecture *designed correctly across all six pillars?*" (a questionnaire → improvement plan). Resource sizing ≠ design review.

Reserved Instance billing quirks:

- When an RI **expires**, the instance keeps running and is billed at the **On-Demand** rate.
- **Terminating** the instance does **not** stop RI charges. You pay for the full term.
- A Reserved Instance is a billing commitment, not a specific machine.

The decision line: SEE spend → Cost Explorer. **ALERT** on spend → Budgets. **RAW detail** → CUR + Athena. **UNUSUAL** spend → Anomaly Detection. **RIGHTSIZE** → Compute Optimizer. **Per-team chargeback** → Cost Allocation Tags.

Question patterns

> "*Receive a notification when forecasted spend reaches 80% of the monthly budget*" → **AWS Budgets** (threshold + notify = Budgets, every time)

> "*Finance needs to attribute AWS costs to individual teams and projects*" → **Cost Allocation Tags** (tag, activate in billing, slice the bill)

> "*Analysts need the most granular billing data, queryable with standard SQL*" → **CUR + Athena** (detailed line items land in S3; Athena is the SQL)

> "*Identify EC2 instances that are overprovisioned and get sizing recommendations*" → **Compute Optimizer** (ML rightsizing)

> "*Get alerted about unexpected spending spikes without setting thresholds*" → **Cost Anomaly Detection** (ML learns 'normal', flags weird)

> "*Understand which service caused last month's bill increase and forecast next quarter*" → **Cost Explorer** (visualize history + 12-month forecast)

> "*Decide how much Savings Plan commitment to purchase*" → **Cost Explorer recommendations** (based on your historical usage)

> "*Check whether the workload's overall design follows AWS best practices (not just instance sizes)*" → **Well-Architected Tool** (Compute Optimizer only rightsizes resources)

> *"A 1-year RI ended last week; what happens to the running instance's bill?"* → **It keeps running at On-Demand rates** (nothing stops, the discount just disappears)

Pocket card

Keyword	Answer
Visualize / analyze spend, forecast	Cost Explorer
Alert at % of budget	AWS Budgets
Most detailed / line items / SQL	CUR + Athena
Unusual spike, ML detection	Cost Anomaly Detection
Rightsize EC2/EBS/Lambda	Compute Optimizer
Cost per team / chargeback	Cost Allocation Tags (activate first)
RI / Savings Plan buy advice	Cost Explorer
Custom internal billing views	Billing Conductor
Review design against pillars	Well-Architected Tool (not Compute Optimizer)
RI expired	Falls back to On-Demand price
One bill across accounts	Organizations consolidated billing

That closes out the governance-and-cost corner of the exam — from here, every scenario about "who pays, who sees, who's warned" should feel like matching verbs to vending machines.

↑ Back to index

Section 32: Direct Connect & VPN

The idea

Everything so far has lived *inside* AWS. But real companies have an office, or a whole datacenter, full of servers that need to talk to their VPCs (Virtual Private Clouds — your private networks in AWS). This section is about building that bridge.

You have two ways to connect your building to AWS. A **Site-to-Site VPN** (Virtual Private Network) is like sending an **armored car on public roads**: your data travels over the ordinary public internet, but wrapped in an encrypted IPsec tunnel so nobody can peek inside. It's fast to arrange and cheap, but you share the roads — traffic jams (latency) happen. **Direct Connect (DX)** is like building **your own private toll road** straight from your building to AWS: a dedicated physical fiber line. Consistent, fast, private — but it takes a long time to pave.

Site-to-Site VPN

- **Encrypted IPsec tunnel over the public internet.**
- **Setup time: HOURS.** Cheap — you pay a small hourly charge plus data transfer.
- Throughput: roughly **~1.25 Gbps per tunnel**; latency varies because the internet is the internet.
- Two components, one on each side:

Component	Lives where	What it is
Virtual Private Gateway (VGW)	AWS side	The VPN endpoint attached to your VPC
Customer Gateway (CGW)	Your side	Represents your on-prem router/firewall

- Same thing, many names: **VPN Connection / IPsec VPN / hardware VPN** (older material also says "software VPN" when the on-prem end is software). Full vocabulary: **VGW** (AWS side) · **Customer Gateway** (the *AWS resource* describing your device) · **Customer Gateway device** (the actual physical/software appliance) · **VPN connection** (the link between them) · **VPN tunnel** (each encrypted path — a connection has **two** for redundancy).
- **One VGW per VPC**: a VPC can have only **one virtual private gateway attached at a time**.
- Best fit: **immediate need, low-to-modest bandwidth, can tolerate internet variability**.
- **Client VPN** is the cousin: it connects **individual laptops** to a VPC — think "remote employees working from home," not "connect the datacenter."

VPN CloudHub — branch offices talking through AWS

A plain VPN links *one* remote network to a VPC; it won't relay traffic **between** two remote sites. **AWS VPN CloudHub** does: several Site-to-Site VPNs terminate on the **same VGW**, and the VGW becomes a **hub-and-spoke** relay so **branch offices can reach each other** (and the VPC) over their existing internet links. Low cost, works as **primary or backup** connectivity. It's for connecting **sites/VPNs, not VPCs** — many VPCs is a Transit Gateway job. Signal: "*multiple branch offices must communicate with each other through AWS*" → **CloudHub**.

Direct Connect (DX)

- **Dedicated private fiber** from your location to AWS. Never touches the public internet.
- **Consistent bandwidth and low latency**: 1, 10, or 100 Gbps.
- **Setup time: WEEKS to MONTHS** (someone has to physically run fiber) to a **DX location**, and it costs real money — so it's the **wrong answer for a one-time data transfer** (use Snowball/DataSync instead).
- A **VIF (virtual interface) = a logical connection** carved out of the physical DX link.
- **VPC peering doesn't extend over DX**: a VPC peered to the DX-connected VPC can't ride its link (no edge-to-edge routing — Section 6).
- **DX + Transit Gateway** is the scalable combo: lower network costs, more bandwidth and more reliability than the internet, reaching many VPCs through one hub.
- **THE trap: Direct Connect is NOT encrypted by default.** Private is not the same as encrypted. If a question demands encryption over DX, the answer is **run a Site-to-Site VPN over the Direct Connect link** (IPsec on the private road).

Virtual interfaces (VIFs) — how traffic gets sorted on the fiber:

VIF type	Goes to
Private VIF	Your VPC
Public VIF	AWS public services (S3, DynamoDB endpoints)
Transit VIF	A Transit Gateway

Direct Connect Gateway lets **one DX connection reach many VPCs across many regions** — no need for a separate fiber per VPC.

Resiliency patterns

- **DX + VPN failover** — THE classic exam pattern. "We need a **cost-effective backup** for our Direct Connect" → add a **Site-to-Site VPN** as standby.
- **Two DX connections** (ideally at different locations) → maximum resiliency, maximum cost.
- **VPN now, DX later** — need connectivity this week while the fiber is being installed? Start with VPN, cut over when DX is ready.

The decision in one breath

- Need it in **hours**, **cheap**, **encrypted** → **Site-to-Site VPN**
- Need **consistent performance**, **large steady data volumes**, or compliance says **"must not traverse the public internet"** → **Direct Connect**
- Want both reliability worlds → **DX primary + VPN backup**

Question patterns

> "*Transferring 5 TB nightly; VPN performance is inconsistent*" → **Direct Connect** (dedicated fiber =

consistent throughput for big regular transfers)

"*Cost-effective backup for an existing Direct Connect link*" → **Site-to-Site VPN failover** ("cost-effective backup" is the VPN's middle name)

"*Data over Direct Connect must be encrypted in transit*" → **Site-to-Site VPN over the DX connection** (DX is private, not encrypted)

"*Must connect on-premises to AWS within days*" → **Site-to-Site VPN** (DX takes weeks–months; VPN takes hours)

"*Remote employees' laptops need secure access to the VPC*" → **AWS Client VPN** (individual devices, not a site)

"*One Direct Connect must reach VPCs in multiple regions*" → **Direct Connect Gateway** (one fiber, many VPCs/regions)

"*On-prem must reach AWS with no traffic on the public internet*" → **Direct Connect** (the only option that skips the internet entirely)

"*Several branch offices with existing internet links must communicate with each other via AWS, low cost*" → **AWS VPN CloudHub**

"*One-time migration of data to AWS*" → **not Direct Connect** (months to provision, ongoing cost) — Snow Family / DataSync

"*Attach a second VGW to the same VPC*" → **not possible** — one VGW per VPC

Pocket card

Keyword	Answer
Encrypted tunnel, quick, cheap	Site-to-Site VPN
VGW + CGW	Site-to-Site VPN components (AWS + customer side)
Consistent bandwidth, dedicated	Direct Connect
Setup in hours	VPN

Keyword	Answer
Setup in weeks–months	Direct Connect
Encrypt Direct Connect	VPN over DX
Cost-effective DX backup	Site-to-Site VPN
One DX → many VPCs/regions	Direct Connect Gateway
Remote workers → VPC	Client VPN
Private VIF / Public VIF / Transit VIF	VPC / public services / Transit Gateway
Branch offices ↔ each other via AWS	VPN CloudHub
VGWs per VPC	One
VIF	Logical connection on a DX link

You now know how ONE office connects to AWS — next up, what happens when dozens of VPCs and offices all need to talk to each other without a wiring nightmare: Transit Gateway.

↑ Back to index

Section 33: Transit Gateway

The idea

Remember VPC peering? It connects exactly **two** VPCs — a **1:1 cable** between two buildings. And it's **non-transitive**: if A peers with B and B peers with C, A still *cannot* reach C. Every pair needs its own cable.

Now do the math. Full connectivity between n VPCs needs **n(n-1)/2** peering connections. **10 VPCs = 45 peerings.** 25 VPCs = 300. Add on-premises links and you have a plate of spaghetti nobody can debug.

Transit Gateway (TGW) is the **airport hub**. Instead of every city flying direct to every other city, everyone flies through the hub. Each VPC, VPN, or Direct Connect link gets **one attachment** to the TGW, and the hub routes between all of them — a **star topology** with **transitive routing** (A can reach C through the hub, no direct cable needed).

What you can attach

Attachment	Note
VPCs	Thousands, one attachment each
Site-to-Site VPN	On-prem via internet
Direct Connect	Via a **transit VIF**
Another TGW	**TGW peering** for cross-region hubs

- **Cross-account sharing**: share a TGW with other AWS accounts using **AWS RAM** (Resource Access Manager) — one hub for the whole organization.
- **Route tables per attachment = segmentation.** Give prod and dev different TGW route tables and **prod can't see dev**, even though both hang off the same hub. This is how you isolate environments while still routing everyone to on-premises.
- **THE trap: TGW is the ONLY AWS service that supports IP multicast.** See the word "multicast" → answer is Transit Gateway, done.
- **Appliance mode**: keeps both directions of a traffic flow going through the *same* network appliance (firewall/inspection VPC) — the keyword for centralized traffic inspection.

Transit VPC — the legacy DIY hub

Before TGW, the pattern was a **transit VPC**: a hub VPC running **VPN appliances on EC2** that you build, patch and scale yourself, with spoke VPCs and data centers VPN-ing into it (it could span Regions). TGW is the **managed replacement** — one connection per VPC, on-prem network or remote office. If an option says "transit VPC", treat it as the **more-operational-overhead** distractor.

The connectivity chooser (where hubs fit):

Need	Answer
Two VPCs, simple	VPC Peering (incl. inter-region)

Need	Answer
Many VPCs/accounts/Regions + on-prem, transitive	**Transit Gateway** (+ DX via transit VIF for cost/bandwidth)
Branch offices talking to each other over VPN	VPN CloudHub (Section 32)
Legacy self-managed hub	Transit VPC

Cost reality check

TGW charges **per attachment per hour + per GB processed**. VPC peering has **no hourly charge** (just data transfer). So:

- **2–3 VPCs** that just need to talk → **peering is still the right answer** (cheaper, simpler).
- **Many VPCs, on-prem connectivity, transitive routing** → TGW earns its keep.

TGW vs PrivateLink (quick recap)

- **TGW / peering = network marriage**: the whole networks join; everything routable can talk.
- **PrivateLink = service window**: expose ONE service through a small hatch; consumer sees only that endpoint, nothing else. "Expose one service to 100 customer VPCs, no network merge, overlapping CIDRs fine" → PrivateLink, not TGW.

Question patterns

> "*25 VPCs and on-premises datacenter need centralized connectivity*" → **Transit Gateway** (hub-and-spoke kills the n(n-1)/2 mesh)

> "*VPC A peers with B, B peers with C, but A can't reach C*" → **Transit Gateway** (peering is non-transitive; TGW routes transitively)

> "*Prod VPCs must not communicate with dev VPCs, but all need on-prem access*" → **TGW with separate route tables per attachment** (segmentation at the hub)

> "*Application requires IP multicast*" → **Transit Gateway** (the only service that supports it)

> "*Just 2 VPCs need to communicate at lowest cost*" → **VPC peering** (free hourly, TGW would be overkill)

> "*Share the network hub across 50 AWS accounts*" → **TGW shared via AWS RAM**

> "*Connect hub networks in us-east-1 and eu-west-1*" → **TGW peering** (cross-region hub-to-hub)

> "*Replace a transit VPC of EC2 VPN appliances with less operational overhead*" → **Transit Gateway**

> "*All inter-VPC traffic must pass through a firewall appliance*" → **TGW with appliance mode** (symmetric flows through the inspection VPC)

Pocket card

Keyword	Answer
Many VPCs + on-prem, central hub	Transit Gateway
Transitive routing	Transit Gateway (peering can't)
n(n-1)/2 peering explosion	Transit Gateway
Multicast	Transit Gateway (only one)
Isolate prod/dev at the hub	TGW route tables
Cross-account TGW	Share via AWS RAM
Cross-region hubs	TGW peering
DX into TGW	Transit VIF
Only 2–3 VPCs, cheapest	VPC peering
Expose one service, no network merge	PrivateLink (not TGW)
Self-managed EC2 VPN hub (legacy)	Transit VPC → replace with TGW
Branch office ↔ branch office over VPN	VPN CloudHub (not TGW)

That's the plumbing done — next we zoom all the way out to how AWS wants you to *think* about architecture: the Well-Architected Framework.

↑ Back to index

Section 34: Well-Architected Framework

The idea

The Well-Architected Framework is AWS's answer to "what does *good* look like in the cloud?" Think of it as a **building inspector's checklist**: a house can stand up and still fail inspection — bad wiring, no fire exits, a heating bill that eats you alive. The framework's **six pillars** are the six things the inspector checks, and on the exam they don't appear as their own questions so much as the *lens* a question forces you to look through.

The six pillars

Pillar	One-line essence	Flagship services
Operational Excellence	Run and improve: everything as code, small reversible changes	CloudFormation, CloudWatch
Security	Least privilege, defense in depth, encrypt everywhere	IAM, KMS, GuardDuty
Reliability	Survive failures, recover automatically, stop guessing capacity	Multi-AZ, Auto Scaling Groups, Route 53
Performance Efficiency	Right tool for the job, serverless-first, go global	Lambda, CloudFront, ElastiCache
Cost Optimization	Pay only for what you use, and measure it	Cost Explorer, Savings Plans, Spot
Sustainability	Maximize utilization, minimize wasted compute	Graviton, serverless

A memory hook: **"SO CRPS"** doesn't sing, so try this instead — **"Operations Secure Reliable Performance Costs Sustainably"**, or just picture the inspector's walkthrough: *how you run it, how you lock it, how it survives, how fast it goes, what it costs, what it burns.*

Quick flavor of each:

- **Operational Excellence** — infrastructure as code (CloudFormation), frequent small deployments you can roll back, learn from every failure.
- **Security** — apply least privilege (grant only what's needed), defend in layers (WAF + security groups + NACLs), encrypt at rest and in transit.
- **Reliability** — assume everything fails; design so nobody notices. Multi-AZ, health checks, auto-recovery.
- **Performance Efficiency** — don't run a database on EC2 when RDS exists; cache with CloudFront/ElastiCache; experiment often.
- **Cost Optimization** — right-size, use Spot for interruptible work, Savings Plans for steady work, and *look at the bill* (Cost Explorer).
- **Sustainability** — the newest pillar: fewer idle servers, efficient hardware (Graviton), serverless where possible.

The Well-Architected Tool is a **free questionnaire in the console**: you answer questions about a workload, pillar by pillar, and it produces an **improvement plan**. "Review our architecture against best practices at no cost" → WA Tool.

Lookalike tools — which question does each answer?

Tool	Question it answers
Well-Architected Tool	"*Is my **architecture** designed correctly?*" It reviews the workload across all six pillars.
Compute Optimizer	"*Are my **resources** the right **size**?*" It rightsizes EC2, EBS, Lambda and ASGs only.
Trusted Advisor	"*Is anything in my **account** misconfigured right now?*" It runs automated checks for cost, security, fault tolerance, performance and service limits.

The exam angle (this is the real payoff)

Pillars almost never show up as "which pillar is this?" Instead they hide in the **qualifier** of the question: "**MOST cost-effective** solution," "improve the **reliability**," "with the **LEAST operational overhead**." That qualifier is the tiebreaker — typically **two answers would technically work**, and the pillar keyword tells you which one AWS wants.

THE trap: answering the *working* solution instead of the *qualified* solution. If two options both solve the problem, the capitalized adjective picks the winner. Read it first, not last.

Question patterns

> "*Two solutions both meet requirements; question asks for the MOST cost-effective*" → **the cheaper one that still works** (Cost Optimization is steering — e.g., Spot over On-Demand for fault-tolerant batch)

> "*Improve reliability of a single-AZ database*" → **Enable Multi-AZ** (Reliability pillar: survive AZ failure with automatic failover)

> "*LEAST operational overhead to run containers*" → **Fargate over EC2 launch type** (Operational Excellence/managed-first: no servers to patch)

> "*Improve performance for global users*" → **CloudFront** (Performance Efficiency: go global, cache at the edge)

> "*Ensure the architecture follows AWS best practices at no additional cost*" → **AWS Well-Architected Tool** (free console questionnaire → improvement plan)

> "*Get recommendations to improve a workload's reliability, security and cost based on AWS best practices*" → **Well-Architected Tool** (whole-design review; Compute Optimizer only resizes resources)

> "*Reduce the environmental impact of the workload*" → **Graviton instances / serverless** (Sustainability: maximize utilization)

Pocket card

Keyword	Answer
IaC, small reversible changes	Operational Excellence (CloudFormation)
Least privilege, encrypt everywhere	Security (IAM, KMS, GuardDuty)
Survive failure, stop guessing capacity	Reliability (Multi-AZ, ASG, Route 53)
Right tool, serverless-first, edge	Performance Efficiency (Lambda, CloudFront)
Pay for what you use, measure spend	Cost Optimization (Cost Explorer, Savings Plans, Spot)
Maximize utilization, Graviton	Sustainability
Free best-practices review	Well-Architected Tool
Right-size instances (not design)	Compute Optimizer
"MOST cost-effective / LEAST overhead"	The qualifier picks between two working answers

Keep those qualifiers in your peripheral vision — they matter most in the next section's territory: disaster recovery, where cost versus recovery speed *is* the whole question.

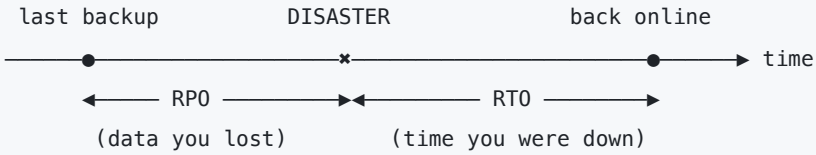
↑ Back to index

Section 35: Disaster Recovery

The idea

Disaster recovery (DR) answers one question: *when a whole region catches fire, how fast are you back, and how much data did you lose?* Two numbers define everything:

- **RPO — Recovery Point Objective**: maximum acceptable **data loss**. How far back does the tape rewind? If you back up nightly, your RPO is up to 24 hours of lost work.
- **RTO — Recovery Time Objective**: maximum acceptable **downtime**. How long until you're serving customers again?



Think of DR strategies as **kinds of spare tires**. Backup & Restore is *no spare — call a tow truck*. Pilot Light is a *space-saver donut in the trunk*. Warm Standby is a *full spare, already inflated*. Active-Active is *a second identical car driving alongside you*. Better spares cost more to carry around; the exam always asks for the **cheapest one that meets the stated RPO/RTO** — the same eliminate-then-cheapest logic you used for S3 storage classes.

The four strategies

Strategy	RTO	RPO	Cost	What runs in the DR region
Backup & Restore	Hours	Hours	$ cheapest	Nothing — backups in S3, rebuild on disaster
Pilot Light	Tens of minutes	Minutes–seconds	$$	Core only: **DB continuously replicated, servers off/minimal**
Warm Standby	Minutes	Seconds	$$$	**Scaled-DOWN full copy always running** → scale UP on disaster
Multi-Site Active-Active	~Zero	~Zero	$$$$	Full capacity in both regions, serving live traffic

Signal phrases:

- "**Minimal DR footprint**, database continuously replicated, compute launched only on disaster" → **Pilot Light** (the pilot flame is lit; the furnace is off).
- "A smaller version of the full environment is **always running**" → **Warm Standby**.
- "**Zero downtime**, both regions active" → **Multi-Site Active-Active**.
- "Lowest cost, RTO of hours is fine" → **Backup & Restore**.

- Multi-Site is also called **"Hot Standby"** — both sites take a share of live traffic.
- Map to Route 53 (see Section 7): **active-active** (Weighted/Latency + health checks, every site serves, no primary/secondary) = Multi-Site; **active-passive** (Failover policy, primary serves, secondary waits) = Pilot Light / Warm Standby.

THE trap: don't buy a better spare tire than the question asks for. If RTO is "a few hours," Active-Active is wrong even though it's "better" — it fails the cost qualifier.

The supporting cast

Service	Role in DR
Aurora Global Database	Cross-region replication, **RPO ~1 second, RTO < 1 minute**
DynamoDB Global Tables	Multi-region, multi-active NoSQL — writes anywhere
S3 Cross-Region Replication (CRR)	Objects copied to a bucket in another region
AWS Backup	**Centralized, cross-service backup plans** (EBS, RDS, DynamoDB, EFS…) + **Vault Lock** for immutable backups
Elastic Disaster Recovery (DRS)	**Continuous block-level replication of servers** (on-prem or EC2) into AWS — cheap DR for lifted servers, RPO seconds / RTO minutes
Route 53 failover routing	Health check fails → DNS flips traffic to the DR region
DMS	Database Migration Service — also does ongoing replication between databases
AMI copy	AMIs are Region-bound — copy them to the DR region ahead of time (see Section 3)
RDS / Redshift cross-region snapshot copy	Ship DB snapshots to the DR region (Redshift: enable on the cluster with destination region + retention) — Backup & Restore tier
ElastiCache Redis Global Datastore	Cross-region cache replicas; promote a secondary in a regional outage

AWS Backup, a bit deeper: one console for **backup policies**, **automated cross-region AND cross-account copies** (long-term retention in an isolated account), covering AWS services **plus on-prem VMware and Storage Gateway volumes**. Contrast with **Data Lifecycle Manager (DLM)** = EBS snapshots/AMIs only. (Notes sometimes say "AWS Backup only works on AWS resources" — outdated; hybrid VMware support exists.)

Database migration with minimal downtime

Requirement	Approach
Migrate with **minimal downtime**	**DMS** (source stays live, full load + CDC)
Ongoing replication between DBs	**DMS**
One-time copy, downtime acceptable	**Snapshot → restore**
Promote a read replica to standalone	Works, but **some downtime** at cut-over
Change engine / deployment type in place	**Usually not possible** → migrate instead

- DMS does **homogeneous** (MySQL → MySQL) and **heterogeneous** (Oracle → Aurora) moves; for heterogeneous, **SCT** converts **schemas, stored procedures, foreign keys** first.
- DMS Basic Schema Copy vs SCT is covered in Section 37.

How to answer any DR question

1. Find the stated **RPO/RTO numbers** (or words like "zero downtime").
2. **Eliminate** strategies that can't meet them.
3. Pick the **cheapest survivor**.

Question patterns

> "*RTO of 10 minutes, RPO of seconds, keep costs reasonable*" → **Warm Standby** (Active-Active also works but costs more; Pilot Light is too slow)

> "*DR needed at the LOWEST cost; hours of downtime acceptable*" → **Backup & Restore** (cheapest strategy that meets a relaxed RTO)

> "*Application must survive a region failure with zero downtime and zero data loss*" → **Multi-Site Active-Active** (only ~zero RPO/RTO option)

> "*Database continuously replicated to DR region; EC2 launched only during a disaster*" → **Pilot Light** (core data live, compute off)

> "*Centrally manage and automate backups of EBS, RDS, and DynamoDB across the organization*" → **AWS Backup** (one service, backup plans + vault lock)

> "*Cost-effective DR for 200 on-premises servers into AWS*" → **Elastic Disaster Recovery (DRS)** (continuous block replication, pay tiny until failover)

> "*Global relational database with cross-region RPO of ~1 second*" → **Aurora Global Database** (RPO 1s, RTO under a minute)

> "*Automatically route users to the standby region when the primary fails*" → **Route 53 failover routing with health checks**

> "*Keep backups in a separate account for long-term retention, automatically*" → **AWS Backup cross-**

> **account copy** (centralized policies)

> "*Migrate a production database to AWS with minimal downtime*" → **DMS** (source stays operational; add SCT if the engine changes)

Pocket card

Keyword	Answer
Max data loss	RPO
Max downtime	RTO
Cheapest, RTO hours	Backup & Restore
DB replicated, servers off	Pilot Light
Scaled-down copy running	Warm Standby
Zero RPO/RTO, both live	Multi-Site Active-Active
RPO 1s / RTO <1 min relational	Aurora Global Database
Multi-region multi-active NoSQL	DynamoDB Global Tables
Centralized cross-service backups	AWS Backup (+ Vault Lock)
Replicate on-prem servers for DR	Elastic Disaster Recovery (DRS)
DNS failover to DR region	Route 53 failover routing
Backups in another account	AWS Backup cross-account copy
Migrate DB, minimal downtime	DMS (+ SCT if engine changes)
One-time DB copy, downtime OK	Snapshot → restore

You now hold the full toolbox — time to sharpen it against the exam's favorite tricks: the classic traps and must-know combos.

↑ Back to index

Section 36: Classic Traps & Must-Know Combos

The idea

The SAA-C03 isn't trying to see if you've *heard* of services — it's checking whether you know where the **tripwires** are. Think of this section as the **map of the minefield**: every trap below is a wrong answer that *sounds* right and appears on the exam over and over, paired with the truth that defuses it. Walk this list twice before exam day and you'll feel the wires before you step on them.

The greatest-hits trap list

THE trap	The truth
"Read from the RDS Multi-AZ standby to offload queries"	**NO** — the standby is invisible; use **read replicas** for reads
"CNAME record at the zone apex (example.com)"	CNAMEs can't sit at the apex — use an **Alias record**
"Add a deny rule to the security group"	Security groups have **no deny rules** (allow-only, stateful) — deny lives in **NACLs**
"Private subnet reaches S3 via NAT Gateway"	Works but costs money — **Gateway VPC Endpoint** is free and private
"Run the 20-minute job in Lambda"	Lambda caps at **15 minutes** — use Fargate/Batch/Step Functions
"Mount EFS on Windows instances"	EFS is Linux-only — Windows shared storage = **FSx for Windows File Server**
"Snowball imports directly into Glacier"	No — Snowball lands in **S3**, then a **lifecycle policy** moves it to Glacier
"A↔B and B↔C peering lets A reach C"	Peering is **non-transitive** — need **Transit Gateway** (or direct A↔C peering)
"Encrypt the existing unencrypted RDS/EBS in place"	Impossible in place — **snapshot → copy snapshot with encryption → restore**
"KMS for FIPS 140-2 Level 3 / single-tenant keys"	That's **CloudHSM** (KMS is Level 2*, multi-tenant)
"Shield Standard stops layer-7 application attacks"	Standard is L3/L4 only — L7 protection = **WAF / Shield Advanced**
"Apply an SCP to restrict the management account"	SCPs **never affect the management account**
"Replay DynamoDB Streams data from last month"	Streams keep **24 hours** — longer retention → **Kinesis Data Streams**
"Give the ALB a static IP"	ALBs have no static IPs — **NLB** has static/Elastic IPs (or NLB in front of ALB)
"Add an LSI to the existing DynamoDB table"	**LSIs only at table creation** — after creation, add a **GSI**

THE trap	The truth
"Accelerate UDP traffic with CloudFront"	CloudFront is HTTP(S) only — UDP/TCP acceleration = **Global Accelerator**
"Parameter Store rotates the DB password automatically"	No native rotation — automatic rotation = **Secrets Manager**
"One NAT Gateway serves all AZs, highly available"	AZ failure kills it — deploy **one NAT Gateway per AZ**
"SNS alone guarantees consumers never miss messages"	SNS doesn't persist for offline consumers — durable fan-out = **SNS → SQS**
"Direct Connect is encrypted because it's private"	**Not encrypted** — run a **VPN over DX** for encryption
"Auto Scaling absorbs a spike that lasts a few seconds"	New instances take **minutes** to boot, so ASG is **NOT suitable** for second-level bursts. Buffer the spike with **SQS**, or use Lambda.
"Stretch one ASG/ELB across two Regions to cut latency"	ELB and ASG are **single-Region**. Use **Route 53** latency routing or **Global Accelerator** for multi-Region.
"Multi-AZ will scale our reads"	Multi-AZ is for **HA/failover only**. Read scaling needs **Read Replicas** or Aurora Replicas.
"Throttle clients at the load balancer"	ELB cannot throttle. **API Gateway** (usage plans/throttling) or **WAF rate-based rules** can.
"CloudWatch alarm emails ops via SES"	Alarms notify through **SNS**. SES is for application, bulk or marketing email.
"Order Direct Connect for a one-time bulk transfer next week"	DX takes **weeks to months** to set up. One-time bulk transfer → **Snowball** or **DataSync** over the internet/VPN.
"Firewall Manager / Security Hub / Trusted Advisor will block the attack"	These only **manage, aggregate or advise**. Blocking is done by **WAF, Shield, NACL or Network Firewall**.
"Attach a security group to the S3 bucket"	S3 has no security groups. Control access with **bucket policies**, **VPC endpoint policies** or Block Public Access.
"Run session state or OLTP on Redshift"	Redshift is an **OLAP** warehouse. Use **DynamoDB or ElastiCache** for sessions and **RDS/Aurora** for OLTP.

Must-know combos (the exam's favorite Lego sets)

Need	The combo
Serverless web app	**CloudFront + S3 (static) + API Gateway + Lambda + DynamoDB**
Reliable fan-out	**SNS → multiple SQS queues** (durable, each consumer own queue)
Global highly available DB	**Aurora Global Database** (SQL) / **DynamoDB Global Tables** (NoSQL)
Protect a web app	**CloudFront + WAF + Shield + ALB**
Real-time streaming pipeline	**Kinesis Data Streams → Lambda → DynamoDB/S3**
Analytics on data lake	**S3 + Glue (catalog/ETL) + Athena (query) + QuickSight (dashboards)**

Need	The combo
Migration toolkit	**DataSync (files) + DMS (databases) + DX/Snow family (transport)**
DR stack	**Route 53 failover + Aurora Global + S3 CRR**
App authentication	**Cognito User Pools (login) + Identity Pools (AWS credentials)**
Classic 3-tier web app	**Web tier**: ALB in **public** subnets with an ASG behind it. **App tier**: EC2/containers in **private** subnets. **DB tier**: RDS Multi-AZ in **private** subnets. Only the load balancer faces the internet.

Design keyword → architectural goal

Keyword in the stem	What AWS wants to see
Highly available	Multi-AZ, redundancy, automatic failover (ASG across AZs, RDS Multi-AZ, one NAT GW per AZ)
Fault tolerant / survive a Region failure	Multi-Region (Route 53 failover, Aurora Global, DynamoDB Global Tables, S3 CRR)
Decouple	A queue or topic between components (**SQS**, SNS, EventBridge), so producers and consumers fail and scale independently
Real-time / streaming	**Kinesis** (or MSK if "Kafka")
Shared file storage	**EFS** (Linux) / **FSx** (Windows, Lustre)
Low-cost archive	**S3 Glacier** tiers (Deep Archive for rarely accessed data)
Serverless	Lambda, Fargate, DynamoDB, API Gateway, S3, Aurora Serverless
Read-heavy / read scalability	**Read Replicas**, Aurora Replicas, ElastiCache, DAX, CloudFront
Secure / encrypt / least privilege	IAM roles, KMS, WAF, private subnets plus VPC endpoints
Cost-efficient / MOST cost-effective	The cheapest option that **still meets every requirement** (Spot, S3 lifecycle, serverless, Savings Plans)
LEAST operational overhead	The most managed or serverless option (Glue over EMR for ETL, Fargate over EC2)
NOT suitable / which will NOT work	The question is inverted. Find the option that **breaks** a requirement (wrong Region scope, time limit, protocol).

Scenario → service quick-match

Scenario	Service
DB connection exhaustion from many Lambdas	**RDS Proxy**
Sudden traffic spikes overwhelm the backend	**SQS** buffer
One event → many consumers	**SNS** fan-out (→ SQS for durability)
Multi-step workflow with retries and branching	**Step Functions**
Multi-Region traffic routing with static IPs	**Global Accelerator**

Scenario	Service
Private access to AWS services from a VPC	**VPC Endpoint** (Gateway for S3/DynamoDB, Interface for the rest)
On-prem file server extended to AWS	**Storage Gateway** (File Gateway)
Read-heavy relational DB	**Read Replica**
Cross-Region relational DR	**Aurora Global Database**
Metrics + alarm + notify ops	**CloudWatch + SNS**
Application/bulk/transactional email	**SES**
Automated, cross-account long-term backup retention	**AWS Backup**
ETL with least operational overhead	**Glue**

Exam technique (the meta-game)

- **Read the last sentence first** — that's where the actual question and qualifier live; the story above it is scenery.
- **Eliminate two** — almost every question has two obviously wrong answers. Kill them, then compare the survivors.
- **The qualifier decides** between two working answers: **cost** → cheapest that still works; **least operational overhead** → most managed/serverless; **highly available** → Multi-AZ; **survive region failure** → multi-region.
- **~2 minutes per question** (130 min, 65 questions). Stuck at 90 seconds? **Flag it and move on** — later questions often jog your memory.
- **Answer everything.** No penalty for wrong answers; a guess beats a blank.

Question patterns

> "*Route example.com (apex) to an ALB*" → **Alias record** (CNAME illegal at apex)

> "*Block a specific malicious IP address*" → **NACL deny rule** (security groups can't deny)

> "*Private subnet needs S3 access at no cost*" → **Gateway VPC Endpoint** (NAT Gateway is the paid decoy)

> "*Encrypt an existing unencrypted RDS instance*" → **Snapshot** → **encrypted copy** → **restore** ("enable encryption" checkbox doesn't exist)

> "*Fan out orders to inventory, billing, shipping — none may be lost*" → **SNS topic** → **three SQS queues**

(durability per consumer)

"*Static IPs required by the firewall team for the load balancer*" → **Network Load Balancer** (ALB has none)

"*Improve performance AND static IPs for global TCP/UDP app*" → **Global Accelerator** (CloudFront can't do UDP)

"*Rotate database credentials automatically every 30 days*" → **Secrets Manager** (Parameter Store is the decoy)

"*Traffic surges for 5–10 seconds and requests are dropped; Auto Scaling is already configured*" → **Put SQS in front to buffer** (ASG reacts in minutes, too slow for second-level bursts)

"*Which architecture is NOT suitable…*" → **find the option that violates a hard limit** (e.g., ELB across Regions, Lambda over 15 min, CNAME at the apex)

"*Decouple the order service from the slow invoicing service*" → **SQS queue between them** ("decouple" is almost always SQS/SNS)

Pocket card

Keyword	Answer
Apex domain	Alias record
Deny traffic	NACL (never SG)
Free S3 access from VPC	Gateway Endpoint
>15 minutes	Not Lambda (Fargate/Batch)
Windows file share	FSx for Windows
Transitive routing	Transit Gateway
Encrypt existing volume/DB	Snapshot dance
FIPS Level 3	CloudHSM
Static IP load balancer	NLB
UDP acceleration	Global Accelerator
Auto-rotation	Secrets Manager
HA NAT	One per AZ
Durable fan-out	SNS → SQS

Keyword	Answer
Encrypted DX	VPN over DX
Seconds-long spike	SQS buffer (not ASG)
Multi-Region load balancing	Route 53 / Global Accelerator (ELB is single-Region)
Throttling	API Gateway / WAF rate-based rule
Decouple	SQS / SNS
Highly available	Multi-AZ
3-tier	ALB public, app + DB private

Traps mapped, combos memorized — one last sweep remains: the small-but-tested gap-fill services that each show up for one or two easy points.

↑ Back to index

Section 37: The Gap-Fill Services (small but tested)

The idea

You've built the skyscrapers — EC2, S3, VPC, RDS. This section is the **spice rack**: dozens of small jars you'll each use once, but when the recipe calls for saffron, only saffron will do. Each of these services shows up on the exam **once or twice**, almost always as a pure **keyword-match** question. You don't need depth; you need to hear the signal word and slap the right label on it. That makes this the highest points-per-minute section you'll study — easy marks hiding in plain sight.

Read each entry as: *signal keyword → service*. That's genuinely how the questions work.

Migration & messaging

Amazon MQ — managed **ActiveMQ / RabbitMQ** message broker. Signal: "*existing application uses **MQTT / AMQP / JMS** (industry-standard messaging protocols) and must migrate with **minimal rewrite**.*" THE trap: for **new** applications, AWS wants **SQS/SNS** — MQ is only for lifting existing broker-based apps.

Application Migration Service (MGN) — the **lift-and-shift (rehost)** tool. Installs an agent on your on-prem servers, does **continuous block-level replication** into AWS, then you **cut over** to EC2 with minutes of downtime. Signal: "*migrate servers to AWS with minimal changes.*"

Elastic Disaster Recovery (DRS) — MGN's twin, but for **DR instead of migration**: same continuous replication, but you only launch the recovery instances when disaster strikes. **RPO in seconds, RTO in minutes**, and it's cheap while idle. Signal: "*cost-effective disaster recovery for physical/virtual/cloud servers.*"

DMS + SCT — **Database Migration Service** moves databases (and can do **ongoing replication**, source stays live during migration). If the engines differ — Oracle → Aurora — that's a **heterogeneous** migration and you add the **Schema Conversion Tool (SCT)** first. Same engine = DMS alone; **different engine = DMS + SCT**. DMS's **Basic Schema Copy** creates only tables and primary keys. It skips **secondary indexes, foreign keys and stored procedures**, so it's fine for a test run but **not suitable for production**. Use SCT for the full schema.

Migration toolkit at a glance:

Need	Service
Inventory on-prem servers, dependencies and utilization **before** planning a migration	**Application Discovery Service**
One dashboard to **track migration progress** across tools	**Migration Hub**
Rehost **servers**: an agent replicates whole disks	**MGN** (its Replication Agent). Legacy name: CloudEndure Migration
Move **files** (NFS/SMB/HDFS) to S3/EFS/FSx, including straight into Glacier Deep Archive	**DataSync**

Need	Service
Move **databases**, minimal downtime, ongoing replication	**DMS** (+ **SCT** for schema/code)
Huge offline transfer, you bring your own drives to an AWS site	**AWS Data Transfer Terminal**: physical locations to plug in and upload at high speed

THE trap: **Replication Agent vs DataSync** = **servers vs files**. Use MGN's agent to move a whole machine. Use DataSync to move a file share.

Look-alikes that are NOT queues:

Service	What it really is
SQS	Decoupling message queue (cloud-native)
Amazon MQ	Managed broker for **existing** JMS/AMQP/MQTT/STOMP apps
Data Firehose	**Delivers** streaming data into S3/Redshift/OpenSearch. It's a pipe, not a queue.
AppStream 2.0	Streams **individual desktop applications** to a browser
WorkSpaces	A full managed **virtual desktop** (Windows/Linux VDI)

The 7 Rs of migration — one line each:

R	Meaning
Rehost	Lift-and-shift, no changes (MGN)
Replatform	Lift-and-tinker (e.g., self-managed MySQL → RDS)
Repurchase	Drop it, buy SaaS instead
Refactor	Re-architect for cloud (e.g., monolith → serverless)
Relocate	Move VMware wholesale (VMware Cloud on AWS)
Retain	Leave it on-prem for now
Retire	Turn it off, nobody used it anyway

Certificates & network odds-and-ends

ACM (AWS Certificate Manager) — **free public TLS/SSL certificates** with **automatic renewal**, attachable to ALB, CloudFront, API Gateway. Two exam facts: a certificate for **CloudFront MUST live in us-east-1** (any region works for a regional ALB), and **you cannot export ACM public certs to install on an EC2 instance** — ACM only attaches to integrated services. ACM stores **certificates**, not passwords.

IAM certificate store is the **legacy** option. You upload a certificate bought from a third party into IAM. Use it only where ACM isn't supported in a Region. Otherwise prefer ACM, because IAM certs don't auto-renew.

Egress-only Internet Gateway — the **IPv6 "NAT gateway."** IPv6 addresses are all public, so NAT doesn't apply; this gateway lets IPv6 instances make **outbound-only** connections while blocking inbound. Signal: "*IPv6 instances need outbound internet, no inbound.*"

Route 53 Resolver endpoints — hybrid DNS plumbing. **Inbound endpoint**: **on-prem → VPC** DNS queries (your datacenter resolves AWS private names). **Outbound endpoint**: **VPC → on-prem** DNS queries (AWS

resolves your corporate names). Remember it from the VPC's point of view: which way are the *queries coming*?

Edge & hybrid infrastructure

Service	One-liner	Signal keyword
Outposts	AWS-managed racks physically installed **in YOUR datacenter**	"data residency," "must stay on-premises but use AWS APIs"
Local Zones	AWS extension in a **metro city** near your users	"**single-digit millisecond** latency to users in Los Angeles"
Wavelength	AWS compute embedded in **5G telecom networks**	"5G," "mobile edge"

Analytics family

Service	One-liner	Signal keyword
Glue	**Serverless ETL** (extract-transform-load) + **Data Catalog** + crawlers that auto-discover schemas	"serverless data preparation/catalog"
EMR	Managed **Hadoop/Spark** cluster; use **Spot for task nodes** to cut cost	"**Apache Spark**," "big data frameworks"
MSK	Managed **Apache Kafka**	the word "**Kafka**" — that's it
Athena	SQL queries directly on S3, pay per query (you know this one)	"query S3 with SQL"
QuickSight	**BI dashboards** and visualizations	"business dashboards for management"
Lake Formation	Build a **data lake** fast + **fine-grained (row/column) access control**	"central data lake with granular permissions"
AppFlow	Move data from **SaaS apps (Salesforce, Slack)** → **S3/Redshift**, no code	"Salesforce data into S3"

THE trap: "streaming" alone → Kinesis; but "**Kafka**" named explicitly → **MSK**. AWS respects the proper noun.

Compute/analytics chooser — "which engine runs this job?"

Job	Best service
Scheduled or parallel scripts, lots of containerized batch jobs	**AWS Batch**
Spark/Hadoop with full framework control	**EMR** (managed clusters)
Serverless ETL, CSV → Parquet, schema discovery	**Glue**
Event-driven short code	**Lambda**
Data warehouse / complex BI SQL	**Redshift**

Job	Best service
Stream processing	**Kinesis**
Interactive ad-hoc SQL on S3	**Athena**
Workflow orchestration	**Step Functions**
Copy database data into S3/Redshift	**DMS**
Export a DynamoDB table to S3	DynamoDB **Export to S3**, or the **Glue DynamoDB connector** inside an ETL job

THE trap: **Glue vs EMR**. Both transform data in S3. **"Least operational overhead"** or "serverless" points to **Glue**. "Need Spark/Hadoop control, custom frameworks" points to **EMR**. Glue jobs are batch ETL, not real-time.

Legacy workflow tools:

- **SWF** (Simple Workflow Service) coordinates tasks and guarantees each task is assigned exactly once. It isn't serverless.
- **Data Pipeline** moved data between AWS and on-prem sources.
- For new designs, both are replaced by **Step Functions**, **Glue** or **MWAA**.

ML one-liners (pure keyword matching)

Service	Does	Signal
Rekognition	Analyze **images/video**: objects, people, text, scenes, activities, faces, inappropriate content	face detection, content moderation
Transcribe	**Speech → text**	call transcripts, subtitles
Polly	**Text → speech** (SSML for pronunciation). Not a chatbot, which is Lex.	"app reads articles aloud"
Translate	Language translation	"localize to 12 languages"
Comprehend	**NLP / sentiment** analysis, no ML experience needed	"customer review sentiment"
Comprehend Medical	NLP that extracts **medical info** from unstructured clinical text	"diagnoses/medications from doctors' notes"
Textract	**Extract text + data from scanned forms/invoices** — understands tables and fields, **beyond simple OCR**	"process scanned documents/forms"
Kendra	**Intelligent document search** (natural-language answers from your docs). Connectors for S3, SharePoint, Confluence and more.	"enterprise search across wikis/manuals"
Fraud Detector	Detects **online fraud**	"fake account signups," "online payment fraud"
Personalize	**Recommendations** engine	"customers also bought"
Forecast	Time-series **predictions**	"predict inventory demand"
Lex	**Chatbots** (the tech behind Alexa)	"conversational bot"
SageMaker	Build/train/deploy **custom ML models**	"data scientists need to train models"

Service	Does	Signal
SageMaker Autopilot	AutoML: builds and tunes models automatically from your data	"build ML models **without ML expertise**"
Pinpoint	Multichannel **marketing/engagement** messages (email, SMS, push, voice), segmentation, analytics. An **event** = a user interaction with your app.	"targeted campaigns," "track user engagement"
Elastic Transcoder (legacy → **MediaConvert**)	Convert media files in S3 into playback formats	"convert videos for mobile devices"

THE trap: scanned invoice/form extraction → **Textract**, not Rekognition (images ≠ documents) and not Comprehend (that's meaning, not extraction).

Systems Manager (SSM) suite

One service, several exam-favorite features:

- **Session Manager** — shell access to instances with **no SSH keys, no bastion host, no open port 22**, all logged. Signal: "*secure instance access without opening inbound ports.*"
- **Run Command** — execute scripts **across a fleet** at once, no SSH.
- **Patch Manager** — **automated OS patching** on a schedule (maintenance windows).
- **Hybrid**: install the SSM agent on **on-premises servers** and manage them alongside EC2.
- **Automation** — runbooks for common maintenance tasks (restart, patch, create AMIs) across EC2 and other resources.
- **OpsCenter** — a central inbox of operational issues. Each issue is an **OpsItem**.
- **Fleet Manager** — a single UI to remotely manage nodes on AWS or on-prem.

Which agent?

Agent	Job
SSM Agent	Lets Systems Manager manage the instance: Run Command, Session Manager, Patch Manager (EC2, on-prem, VMs)
CloudWatch agent	Ships **memory/disk custom metrics and logs** to CloudWatch
Inspector	Vulnerability/CVE scanning. Inspector v2 uses the **SSM Agent**; the old standalone Inspector agent is legacy.

AWS Health dashboards:

- **Service Health Dashboard** shows **public** status: "is AWS down in this Region for everyone?"
- **Personal Health Dashboard**, now the **AWS Health Dashboard – your account**, shows events affecting **your** resources (e.g., scheduled maintenance on your EC2 instance).

Developer & app services

AWS Batch — run **long batch jobs** as Docker containers; schedules onto EC2/Spot for you. Signal: "*job runs for hours*" — the escape hatch from **Lambda's 15-minute limit**.

AppSync — managed **GraphQL** API. Signal words: "**GraphQL**," "**real-time subscriptions**," "**offline sync**" for mobile. Any of the three → AppSync.

Amplify — full-stack web/mobile app **quickstart**: hosting, auth, backend wired together. Signal: "*frontend team wants to build and deploy quickly without managing infrastructure.*"

SES (Simple Email Service) — send **application emails** at scale (receipts, notifications, marketing). Signal: "*application must send emails.*" (SNS emails are plain-text alerts for ops; SES is for real email.)

S3 Batch Operations — run one operation across **billions of EXISTING objects**: copy, tag, restore, or — the exam favorite — **"encrypt all existing objects in the bucket."** Default bucket encryption only covers *new* uploads; Batch Operations fixes history.

Aurora cloning (recap) — **copy-on-write clone** of a production Aurora DB in minutes, no full copy, cheap. Signal: "*test against production data quickly without impacting production.*"

X-Ray — **distributed tracing**: follow one request across microservices to find the slow/broken hop. Signal: "*identify performance bottlenecks across microservices.*" (CloudWatch = metrics/logs; X-Ray = the request's journey.)

App Runner — fully managed service that deploys a **containerized web app or API** from source code or a container image. It handles load balancing, scaling and TLS. Signal: "*deploy a web app from a container with no infrastructure/cluster management.*"

AppConfig (part of Systems Manager) — **safely roll out application configuration and feature flags**, with validation and gradual deployment plus automatic rollback. Signal: "*change feature flags without redeploying code.*"

Amazon Connect — cloud **contact center**. **Contact flows** define the caller experience: IVR menus, routing to queues or agents, callbacks.

OpsWorks (legacy — end of life 2024) — configuration management with **managed Chef and Puppet**: automates how servers are configured, deployed and managed across **EC2 and on-prem** machines. Signal: "*Chef*", "*Puppet*", "*recipes/cookbooks*" → OpsWorks. (Today the answer is usually SSM State Manager.)

The Code* toolchain — **CodeCommit** = managed **private Git repositories** (source control; closed to new customers in 2024 but still a valid answer). **CodeDeploy** = fully managed **deployment automation** to **EC2, Fargate, Lambda and on-prem servers**. Signal: "*automate software deployments across EC2 and on-prem*" → CodeDeploy. (For context, **User Acceptance Testing (UAT)** is the last phase of software testing — the final environment before production.)

AWS ParallelCluster — AWS-supported **open-source cluster management tool** to deploy and manage **HPC clusters**. Signal: "*deploy and manage an HPC cluster*" → ParallelCluster (pair with Cluster placement group + EFA from Section 3).

Odds and ends:

- **Device Farm**: test mobile and web apps on many real devices.
- **Lightsail**: simple VPS bundles for beginners.
- **Cloud Map**: service discovery by custom names.
- **App Mesh**: service mesh for microservices.
- **IoT Core**: connect devices securely to the cloud.

WorkDocs (legacy — AWS ended support in 2025) — managed document creation, editing and sharing. It was **not** for serving static or object content; that's S3.

AWS Artifact — self-service portal to download **AWS compliance reports** (SOC, PCI, ISO). Signal: "*auditors need proof of AWS's compliance.*" No agents, no config — just the download counter for paperwork.

Question patterns

> "*Existing on-prem app uses RabbitMQ with AMQP; migrate with minimal code changes*" → **Amazon MQ** (protocol keywords = MQ; SQS would mean rewriting)

> "*HTTPS on CloudFront with a custom domain — where's the certificate?*" → **ACM in us-east-1** (CloudFront only reads certs from N. Virginia)

> "*IPv6-only instances need outbound internet access but must block inbound*" → **Egress-only Internet Gateway** (the IPv6 answer; NAT gateway is IPv4)

> "*Automatically extract fields and tables from scanned invoices*" → **Textract** (forms/documents, beyond OCR — not Rekognition)

> "*Migrate a self-managed Apache Kafka cluster with no application changes*" → **MSK** (the word Kafka does all the work)

> "*Mobile app needs a GraphQL API with real-time data sync*" → **AppSync** (GraphQL/subscriptions/offline = AppSync every time)

> "*Automatically apply OS security patches to 500 EC2 instances monthly*" → **SSM Patch Manager** (fleet patching on a schedule)

> "*Encrypt all existing objects already stored in an S3 bucket*" → **S3 Batch Operations** (default encryption only covers new objects)

> "*Data team needs managed Apache Spark for nightly processing at low cost*" → **EMR with Spot task nodes** (Spark = EMR; Spot = cheap)

"*Regulations require workloads to run in the company's own datacenter using AWS services*" → **Outposts** (AWS racks on your floor — data residency)

"*Shell access to private instances without SSH keys or bastion hosts*" → **SSM Session Manager** (no port 22, fully audited)

"*Migrate Oracle to Aurora PostgreSQL*" → **DMS + SCT** (different engines = heterogeneous = add the Schema Conversion Tool)

"*Discover on-prem servers and their dependencies to plan a migration*" → **Application Discovery Service** (inventory first; track progress in Migration Hub)

"*Convert CSV in S3 to Parquet with the LEAST operational overhead*" → **Glue** (serverless ETL; EMR only if you need Spark/Hadoop control)

"*Detect fake account signups / online payment fraud*" → **Fraud Detector** (not GuardDuty, which detects AWS account threats)

"*Build ML models without ML expertise*" → **SageMaker Autopilot** (AutoML)

"*Roll out feature flags gradually with automatic rollback*" → **AppConfig** (not Parameter Store alone, which has no validated rollout)

"*Deploy a containerized web app with no infrastructure management*" → **App Runner** (simpler than ECS/EKS)

"*Be notified when AWS maintenance affects YOUR instances*" → **AWS Health Dashboard – your account** (+ EventBridge), not the public Service Health page

"*Stream a desktop app to users' browsers*" → **AppStream 2.0** (a full desktop → WorkSpaces; neither is a queue)

Pocket card

Keyword	Answer
MQTT / AMQP / JMS, minimal rewrite	Amazon MQ
Lift-and-shift servers (rehost)	MGN
DR replication, RPO seconds	Elastic Disaster Recovery (DRS)
DB migration, different engines	DMS + SCT
Free auto-renewing TLS certs	ACM
CloudFront certificate region	us-east-1 (always)
IPv6 outbound-only	Egress-only Internet Gateway
On-prem → VPC DNS	Resolver inbound endpoint
VPC → on-prem DNS	Resolver outbound endpoint
AWS racks in your datacenter	Outposts
Single-digit ms to a metro city	Local Zones
5G edge	Wavelength
Serverless ETL + catalog	Glue
Spark / Hadoop	EMR
Kafka	MSK
BI dashboards	QuickSight
Data lake + fine-grained access	Lake Formation
SaaS → S3	AppFlow
Images/video analysis	Rekognition
Speech → text / text → speech	Transcribe / Polly
Sentiment	Comprehend
Scanned forms/invoices	Textract
Intelligent doc search	Kendra
Recommendations / forecasts / chatbots	Personalize / Forecast / Lex
Custom ML	SageMaker
No-SSH access / fleet scripts / auto-patching	Session Manager / Run Command / Patch Manager
Batch jobs > 15 min, Docker	AWS Batch
GraphQL, offline sync	AppSync
Full-stack quickstart	Amplify
Application emails	SES
Bulk ops on existing S3 objects	S3 Batch Operations
Instant prod DB copy for testing	Aurora cloning
Distributed tracing	X-Ray

Keyword	Answer
Compliance reports for auditors	AWS Artifact
Inventory on-prem servers pre-migration	Application Discovery Service
Track migration progress	Migration Hub
Rehost servers (agent) vs move files	MGN vs DataSync
DMS Basic Schema Copy	Tables + PKs only → use SCT for prod
Serverless ETL / CSV→Parquet	Glue
Spark/Hadoop control	EMR
Batch jobs at scale	AWS Batch
Online fraud	Fraud Detector
AutoML	SageMaker Autopilot
Medical text NLP	Comprehend Medical
Marketing campaigns, user events	Pinpoint
Containerized web app, zero infra	App Runner
Chef / Puppet	OpsWorks (legacy)
Private Git repos / automated deploys to EC2 + on-prem	CodeCommit / CodeDeploy
Deploy & manage HPC clusters	AWS ParallelCluster
Feature flags / safe config rollout	AppConfig
Maintenance runbooks	SSM Automation
Ops issues inbox	SSM OpsCenter (OpsItems)
Memory/disk metrics	CloudWatch agent (not SSM Agent)
"Is AWS down?" vs "Is MY resource affected?"	Service Health vs AWS Health – your account
Stream one app vs full desktop	AppStream 2.0 vs WorkSpaces
Contact center / IVR	Amazon Connect
Third-party cert where ACM unsupported	IAM certificate store (legacy)
Bring your own drives to AWS site	Data Transfer Terminal
SWF / Data Pipeline / CloudEndure	Legacy → Step Functions / Glue / MGN

That's the last jar on the spice rack — you've now covered the whole SAA-C03 kitchen. Sleep well, read the last sentence first, and go collect your certificate.